

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	9
1 Обзор предметной области.....	11
1.1 Описание предметной области.....	11
1.2 Обзор аналогов.....	12
1.2.1 Сервис «Bumble».....	12
1.2.2 Сервис «Meetup».....	14
1.2.3 Сервис «Foursquare / Swarm».....	15
1.2.4 Сервис «Eventbrite».....	17
1.2.5 Сервис «Тусовка / Афиша».....	18
1.2.6 Сравнение аналогов.....	19
1.3 Проблематика предметной области.....	21
1.4 Общая постановка задачи.....	22
2 Расширенное техническое задание.....	23
2.1 Назначение системы.....	23
2.2 Цель создания системы.....	25
2.3 Описание системы.....	26
2.4 Требования к системе в целом.....	27
2.4.1 Публикация и обнаружение событий.....	27
2.4.2 Геолокационный функционал.....	28
2.4.3 Социальное взаимодействие и мэтчинг.....	28
2.4.4 Медиаконтент.....	29
2.4.5 Безопасность системы.....	29
2.5 Требования к составу пользовательских ролей.....	30
2.6 Функциональные требования.....	31
2.6.1 Требования к серверной части.....	31

					090207.193754.ПЗ							
Изм.	Лист	Фамилия	Подпись	Дата								
Разраб.		Солянов			Разработка мобильного приложения геолокационной социальной сети				Лит.	Лист	Листов	
Провер.		Кошкин									4	128
Реценз									Колледж ВятГУ			
Н. Контр.		Метелева										
Утверд.												

2.6.1.1	Архитектурное описание подсистемы.....	31
2.6.1.2	Компоненты подсистемы.....	32
2.6.1.2.1	API-шлюз.....	32
2.6.1.2.2	Сервис аутентификации.....	32
2.6.1.2.3	Сервис управления пользователями.....	33
2.6.1.2.4	Сервис событий.....	33
2.6.1.2.5	Сервис мэтчинга.....	34
2.6.1.2.6	Сервис загрузки файлов.....	34
2.6.1.2.7	Подсистема хранения данных.....	34
2.6.2	Требование к интерфейсу.....	35
2.6.2.1	Экраны приветствия и онбординга.....	35
2.6.2.2	Экран авторизации.....	36
2.6.2.3	Экран карты событий.....	36
2.6.2.4	Экран афиши событий.....	36
2.6.2.5	Экран детальной карточки события.....	37
2.6.2.6	Экран создания события.....	37
2.6.2.7	Экран мэтчинга.....	37
2.6.2.8	Экран профиля пользователя.....	38
2.6.2.9	Экран администратора.....	38
2.6.2.10	Экран модератора.....	38
2.7	Нефункциональные требования.....	39
2.7.1	Масштабируемость и производительность.....	39
2.7.2	Надёжность и отказоустойчивость.....	40
2.7.3	Безопасность.....	40
2.7.4	Показатели назначения.....	41
2.7.5	Требования к эргономике.....	41
2.7.6	Поддерживаемость и расширяемость.....	42
2.7.7	Совместимость.....	43
2.7.8	Логирование и мониторинг.....	43
2.8	Ожидаемые результаты.....	44

3	Проектирование программного продукта.....	46
3.1	Архитектура программного решения	46
3.2	Пользовательские сценарии.....	48
3.2.1	Регистрация, авторизация пользователя и разграничение прав доступа.....	48
3.2.2	Заполнение профиля и прохождение онбординга.....	49
3.2.3	Обновление геолокации пользователя.....	51
3.2.4	Публикация и модерация событий.....	52
3.2.5	Поиск и отображение событий.....	53
3.2.6	Участие в событии.....	54
3.2.7	Обработка мэтчей и генерация взаимного совпадения	54
3.2.8	Загрузка медиаконтента.....	55
3.2.9	Формирование ленты рекомендации в мэтчах.....	57
3.2.10	Расчет ключевых показателей.....	58
3.2.11	Просмотр активности через профиль.....	58
3.2.12	Модерация событий и обработка жалоб	59
3.2.13	Управление настройками приватности	60
3.2.14	Просмотр сводной статистики администратором.....	61
3.3	Передача и хранение данных.....	62
3.3.1	Потоки передачи данных.....	64
3.3.2	Логическая модель данных.....	67
3.4	Серверная часть.....	71
3.4.1	Контекстная диаграмма IDEF0	72
3.4.2	Декомпозиция IDEF0	74
3.5	Пользовательская часть.....	75
3.5.1	Интерфейс приложения	75
3.5.2	Потоки работы пользователя.....	76
3.5.2.1	Авторизация, регистрация, восстановление.....	76
3.5.2.2	Навигация по главному экрану.....	77
3.5.2.3	Карта событий.....	78

3.5.2.4 Афиша событий.....	79
3.5.2.5 Детали события и участие.....	80
3.5.2.6 Создание события.....	80
3.5.2.7 Мэтчи.....	81
3.5.2.8 Профиль и настройки.....	81
3.5.2.9 Панель модератора и администратора.....	82
3.5.3 Основные экраны и их функции.....	82
3.5.4 Ролевые особенности интерфейса.....	83
3.5.5 Безопасность и UX-детали.....	84
3.6 Вывод по проектированию программного продукта.....	84
4 Реализация программного продукта.....	86
4.1 Выбор и обоснование выбора средств реализации разработанной системы.....	86
4.1.1 Серверная часть.....	87
4.1.1.1 Система управления базами данных.....	89
4.1.1.2 Доступ к базе данных.....	89
4.1.1.3 Объектное хранилище.....	90
4.1.1.4 Аутентификация.....	91
4.1.2 Пользовательская часть.....	92
4.2 Реализация серверной части.....	93
4.2.1 Реализация целевой архитектуры.....	93
4.2.2 Структура базы данных.....	93
4.2.2.1 Таблица user (Схема “Users”).....	97
4.2.2.2 Таблица event (Схема “Events”).....	98
4.2.2.3 Таблица participant (Схема “events”).....	98
4.2.2.4 Таблица match (Схема “public”).....	99
4.2.2.5 Таблица Report.....	99
4.2.2.6 Вспомогательные таблицы.....	100
4.2.3 Сервисная архитектура и процессы.....	100
4.2.3.1 Сервис auth-service.....	102

4.2.3.2 Сервис users-service.....	102
4.2.3.3 Сервис events-service.....	103
4.2.3.4 Сервис match-service.....	104
4.2.3.5 Сервис upload-service.....	104
4.2.4 Подсистема маршрутизации.....	105
4.2.5 Безопасность серверной части.....	106
4.3 Реализация клиентского приложения пользователя.....	107
4.3.1 Общее описание.....	107
4.3.1.1 Экран приветствия.....	108
4.3.1.2 Экран авторизации.....	108
4.3.1.3 Экраны онбординга.....	111
4.3.1.4 Главный экран «Карта».....	111
4.3.1.5 Экран «Афиша».....	113
4.3.1.6 Экраны создания события.....	114
4.3.1.7 Экраны деталей события.....	115
4.3.1.8 Экран мэтчей.....	116
4.3.1.9 Экран профиля.....	117
4.3.1.10 Административная панель.....	119
4.4 Вывод по реализации программного продукта.....	123
ЗАКЛЮЧЕНИЕ.....	124
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	127
ПРИЛОЖЕНИЕ А (обязательное) Список сокращений.....	129
ПРИЛОЖЕНИЕ Б (обязательное) Листинг серверной части.....	131
ПРИЛОЖЕНИЕ В (обязательное) Листинг пользовательской части.....	156

ВВЕДЕНИЕ

В рамках настоящего дипломного проекта рассматривается задача создания мобильного приложения, которое соединяет людей через реальную городскую среду посредством обнаружения и публикации как спонтанных, так и плановых мероприятий с использованием геолокации и механизмов социального взаимодействия.

Актуальность темы обусловлена парадоксальной ситуацией современного городского жителя: находясь в плотной информационной среде, он часто оказывается изолированным от событий, происходящих прямо рядом с ним. Традиционные каналы (афиши, сайты анонсов, городские порталы) ориентированы преимущественно на плановые организованные мероприятия и практически не учитывают стихийную, живую жизнь города — уличные концерты, перформансы, флешмобы, неформальные встречи и субкультурные акции. При этом растёт запрос на социальные знакомства, основанные на реальной географической и событийной близости. Существующие решения (геолокационные приложения знакомств, событийные агрегаторы и социальные сети) фрагментированы и не образуют единого информационного пространства, где пользователи сами становятся источниками информации о происходящем вокруг в режиме реального времени.

Цель дипломного проекта — спроектировать и разработать мобильное приложение геолокационной социальной сети, которое позволяет пользователям находить и публиковать события в городе и на улице, включая неформальные и спонтанные активности, просматривать афишу мероприятий, участвовать в онлайн-событиях, а также взаимодействовать друг с другом через знакомства по географическому положению и по интересующим событиям..

Для достижения поставленной цели необходимо решить следующие задачи:

					090207.193754.ПЗ	Лист 9
Изм.	Лист	№ докум.	Подпись	Дата		

~ изучить предметную область, проанализировать существующие аналоги (Bumble, Meetup, Foursquare/Swarm, Eventbrite, отечественные афиши) и выявить незанятые рыночные ниши;

~ сформулировать проблематику и требования к системе;

~ спроектировать архитектуру приложения, включая геолокационные сервисы, механизмы публикации событий, систему матчинга и пользовательский интерфейс;

~ реализовать клиентскую и серверную части приложения с поддержкой краудсорсинга событий, реального времени и социальных функций;

~ обеспечить безопасность данных, удобство использования и масштабируемость решения.

Объектом исследования является процесс взаимодействия горожан с городской событийной средой и социальными связями на основе геолокации и общих интересов.

Предметом исследования выступают методы и программные средства создания геолокационных событийных социальных сетей, механизмы краудсорсинга пользовательского контента, геолокационного обнаружения событий и мэтчинга пользователей по месту и участию в мероприятиях.

Теоретическая значимость проекта заключается в систематизации подходов к объединению геолокационных сервисов, событийных платформ и социальных сетей знакомств в рамках единой системы, ориентированной на живую городскую среду.

Практическая значимость проекта состоит в создании мобильного приложения, которое позволит горожанам оперативно узнавать о происходящем рядом, публиковать спонтанные события, находить единомышленников по геолокации или участию в мероприятиях, а также объединит плановые и неформальные активности в одной платформе.

1 Обзор предметной области

1.1 Описание предметной области

Предметная область разрабатываемой системы связана с распространением информации о городских событиях и организацией взаимодействия пользователей на основе их местоположения. Использование геолокационных технологий позволяет привязывать контент и пользователей к конкретным координатам, формируя локальные информационные пространства.

Городская событийная среда включает как организованные мероприятия, так и спонтанные активности, информация о которых может поступать от различных участников. В связи с этим возрастает значение платформ, обеспечивающих сбор, отображение и распространение событийных данных в режиме реального времени, а также поддержку социальных взаимодействий между пользователями на основе общего местоположения и интересов.

Параллельно наблюдается устойчивый рост запроса на социальные знакомства, основанные на реальной близости и общих интересах. Существующие платформы знакомств и социальные сети имеют глобальный или региональный охват, но лишены привязки к конкретному месту и моменту. Тем не менее именно совместное присутствие на одном событии или в одном месте является одним из наиболее естественных оснований для знакомства.

Таким образом, предметная область настоящего дипломного проекта охватывает пересечение трёх направлений:

- геолокационные сервисы, обеспечивающие привязку контента к реальным координатам пользователя;
- событийные платформы, агрегирующие информацию о городских мероприятиях — как плановых, так и спонтанных;

– социальные сети с механизмами знакомств, основанными на географической и событийной близости.

Данное сочетание порождает принципиально новый класс приложений — геолокационные социальные сети событий, задача которых состоит в соединении людей через живую, реальную городскую среду, а не через виртуальные алгоритмические рекомендации.

Одной из особенностей предметной области является формирование событийного контента непосредственно пользователями. В отличие от традиционных афиш, современные цифровые платформы позволяют участникам самостоятельно публиковать информацию о событиях, дополняя её описаниями и мультимедийными материалами.

Наряду с пользовательским контентом используются официальные анонсы мероприятий, публикуемые организаторами. Интеграция событийной информации с социальными механизмами взаимодействия пользователей позволяет формировать связи между людьми на основе общего местоположения и участия в событиях. Таким образом, предметная область геолокационных событийных сервисов объединяет геолокационные технологии, пользовательский контент и социальные взаимодействия.

1.2 Обзор аналогов

1.2.1 Сервис «Bumble»

Bumble — это приложение для знакомств с элементами геолокации, позволяющее пользователям находить других людей в своём городе или в пределах заданного радиуса. Сервис приобрёл широкую популярность благодаря концепции, при которой при разнополом мэтче инициатива первого сообщения предоставляется женщине. На рисунке 1.1. представлена экранная форма сервиса «Bumble».

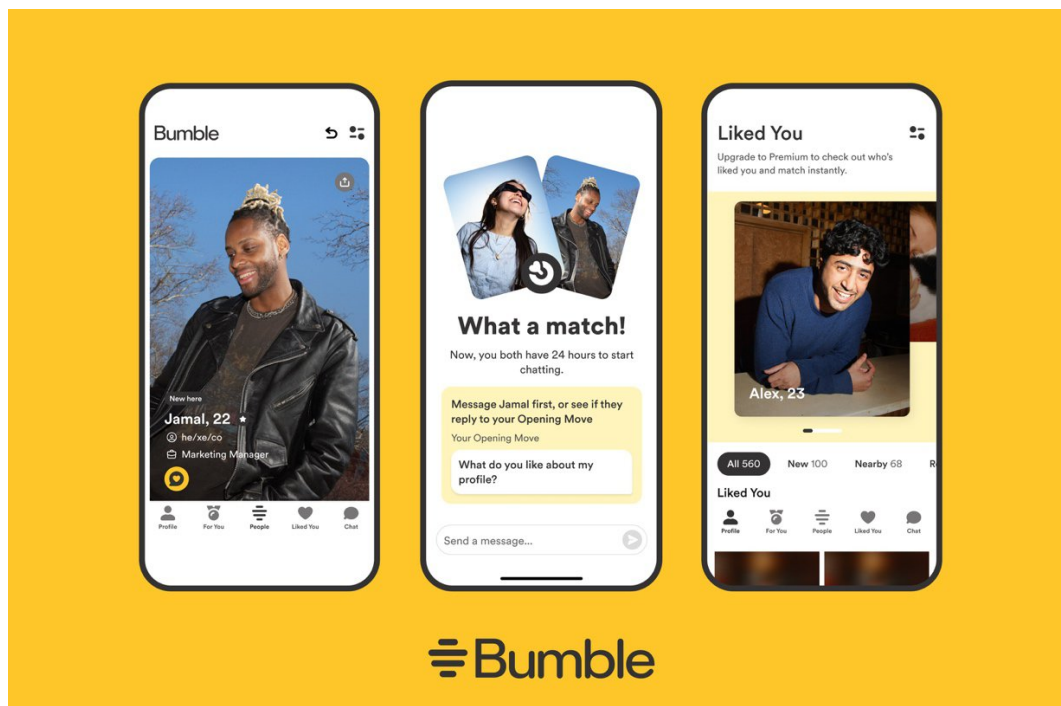


Рисунок 1.1 – Экранные формы сервиса «Bumble»

Сервис обладает следующим функционалом:

- геолокационный поиск пользователей в заданном радиусе;
- система мэтчинга на основе взаимной симпатии;
- разделы для романтических знакомств, дружеских связей и деловых контактов;
- функция ночного режима и временного расширения географии поиска.

Сервис имеет следующие особенности:

- высокая база пользователей в крупных городах;
- интуитивно понятный интерфейс;
- наличие верификации профилей;
- кроссплатформенность (iOS, Android).

К недостаткам сервиса относится отсутствие связи процесса знакомств с конкретными событиями и контекстом реальной городской активности. Взаимодействие пользователей строится преимущественно на основе

профилей и географической близости, что ограничивает возможность установления контактов в рамках общего присутствия на мероприятиях или в одном месте в определённый момент времени.

1.2.2 Сервис «Meetup»

Meetup — это глобальная платформа для организации и поиска тематических встреч и мероприятий. Сервис ориентирован на сообщества по интересам и предоставляет организаторам инструменты для анонсирования групповых встреч. На рисунке 1.2. представлена экранная форма «Meetup».

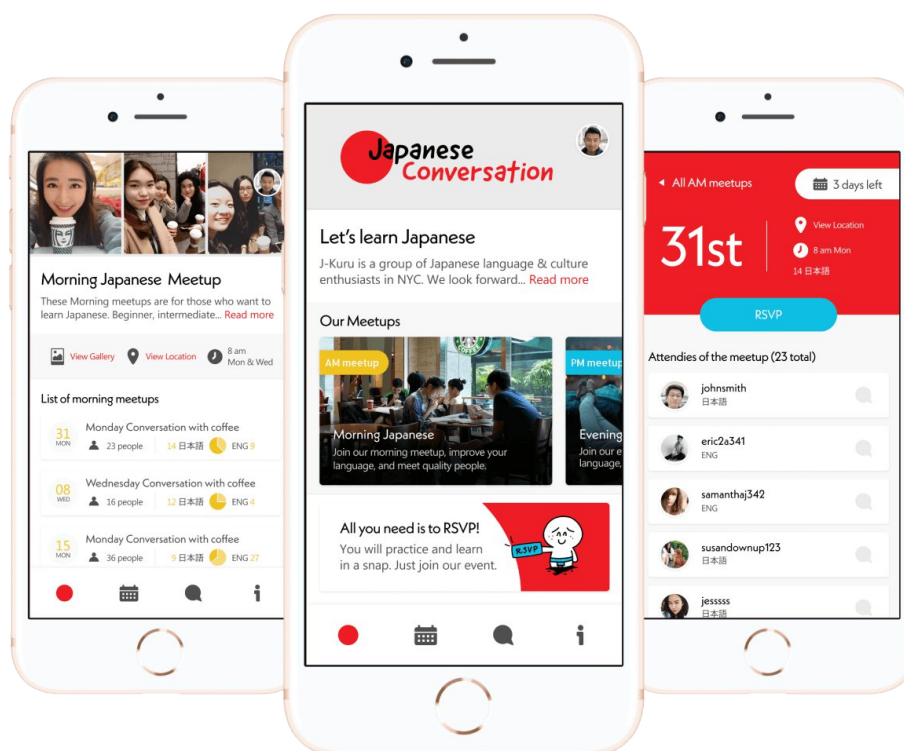


Рисунок 1.2 – Экранная форма сервиса «Meetup»

Сервис обладает следующим функционалом:

- создание и анонсирование плановых мероприятий с описанием, датой и местом;

- подписка на тематические сообщества по интересам;
- RSVP-система для отметки о своём участии в событии;
- фильтрация событий по категориям, дате и расстоянию;
- поддержка онлайн-форматов мероприятий.

Сервис имеет следующие особенности:

- большое количество профессиональных и образовательных сообществ;
- удобная система управления событиями для организаторов;
- интеграция с картами для отображения места проведения;
- поддержка платных мероприятий.

Ключевым недостатком является ориентация исключительно на организованные, плановые события. Платформа полностью игнорирует спонтанную городскую активность. Помимо этого, функционал социальных знакомств ограничен: сервис не предоставляет механизма матчинга между участниками, не поддерживает личный обмен сообщениями на уровне основного функционала и не интегрирует геолокацию для обнаружения событий в режиме реального времени.

1.2.3 Сервис «Foursquare / Swarm»

Foursquare и его дочернее приложение Swarm — это сервисы геолокационных чек-инов, позволяющие пользователям отмечать посещённые места и получать рекомендации на основе своей геоистории. Foursquare ориентирован на обнаружение заведений и мест, тогда как Swarm сосредоточен на социальной составляющей чек-инов. На рисунке 1.3. представлена экранная форма «Foursquare / Swarm».

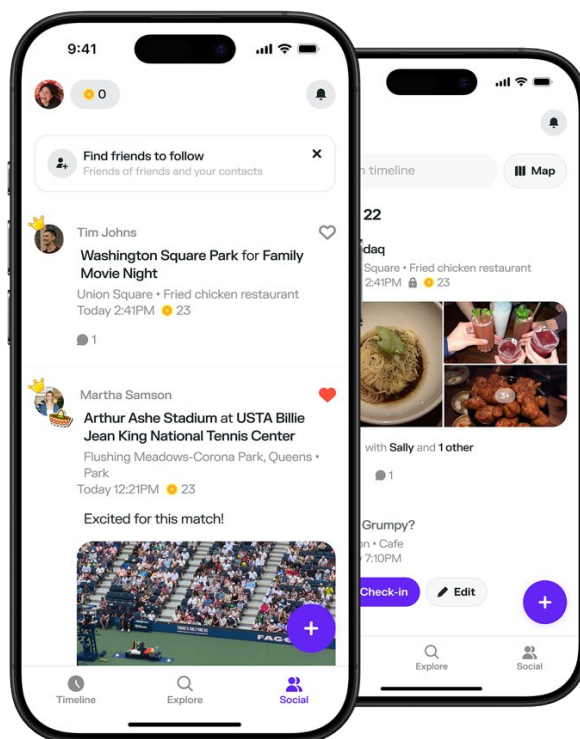


Рисунок 1.3 – Экранная форма сервиса «Foursquare / Swarm»

Сервис обладает следующим функционалом:

- чекины в заведениях и общественных местах;
- социальные связи и просмотр активности друзей на карте;
- рекомендации заведений на основе геоданных и предпочтений;
- система значков и рейтингов за активность.

Сервис имеет следующие особенности:

- богатая база данных мест с отзывами и фотографиями;
- продвинутые алгоритмы геолокационных рекомендаций;
- широкая аудитория в крупных городах.

Сервис ориентирован на места, а не на события. Он фиксирует, где находится человек, но не что происходит в этом месте прямо сейчас. Отсутствует механизм публикации спонтанных событий, нет мэтчинга пользователей и системы социальных знакомств. Приложение используется преимущественно для обнаружения заведений, а не для социальной коммуникации вокруг событий.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
16

1.2.4 Сервис «Eventbrite»

Eventbrite — это ведущая мировая платформа для организации и продажи билетов на мероприятия. Сервис охватывает широкий спектр событий: от корпоративных конференций до музыкальных фестивалей и спортивных соревнований. На рисунке 1.4. представлена экранная форма «Eventbrite».

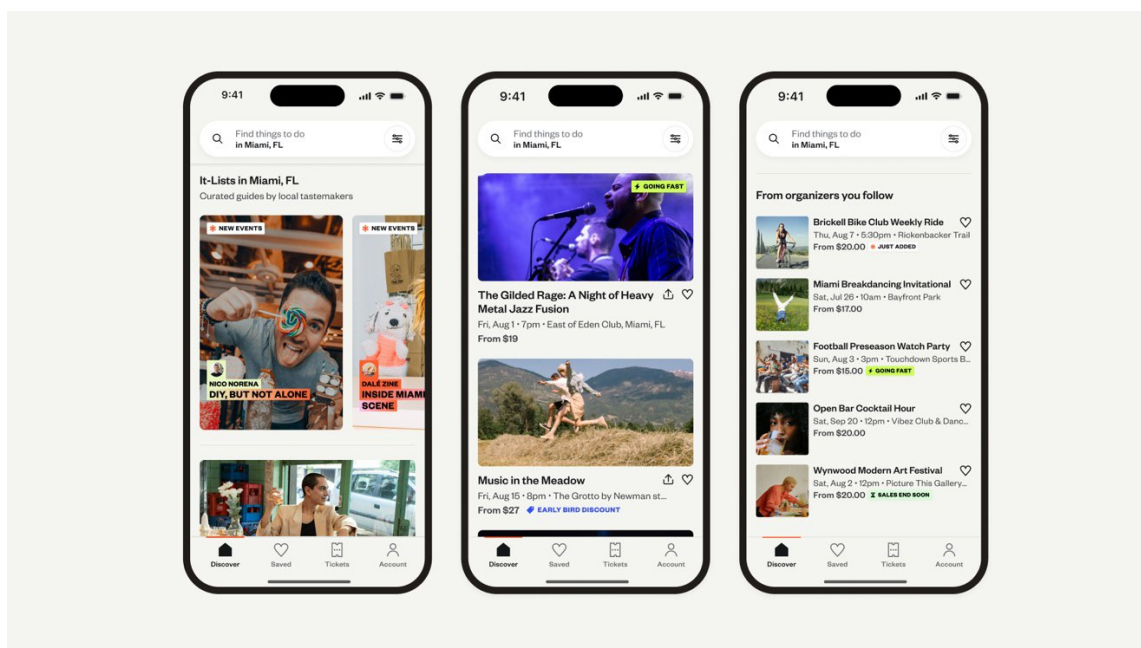


Рисунок 1.4 – Экранная форма сервиса «Eventbrite»

Сервис обладает следующим функционалом:

- создание событий с детальным описанием, расписанием и медиаматериалами;
- продажа и распространение билетов, включая бесплатные и платные форматы;
- поиск мероприятий по категории, дате, городу и цене;
- аналитика для организаторов (продажи, охват аудитории);
- поддержка онлайн- и гибридных форматов событий.

Сервис имеет следующие особенности:

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
17

- развитая инфраструктура для организаторов;
- интеграция с платёжными системами;
- наличие мобильных приложений для iOS и Android;
- широкий международный охват.

Сервис полностью ориентирован на коммерческие и организованные мероприятия. Отсутствует возможность публикации спонтанных событий рядовыми пользователями. Функционал социального взаимодействия между участниками крайне ограничен — платформа рассматривает пользователя как покупателя билета, а не как члена сообщества. Геолокация используется только для поиска, а не для открытия событий в режиме реального времени.

1.2.5 Сервис «Тусовка / Афиша»

Городские афиши, такие как «Афиша», KudaGo и их аналоги, представляют собой агрегаторы городских событий, ориентированные на российскую аудиторию. Они публикуют анонсы мероприятий в сфере культуры, развлечений и активного отдыха.

Сервисы обладают следующим функционалом:

- публикация анонсов концертов, выставок, спектаклей, кино и других мероприятий;
- фильтрация по городу, категории, дате и стоимости;
- редакционные подборки и рекомендации;
- продажа билетов через партнёрские сервисы.

Сервисы имеют следующие особенности:

- хорошая информированность о культурных событиях крупных городов;
- развитая редакционная составляющая;
- наличие мобильных приложений.

К недостаткам перечисленных сервисов можно отнести отсутствие пользовательских событий и спонтанной активности. Контент формируется редакцией или организаторами, рядовые пользователи не могут публиковать события. Отсутствует геолокационное обнаружение событий в реальном времени. Социальный функционал и механизм знакомств не предусмотрены.

1.2.6 Сравнение аналогов

На основании проведённого в подразделе 1.2 обзора существующих программных решений, была сформирована итоговая таблица, содержащая обобщённые сведения о функциональных возможностях, особенностях, преимуществах и недостатках каждого инструмента.

Таблица 1.1 – Сравнение аналогов

Название	Основные функции	Особенности	Плюсы	Минусы
Bumble	Геолокационный поиск пользователей, система матчинга, разделы для разных типов знакомств	Инициатива первого сообщения у женщин, ночной режим	Большая база пользователей, удобный интерфейс, верификация профилей	Отсутствует привязка к событиям и реальной городской активности
Meetup	Создание и поиск плановых мероприятий, подписка на сообщества, RSVP-система	Ориентация на тематические встречи и сообщества	Удобные инструменты для организаторов, интеграция с картами	Только плановые события, слабый функционал знакомств и матчинга
Foursquare / Swarm	Геолокационные чекины, рекомендации мест, социальная активность друзей	Чекины и значки, богатая база заведений	Хорошая геолокационная составляющая, рекомендации	Ориентация на места, а не на события; нет публикации спонтанных событий

Продолжение таблицы 1.1

Название	Основные функции	Особенности	Плюсы	Минусы
Eventbrite	Создание событий, продажа билетов, поиск мероприятий	Интеграция с платёжными системами, аналитика	Широкий охват, удобство для организаторов	Только коммерческие и плановые мероприятия, слабый социальный функционал
Тусовка / Афиша (KudaGo и др.)	Агрегация городских событий, редакционные подборки, продажа билетов	Хорошее покрытие культурных событий в РФ	Актуальная информация о плановых мероприятиях	Отсутствие пользовательских и спонтанных событий, нет матчинга
Разрабатываемое приложение	Публикация спонтанных и плановых событий, геолокационный поиск, матчинг пользователей	Краудсорсинг событий в реальном времени, матчинг по событию и геолокации	Полноценное объединение афиши, спонтанных событий и социального матчинга	—

Как следует из таблицы 1.1, ни один из рассмотренных аналогов не совмещает все необходимые компоненты: поддержку спонтанных событий через пользовательский контент, геолокационное обнаружение в реальном времени и систему мэтчинга по событиям.

Проведённый анализ показал, что существующие решения носят фрагментированный характер и не закрывают потребность горожан в единой платформе, объединяющей живую городскую активность и социальные знакомства на основе реальной близости.

Таким образом, создаваемая система занимает свободную нишу на рынке и обладает существенными конкурентными преимуществами.

1.3 Проблематика предметной области

На основании анализа предметной области и рассмотрения существующих аналогов был сформулирован комплекс проблем, решение которых определяет цель настоящей разработки.

Первая и наиболее очевидная проблема — информационная невидимость стихийных городских событий. Уличные концерты, перформансы, флешмобы и иные формы городской самодеятельности существуют только в момент своего проведения и практически не попадают в поле зрения горожан, находящихся в непосредственной близости. Традиционные афиши и анонсы не успевают фиксировать события, которые возникают без предварительного планирования.

Вторая проблема связана с разрывом между физическим присутствием в городе и осведомлённостью о происходящем. Человек, проходящий по улице, не имеет инструмента для получения агрегированной информации о том, что происходит в радиусе нескольких сотен метров прямо сейчас. Существующие геолокационные сервисы решают задачу поиска мест, но не событий.

Третья проблема касается механизмов социальных знакомств. Современные приложения для знакомств работают с абстрактными геолокационными радиусами, не связывая знакомство с конкретным местом или событием. При этом общее присутствие на одном мероприятии или в одном районе города является естественной точкой соприкосновения, которая могла бы служить органичным основанием для знакомства.

Четвёртая проблема — фрагментированность городского информационного пространства. Пользователь вынужден использовать несколько разрозненных приложений: одно для поиска плановых событий, другое для знакомств, третье для ориентирования в городе. Отсутствие единой платформы снижает как удобство использования, так и социальный эффект от взаимодействия пользователей.

Пятая проблема — заключается в недостаточной представленности пользовательского контента в системах распространения городской событийной информации. Существующие платформы преимущественно ориентированы на публикацию заранее организованных мероприятий, что затрудняет оперативное распространение сведений о событиях, возникающих непосредственно в городской среде и не имеющих официальных организаторов.

Перечисленные проблемы образуют взаимосвязанную систему, решение которой требует разработки нового класса мобильного приложения — геолокационной социальной сети, ориентированной на события.

1.4 Общая постановка задачи

В ходе анализа предметной области были изучены современные тенденции развития геолокационных сервисов, событийных платформ и социальных приложений для знакомств. Проведённый обзор показал, что каждое из рассмотренных направлений в отдельности имеет зрелые, хорошо развитые решения — однако их сочетание в рамках единой системы на сегодняшний день практически не представлено на рынке.

Анализ пяти аналогичных сервисов — Bumble, Meetup, Foursquare/Swarm, Eventbrite и отечественных городских афиш — подтвердил наличие значимой незанятой ниши. Ни один из рассмотренных сервисов не обеспечивает одновременно: обнаружение спонтанных событий через краудсорсинговый механизм, геолокационный мэтчинг пользователей по городу и по конкретному событию, а также полноценную событийную афишу с поддержкой как офлайн-, так и онлайн-мероприятий.

Выявленная проблематика подтверждает актуальность разработки: горожане не имеют инструмента, позволяющего увидеть живую, неформальную жизнь города и познакомиться с людьми, разделяющими их

интересы, опираясь на реальную городскую среду, а не на абстрактные алгоритмические рекомендации.

Таким образом, разработка мобильного приложения геолокационной социальной сети является обоснованной и актуальной задачей. Ключевыми отличительными характеристиками разрабатываемой системы, выделяющими её на фоне существующих решений, являются:

- краудсорсинговый механизм публикации спонтанных событий рядовыми пользователями;
- геолокационное обнаружение событий;
- система мэтчинга пользователей как по общей геолокации, так и по участию в конкретном событии;
- объединение в единой платформе афиши, спонтанных событий, онлайн-мероприятий и социального взаимодействия.

Таким образом, создаваемая система займёт свободную нишу на рынке мобильных приложений, сочетая в себе функции городского путеводителя, событийной афиши и социальной сети знакомств с геолокационной составляющей. Она будет отличаться высокой актуальностью контента, удобством использования и ориентацией на реальную городскую жизнь, что позволит значительно повысить вовлечённость пользователей и качество социальных связей в городской среде.

2 Расширенное техническое задание

					090207.193754.ПЗ	Лист 23
Изм.	Лист	№ докум.	Подпись	Дата		

2.1 Назначение системы

Разрабатываемый программный продукт предназначен для создания единой мобильной платформы, объединяющей функции обнаружения городских событий, событийной афиши и социального взаимодействия между пользователями.

Система ориентирована на следующие категории пользователей:

- рядовых горожан, заинтересованных в обнаружении уличных событий — концертов, перформансов, флешмобов и иных форм активности;
- организаторов мероприятий — для анонсирования событий и привлечения аудитории;
- пользователей, заинтересованных в знакомствах на основе географической близости или совместного посещения конкретных событий;
- авторов онлайн-трансляций и цифровых событий, желающих привлечь участников через городскую платформу.

Программный продукт должен предоставлять следующие инструменты:

- краудсорсинговую публикацию событий — авторизованный пользователь может зафиксировать происходящее вокруг него и отметить событие на карте с описанием, фото и временными метками;
- геолокационную ленту событий — отображение всех актуальных событий в заданном радиусе от текущего положения пользователя;
- официальную афишу города — агрегатор мероприятий, публикуемых организаторами;
- механизм мэтчинга — подбор знакомств среди пользователей;
- поддержку онлайн-событий — формат цифровых мероприятий с соответствующей меткой и возможностью присоединения без географической привязки;

2.2 Цель создания системы

					090207.193754.ПЗ	Лист 24
Изм.	Лист	№ докум.	Подпись	Дата		

Целью разработки системы является создание мобильного приложения геолокационной социальной сети, обеспечивающего пользователям возможность в режиме реального времени обнаруживать стихийные и плановые городские события, публиковать собственный контент о происходящем вокруг них, а также знакомиться с людьми через общую городскую среду, опираясь на геолокационную близость или совместное участие в конкретных событиях.

Система должна обеспечить:

- краудсорсинговую публикацию и обнаружение событий в режиме реального времени посредством геолокационных механизмов с пространственной индексацией;
- формирование коллективной, актуальной карты городской активности, доступной всем пользователям в радиусе их интереса;
- работу официальной событийной афиши для плановых мероприятий с поддержкой офлайн- и онлайн-форматов;
- алгоритмический подбор потенциальных знакомств на основе геолокации пользователя и его участия в событиях;
- безопасное хранение и предоставление медиаконтента пользователей;
- надёжную аутентификацию пользователей с делегированием управления учётными данными провайдеру Firebase Authentication;

Основными задачами, которые необходимо решить для достижения указанной цели, являются:

- разработка серверной части системы на основе микросервисной архитектуры с чётким разделением функциональности между независимыми компонентами;
- реализация механизмов геолокационного поиска и обработки пространственных данных для отображения пользователей и событий с учётом их местоположения;

- разработка подсистемы социального взаимодействия пользователей, обеспечивающей формирование рекомендаций и взаимных совпадений на основе действий пользователей;
- реализация подсистемы загрузки, хранения и обработки медиаконтента, используемого в профилях пользователей и карточках событий;
- разработка мобильного клиентского приложения с современным пользовательским интерфейсом, обеспечивающего удобное взаимодействие со всеми функциями системы;
- организация единой среды развёртывания и эксплуатации программного комплекса, обеспечивающей надёжность, масштабируемость и удобство сопровождения системы.

2.3 Описание системы

Разрабатываемая система должна иметь клиент-серверную архитектуру и состоит из двух основных частей: серверной и клиентской (мобильного приложения).

Серверная часть построена на принципах микросервисной архитектуры и включает пять независимых сервисов. Каждый сервис выполняет строго определённую бизнес-функцию, обладает изолированной базой данных и может разрабатываться, масштабироваться и обновляться независимо от остальных компонентов системы. Взаимодействие между микросервисами и внешними клиентами осуществляется через единый API-шлюз, что обеспечивает централизованное управление запросами, балансировку нагрузки и повышение общей безопасности системы.

Клиентская часть представлена кроссплатформенным мобильным приложением, разработанным с использованием единой кодовой базы для платформ iOS и Android. Архитектура приложения построена с соблюдением

принципов разделения ответственности, что позволяет чётко отделить бизнес-логику, управление состоянием и слой представления. Приложение обеспечивает удобный и интуитивно понятный пользовательский интерфейс для взаимодействия со всеми функциями системы, включая геолокационные сервисы, социальное взаимодействие и работу с медиаконтентом.

Такая архитектура системы в целом направлена на обеспечение высокой масштабируемости, надёжности, удобства сопровождения и возможности дальнейшего расширения функциональности.

2.4 Требования к системе в целом

2.4.1 Публикация и обнаружение событий

Система должна обеспечивать:

- возможность публикации события рядовым пользователем с указанием названия, описания, координат, даты и времени, типа события (офлайн / онлайн) и прикреплением медиаматериалов;
- немедленное отображение опубликованного события на карте и в геолокационной ленте других пользователей, находящихся в пределах заданного радиуса;
- поддержку официальной афиши — публикацию плановых событий организаторами с расширенным описанием и временными метками;
- поддержку онлайн-событий с соответствующей меткой и возможностью участия без географической привязки;
- геолокационный поиск событий на основе текущих координат пользователя с фильтрацией по расстоянию, типу, дате и категории;
- возможность участия в событии (чек-ин) с фиксацией пользователя как участника и просмотром списка участников.

2.4.2 Геолокационный функционал

Система должна обеспечивать:

- получение и обновление координат пользователя в режиме реального времени с немедленной передачей на сервер и сохранением в базе данных;
- отображение событий и пользователей на интерактивной карте Yandex MapKit с возможностью перехода к детальной карточке объекта;
- корректное вычисление расстояний с учётом кривизны земной поверхности посредством расширения PostGIS и типа geography(Point, 4326);
- поддержку настраиваемого радиуса поиска событий и пользователей — от нескольких сотен метров до 50 км;
- автоматическое исключение пользователей с отключённой видимостью местоположения из геолокационных выборок на уровне SQL-запросов.

2.4.3 Социальное взаимодействие и мэтчинг

Система должна обеспечивать:

- отображение ленты потенциальных знакомств на основе текущей геолокации пользователя — мэтчинг по городу — с фильтрацией по возрасту, полу и максимальному расстоянию;
- отображение участников конкретного события в качестве потенциальных знакомств — мэтчинг по событию;
- механизм свайпов: действия лайк и дизлайк в отношении профилей других пользователей;
- автоматическую генерацию взаимного совпадения при взаимном лайке двух пользователей;

– отображение списка актуальных взаимных совпадений пользователя.

2.4.4 Медиаконтент

Система должна обеспечивать:

- загрузку фотографий профиля и медиаматериалов событий с валидацией формата и размера файла;
- потоковую передачу файлов напрямую в объектное хранилище MinIO S3 без промежуточной буферизации на сервере;
- формирование и возврат публичных URL для доступа к загруженным медиафайлам;
- изолированное хранение медиаконтента по бакетам.

2.4.5 Безопасность системы

Система должна обеспечивать:

- криптографическую аутентификацию: проверку подлинности пользователей на стороне сервера путем верификации цифровой подписи токенов доступа с помощью публичных ключей доверенного поставщика идентичности;
- безопасность запросов: передачу авторизационного токена в заголовке Authorization: Bearer при каждом обращении к защищенным ресурсам;
- ролевую модель доступа: разграничение прав для ролей «Пользователь», «Модератор» и «Администратор» на уровне декларативных правил (аннотаций) программного кода;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
29

- защиту конфигурационных данных: хранение секретов (API-ключей и параметров интеграции) исключительно в переменных среды окружения, изолированных от системы контроля версий;
- конфиденциальность при поиске: автоматическое исключение скрытых профилей из поисковых выборок на уровне СУБД с помощью индексируемого условия видимости.

2.5 Требования к составу пользовательских ролей

Система должна поддерживать многопользовательский режим с разграничением прав доступа на основе ролевой модели. В системе определены следующие роли:

Пользователь (USER) — базовая роль, присваиваемая всем зарегистрированным аккаунтам. Имеет право:

- создавать и редактировать собственный профиль, публиковать события и медиаматериалы;
- просматривать геолокационную ленту событий и карту в заданном радиусе;
- выполнять свайпы, просматривать взаимные совпадения и управлять настройками приватности;
- участвовать в событиях (чек-ин) и просматривать их участников.

Модератор (MODERATOR) — расширенная роль с правами на:

- просмотр всех событий и профилей пользователей вне зависимости от настроек приватности;
- редактирование и удаление пользовательских событий, нарушающих правила платформы;
- временную блокировку пользователей без права изменения ролевой модели.

Администратор (ADMIN) — роль с полным доступом к системе. Дополнительно имеет право:

- управлять ролями пользователей и изменять их права;
- осуществлять полное удаление пользователей и их контента из базы данных;
- просматривать системные метрики и управлять параметрами развёртывания.

Каждому пользователю предоставляется персонализированный интерфейс мобильного приложения, отображающий инструменты и данные, соответствующие его роли.

2.6 Функциональные требования

2.6.1 Требования к серверной части

Серверная часть системы построена по микросервисной архитектуре. Каждый сервис выполняет чётко определённую функцию и может разрабатываться, масштабироваться и обновляться независимо. Взаимодействие всех клиентских запросов с сервисами осуществляется исключительно через единую точку входа — API-шлюз Traefik.

2.6.1.1 Архитектурное описание подсистемы

Серверная часть включает следующие основные компоненты:

- API-шлюз — центральная точка входа, обрабатывает все входящие HTTP-запросы, выполняет проксирование и маршрутизацию на соответствующие сервисы по правилам;
- сервисный слой — содержит бизнес-логику работы с профилями пользователей, событиями и мэтчингом;

- подсистема хранения данных — единая физическая база данных PostgreSQL 16 с расширением PostGIS, разделённая на логические схемы по принципу Single Database, Multiple Schemas;
- объектное хранилище — полностью совместимое с Amazon S3 API, для хранения медиаконтента пользователей и событий;

2.6.1.2 Компоненты подсистемы

2.6.1.2.1 API-шлюз

API-шлюз должен обеспечивать:

- приём всех входящих HTTP-запросов от мобильного клиента;
- прозрачную маршрутизацию запросов на соответствующие микросервисы согласно пути запроса;
- автоматическое обнаружение сервисов и конфигурацию маршрутов через метки контейнеров без ручного редактирования файлов конфигурации;
- работу в режиме reverse проху с поддержкой балансировки нагрузки при горизонтальном масштабировании сервисов.

2.6.1.2.2 Сервис аутентификации

Сервис должен обеспечивать:

- верификацию посредством скачивания и кэширования публичных ключей JWKS от Google;
- извлечение uid пользователя из верифицированного токена и резолвинг соответствующего внутреннего идентификатора из базы данных;
- функционирование в режиме stateless — без хранения состояния сессий на сервере;

- блокировку запросов с недействительным, истёкшим или отсутствующим токеном с возвратом статуса HTTP 401.

2.6.1.2.3 Сервис управления пользователями

Сервис должен обеспечивать:

- создание профиля пользователя при первой авторизации с привязкой к Firebase UID;
- чтение и обновление полей профиля;
- обновление координат пользователя;
- формирование алгоритмической ленты потенциальных знакомств;
- управление настройками приватности;

2.6.1.2.4 Сервис событий

Сервис должен обеспечивать:

- создание нового события;
- геолокационный поиск событий на основе координат запроса с использованием пространственных функций — выборку событий в заданном радиусе с сортировкой по расстоянию;
- обработку чек-инов;
- предоставление списка участников события для механизма мэтчинга по событию;
- поддержку фильтрации событий по типу, категории, дате и расстоянию.

2.6.1.2.5 Сервис мэтчинга

Сервис должен обеспечивать:

- приём и обработку действий лайк и дизлайк от пользователя с указанием идентификатора целевого пользователя;
- проверку наличия встречного лайка от целевого пользователя и генерацию сущности при двустороннем совпадении;
- предоставление списка активных взаимных совпадений пользователя.

2.6.1.2.6 Сервис загрузки файлов

Сервис должен обеспечивать:

- приём мультиформатных объектов с валидацией типа и размера файла;
- потоковую передачу файлов напрямую в хранилище без промежуточной буферизации в памяти сервера;
- маршрутизацию файлов в соответствующий бакет на основе параметра запроса;
- формирование и возврат публичного URL загруженного файла для сохранения в профиле пользователя или карточке события.

2.6.1.2.7 Подсистема хранения данных

Подсистема хранения данных должна обеспечивать:

- хранение структурированных данных в единой физической базе PostgreSQL 16 с расширением PostGIS;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
34

- пространственную индексацию координат пользователей и событий с применением типа geography(Point, 4326) для корректного расчёта расстояний в метрах с учётом кривизны земной поверхности;
- составной гео-индекс с фильтрацией по статусу готовности и видимости профиля пользователя, оптимизирующий высоконагруженную выборку данных для рекомендательной ленты.
- версионирование схем и автоматическое применение миграций при старте каждого сервиса;
- хранение медиаконтента в объектном хранилище с разделением по бакетам.

2.6.2 Требование к интерфейсу

Пользовательский интерфейс мобильного приложения геолокационной социальной сети должен быть организован посредством нижней панели навигации, обеспечивающей быстрый доступ к четырём основным разделам приложения: карте событий, ленте событий, мэтчингу и профилю пользователя.

2.6.2.1 Экраны приветствия и онбординга

Экраны приветствия и онбординга должны обеспечивать:

- пошаговое знакомство с ключевыми функциями приложения — обнаружение событий, знакомства, публикация контента;
- переход к экрану регистрации или авторизации;
- последовательное заполнение профиля в процессе онбординга: загрузка фотографии, ввод имени, возраста, пола и интересов;
- переход на главный экран приложения после успешного завершения онбординга.

2.6.2.2 Экран авторизации

Экран авторизации должен обеспечивать:

- вход по электронной почте и паролю;
- вход через поддерживаемые социальные сети (Google);
- восстановление пароля;
- автоматический вход при наличии действующего токена в хранилище устройства.

2.6.2.3 Экран карты событий

Экран карты событий должен отображать:

- интерактивную карту Яндекса с маркерами событий, актуальных в радиусе от текущего положения пользователя;
- виджет с событиями рядом(5км);
- переход к детальной карточке события при нажатии на маркер или на виджет;
- поисковую строку для поиска событий;
- интерактивные кнопки отследить себя, приближение, отдаление.

2.6.2.4 Экран афиши событий

Экран ленты событий должен выглядеть в виде вертикального списка карточек. Должен отображать:

- список актуальных событий;
- карточку каждого события, включающую: название, фотографию, категорию, количество участников, рейтинг события, месту;
- фильтры по типу, категории, дате, городу и максимальному расстоянию.

2.6.2.5 Экран детальной карточки события

Экран детальной карточки события открывается при нажатии на маркер карты или карточку ленты. Должен отображать и обеспечивать:

- полную информацию о событии: название, описание, категорию, адрес, дату и время, медиаматериалы;
- список участников события и возможностью перехода к мэтчингу по событию;
- кнопку чек-ина — участия в событии;
- кнопку для добавления события в избранное;
- кнопку для жалобы на событие.

2.6.2.6 Экран создания события

Экран создания события должен обеспечивать:

- ввод названия, описания и категории события;
- загрузку фотографий;
- кнопку публикации;
- кнопку для сохранения черновика и выбора черновика;
- кнопку для выбора формата события(онлайн/оффлайн)

2.6.2.7 Экран мэтчинга

Экран мэтчинга должен обеспечивать:

- карточку профиля, содержащую фотографию, имя, возраст, описание, общие интересы;
- действия свайпа: лайк (свайп вправо), дизлайк (свайп влево), инфо (свайп вниз), еще подумаю (свайп вверх)

2.6.2.8 Экран профиля пользователя

Экран профиля предоставляет доступ к личным данным и настройкам аккаунта. Должен отображать и обеспечивать:

- просмотр и редактирование основных данных профиля: фотография, имя, возраст, пол, описание, интересы, ссылки на социальные сети;
- специальные поля с показом событий по категориям (созданные мной, я иду, мне понравились);
- информацию по мэтчам, в которой отображается вся статистика по мэтчам;
- управление настройками приватности: показ посещенных мероприятий, показ профиля в мэтчах, инкогнито режим, статус в сети;
- в зависимости от роли пользователя показывать кнопку панель администратора/модератора;
- выход из аккаунта с очисткой токена авторизации.

2.6.2.9 Экран администратора

Экран должен обеспечивать:

- возможность управления пользователями(роль, назначение санкций, просмотр жалоб);
- возможность просмотра текущего состояния по событиям и назначение им санкций;
- возможность просмотра жалоб на пользователей и на события;
- возможность просмотра журнала прошлых действий;

2.6.2.10 Экран модератора

Экран для модератора должен показывать:

					090207.193754.ПЗ	Лист 38
Изм.	Лист	№ докум.	Подпись	Дата		

- возможность просмотра текущего состояния по событиям и назначение им санкций;
- возможность просмотра жалоб на пользователей и на события;

2.7 Нефункциональные требования

2.7.1 Масштабируемость и производительность

Система должна обеспечивать эффективную обработку одновременных запросов большого количества пользователей и сохранять высокую производительность при росте нагрузки. Для этого должны быть реализованы следующие требования:

- микросервисная архитектура должна поддерживать независимое горизонтальное масштабирование отдельных компонентов без остановки работы платформы;
- время отклика API на основные запросы не должно превышать 500 мс при нормальной нагрузке;
- геолокационные запросы с использованием пространственной индексации должны выполняться не более чем за 1 секунду при выборке данных в радиусе до 50 км;
- сервис подбора совпадений должен обеспечивать высокую конкурентную производительность при обработке интенсивного потока взаимодействий пользователей;
- загрузка медиафайлов должна осуществляться в потоковом режиме без полной буферизации на сервере, что позволит минимизировать потребление оперативной памяти.

2.7.2 Надёжность и отказоустойчивость

Сбой одного из микросервисов не должен приводить к остановке работы всей платформы. Для обеспечения высокой надёжности и отказоустойчивости системы должны быть реализованы следующие требования:

- все компоненты системы должны быть контейнеризованы и поддерживать автоматический перезапуск при возникновении сбоев;
- миграции схем базы данных должны применяться автоматически при запуске соответствующих сервисов без необходимости ручного вмешательства;
- каждый сервис должен самостоятельно управлять миграциями своей части схемы базы данных, обеспечивая изоляцию изменений между компонентами;
- в среде развёртывания должен быть предусмотрен централизованный механизм управления сборкой, запуском и обновлением компонентов системы из единой точки входа.

2.7.3 Безопасность

Система работает с персональными данными пользователей, включая геолокационную информацию и медиаконтент, в связи с чем должны быть обеспечены следующие требования безопасности:

- аутентификация пользователей должна осуществляться с использованием защищённых токенов, верификация которых выполняется криптографическими методами без хранения секретов в исходном коде;
- все чувствительные конфигурационные данные (API-ключи, учётные данные сервисов и т.п.) должны храниться в файлах окружения, исключённых из системы контроля версий;

- доступ к административным и модераторским функциям должен быть строго ограничен на основе ролевой модели доступа (RBAC);
- пользователи, скрывшие свой профиль или геолокацию, должны автоматически исключаться из поисковых выборок и рекомендаций на уровне базы данных путём соответствующих условий в запросах.

2.7.4 Показатели назначения

Пользователь ожидает высокого быстродействия и оперативного отклика системы, особенно при работе с геолокационными функциями в режиме реального времени. Для этого должны быть обеспечены следующие требования:

- обновление координат пользователя на сервере должно выполняться с периодичностью в 30 секунд при активном использовании приложения, а так же при перезапуске приложения;
- среднее время загрузки карты с маркерами событий не должно превышать 2 секунд при количестве объектов до 1000 и стабильном интернет соединении;

2.7.5 Требования к эргономике

Приложение ориентировано на широкую аудиторию горожан, не обладающих специальными техническими знаниями. В связи с этим должны быть реализованы следующие требования к пользовательскому интерфейсу:

- процесс первоначальной настройки (онбординг) должен быть максимально простым и завершаться не более чем за пять шагов;
- основные сценарии использования, такие как публикация нового события, должны быть доступны не более чем за три нажатия от главного экрана;

- все интерактивные элементы должны иметь чёткую визуальную обратную связь при взаимодействии;
- карточки событий и пользовательских профилей должны содержать всю ключевую информацию без необходимости дополнительных переходов на другие экраны.

2.7.6 Поддерживаемость и расширяемость

Код системы должен обеспечивать возможность расширения функциональности за счёт добавления новых модулей и типов данных без изменения существующего бизнес-кода. Для этого архитектура системы должна основываться на принципах модульности и разделения ответственности, обеспечивая слабую связанность компонентов и независимость их развития

- архитектура клиентской части должна обеспечивать чёткое разделение бизнес-логики и пользовательского интерфейса, упрощая тестирование и добавление новых экранов;
- слои абстракции клиентского приложения должны быть изолированы друг от друга в соответствии с принципами чистой архитектуры;
- каждый микросервис должен иметь чётко определённую зону ответственности, что позволит добавлять новые сервисы без модификации существующих компонентов;
- структура хранения данных должна обеспечивать возможность дальнейшего масштабирования и миграции к более изолированным схемам хранения при необходимости.

2.7.7 Совместимость

Система должна обеспечивать корректную работу на мобильных устройствах под управлением операционных систем iOS и Android, поддерживающих актуальные версии платформ, используемые основной долей пользователей мобильных устройств.. Для этого должны быть реализованы следующие требования:

- мобильное приложение должно обеспечивать поддержку основных мобильных платформ (iOS и Android) с минимальными версиями, охватывающими большинство актуальных устройств, из единой кодовой базы;
- HTTP-взаимодействие клиентской части с сервером должно осуществляться через стандартные средства с дополнительными обёртками для логирования, без использования сторонних HTTP-клиентов;
- формат всех ответов API должен соответствовать подходам REST, обеспечивающей единообразную обработку ошибок и данных на стороне клиента.

2.7.8 Логирование и мониторинг

Для оперативного обнаружения и устранения проблем система должна вести подробное и структурированное логирование. В связи с этим должны быть реализованы следующие требования:

- все компоненты системы должны выполнять структурированное логирование ключевых событий с указанием уровня логирования (INFO, WARN, ERROR) и контекста выполняемого запроса;
- секреты, ключи и персональные данные пользователей должны быть исключены из логов;

2.8 Ожидаемые результаты

После ввода системы в эксплуатацию пользователи получат возможность находить интересующих их людей и мероприятия с учётом текущего местоположения. Система обеспечит отображение актуальной информации о событиях и участниках, находящихся поблизости, что позволит повысить уровень вовлечённости пользователей в общественную и культурную жизнь города.

Параллельно с этим будет реализован механизм публикации и распространения информации о мероприятиях. Организаторы смогут создавать и редактировать события, а пользователи — получать сведения о доступных активностях в удобной форме. Это позволит расширить аудиторию мероприятий и упростить процесс взаимодействия между организаторами и посетителями.

Одним из ключевых результатов станет создание среды для социального взаимодействия пользователей. Система предоставит инструменты поиска новых знакомств на основе общих интересов и территориальной близости, а также механизмы установления взаимных контактов между участниками платформы. Такой подход позволит объединить возможности событийного сервиса и социальной сети в рамках единого решения.

Для обеспечения надёжной работы будет реализован комплекс серверных и клиентских компонентов, обеспечивающих обработку пользовательских запросов, хранение данных, управление учётными записями и взаимодействие между участниками системы. Архитектура решения будет предусматривать возможность дальнейшего развития функциональности и увеличения числа пользователей без существенных изменений структуры системы.

Особое внимание будет уделено вопросам безопасности и конфиденциальности. Система обеспечит контроль доступа к данным, защиту

пользовательской информации и возможность настройки параметров приватности. Пользователи смогут самостоятельно определять уровень доступности своих данных для других участников платформы.

Таким образом, результатом выполнения проекта станет создание программной системы, предназначенной для организации социального взаимодействия пользователей и поиска мероприятий с использованием геолокационных технологий. Внедрение системы позволит повысить доступность информации о городской активности и создать удобный инструмент для общения, знакомства и участия в общественной жизни.

					090207.193754.ПЗ	Лист 45
Изм.	Лист	№ докум.	Подпись	Дата		

3 Проектирование программного продукта

3.1 Архитектура программного решения

В рамках реализации программного продукта была выбрана микросервисная архитектура со слабо связанными сервисами и общей инфраструктурой хранения данных. Такое архитектурное решение обеспечивает высокую масштабируемость, отказоустойчивость, удобство параллельной разработки и последующего сопровождения разрабатываемой геосоциальной платформы. Схема деления структурная показана на рисунке 3.1

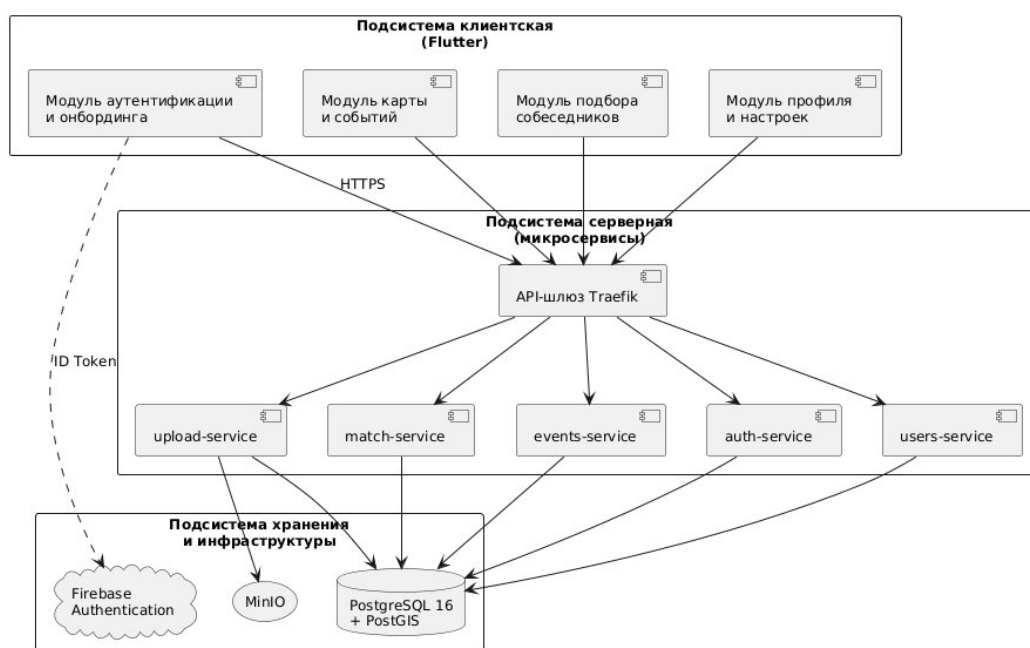


Рисунок 3.1 – Схема деления структурная

Система разделена на несколько независимых слоёв и компонентов, каждый из которых выполняет строго определённые функции и имеет чётко очерченную зону ответственности:

- клиентский слой — кроссплатформенное мобильное приложение, обеспечивающее интерактивное взаимодействие пользователей с системой на

целевых мобильных операционных системах из единой кодовой базы. Приложение архитектурно изолировано от бизнес-логики обработки данных, отвечая исключительно за презентационный уровень и формирование запросов к серверу;

- API-шлюз (единая точка входа) — узел маршрутизации на базе обратного прокси-сервера. Принимает все входящие сетевые запросы и выполняет их прозрачную динамическую маршрутизацию к целевым микросервисам на основе заданных конфигурационных правил;

- сервисный слой базовой бизнес-логики — группа микросервисов, реализующих основные функции платформы: криптографическую верификацию токенов внешнего провайдера аутентификации, управление профилями, применение ролевой модели доступа (RBAC) и выполнение сложных пространственных геолокационных выборок;

- сервисный слой высоконагруженных операций — выделенная группа микросервисов, спроектированных специально для задач с интенсивным конкурентным вводом-выводом. Применение встроенных механизмов асинхронной обработки и легковесных потоков позволяет эффективно обслуживать потоковую загрузку медиафайлов и тысячи одновременных запросов пользовательских действий (свайпов) без деградации общей производительности;

- слой реляционного хранения данных — единая физическая база данных, логически разделённая по паттерну «Single Database, Multiple Schemas» для изоляции данных отдельных сервисов. Для метрически точного расчёта расстояний по геосфере Земли интегрировано специализированное пространственное (геоинформационное) расширение. Версионирование структуры базы данных автоматизировано с помощью инструментов миграции;

- объектное хранилище — выделенный сервис для безопасного изолированного хранения неструктурированного пользовательского

медиаконтента (аватары, фотографии событий) с выдачей публичных ссылок. Вынесение бинарных данных в отдельное хранилище радикально снижает нагрузку на основную реляционную базу;

– слой кэширования — хранилище данных в оперативной памяти, применяемое для управления временными данными, оптимизации ресурсоёмких запросов и снижения задержек при формировании персональных рекомендаций.

Особое внимание при проектировании архитектуры уделено строгому разделению ответственности и принципу технологической гетерогенности (Polyglot Microservices). Разделение серверной логики на независимые слои обосновано объективными требованиями: задачи управления сложными бизнес-правилами и сущностями решаются инструментами с развитой объектно-ориентированной экосистемой, а узкие места, чувствительные к высоким нагрузкам, реализованы на базе высокопроизводительных системных подходов.

Вся серверная инфраструктура программного комплекса полностью изолирована. Использование современных технологий контейнеризации обеспечивает стопроцентную идентичность сред разработки, тестирования и промышленной эксплуатации, а также позволяет развертывать все компоненты платформы (включая базы данных и хранилища) согласованно и автоматизировано.

3.2 Пользовательские сценарии

3.2.1 Регистрация, авторизация пользователя и разграничение прав доступа

Процесс аутентификации построен на механизме делегирования: управление учетными данными передано внешнему провайдеру, что

					090207.193754.ПЗ	Лист 48
Изм.	Лист	№ докум.	Подпись	Дата		

исключает необходимость хранения паролей на сервере. При успешном входе через мобильное приложение клиент получает JWT-токен, который в дальнейшем прикрепляется к каждому HTTP-запросу. Серверная часть выполняет криптографическую верификацию токена (с использованием публичных ключей JWKS) и сопоставляет извлеченный идентификатор с профилем пользователя в базе данных PostgreSQL.

Разграничение доступа реализовано через ролевую модель (RBAC) с разделением на обычных пользователей, модераторов и администраторов. Базовый доступ к функционалу платформы предоставляется только после подтверждения электронной почты. Привилегированные пользователи (модераторы и администраторы) получают расширенный доступ к панели управления для проверки публикуемого контента, обработки жалоб и применения санкций к нарушителям.

На рисунке 3.2 представлена Use-Case диаграмма авторизации и управления правами доступа.

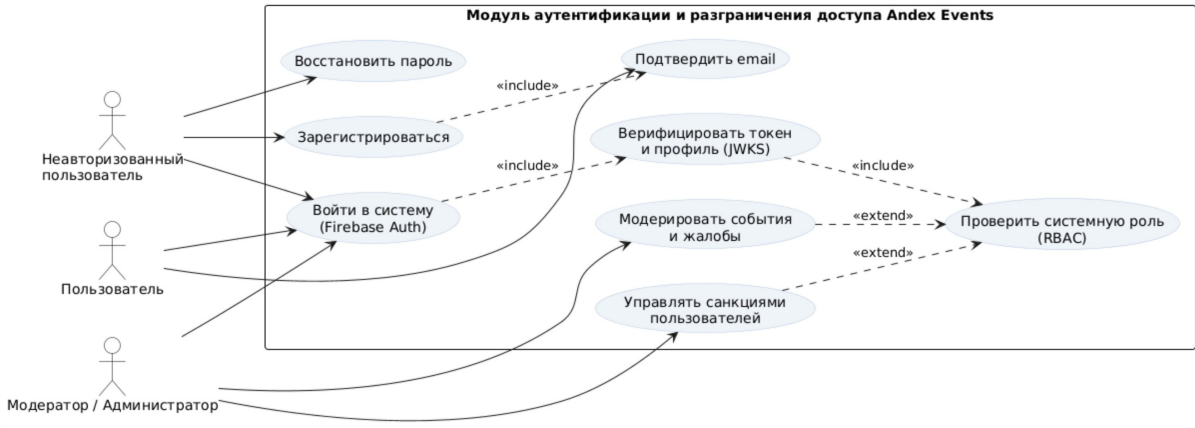


Рисунок 3.2 – Процесс авторизации и управление правами доступа

3.2.2 Заполнение профиля и прохождение онбординга

После первой авторизации пользователь проходит процедуру первоначальной настройки профиля. На данном этапе осуществляется ввод

персональных данных, выбор интересов и настройка параметров отображения профиля.

Процесс онбординга позволяет обеспечить наличие минимального объёма информации, необходимого для корректной работы алгоритмов поиска событий и формирования рекомендаций. После завершения настройки пользователь получает доступ ко всем функциям системы.

На рисунке 3.3 представлена Use-Case диаграмма заполнение профиля и прохождение онбординга.



Рисунок 3.3 – Заполнение профиля и прохождение онбординга

3.2.3 Обновление геолокации пользователя

Одной из ключевых функций программного продукта является работа с географическими данными. Для обеспечения актуальности информации система периодически получает текущие координаты пользователя и сохраняет их в базе данных.

Использование актуальных координат позволяет выполнять поиск ближайших событий, формировать рекомендации и обеспечивать корректную работу геозависимых сервисов.

На рисунке 3.4 представлена Use-Case диаграмма обновления геолокации пользователя.



Рисунок 3.4 – Обновление геолокации пользователя

3.2.4 Публикация и модерация событий

Процесс публикации мероприятия начинается с заполнения пользователем информации о событии, включая его название, описание, категорию, дату, место проведения и дополнительные сведения. При необходимости к публикации могут быть прикреплены изображения. После сохранения событие автоматически становится доступным другим пользователям и отображается в общей ленте мероприятий, поисковой выдаче и на карте.

Для обеспечения качества публикуемого контента в системе реализован механизм последующей модерации. Пользователи могут отправлять жалобы на мероприятия, нарушающие правила платформы. Полученные обращения рассматриваются модераторами и администраторами, которые принимают решение об отклонении жалобы либо применении ограничительных мер к опубликованному событию. Такой подход позволяет обеспечить быстрый доступ к новым мероприятиям при сохранении контроля над качеством размещаемой информации.

На рисунке 3.5 представлена Use-Case диаграмма публикация и модерация события.

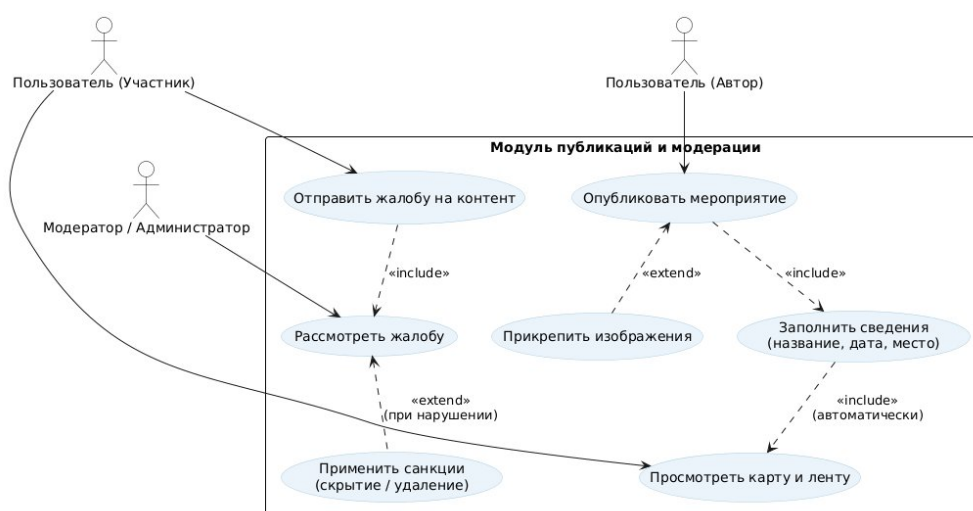


Рисунок 3.5 – Публикация и модерация события

3.2.5 Поиск и отображение событий

Поиск мероприятий в системе осуществляется посредством интерактивной карты и ленты событий. При использовании карты пользователю отображаются мероприятия, расположенные в текущей области просмотра, при этом данные автоматически обновляются при изменении масштаба или перемещении карты.

Лента событий предоставляет список опубликованных мероприятий с возможностью применения фильтров по категории, дате проведения, формату участия и другим параметрам. Дополнительно предусмотрены механизмы сортировки, позволяющие отображать наиболее актуальные и популярные события.

На рисунке 3.6 представлена Use-Case диаграмма поиска и отображения событий.

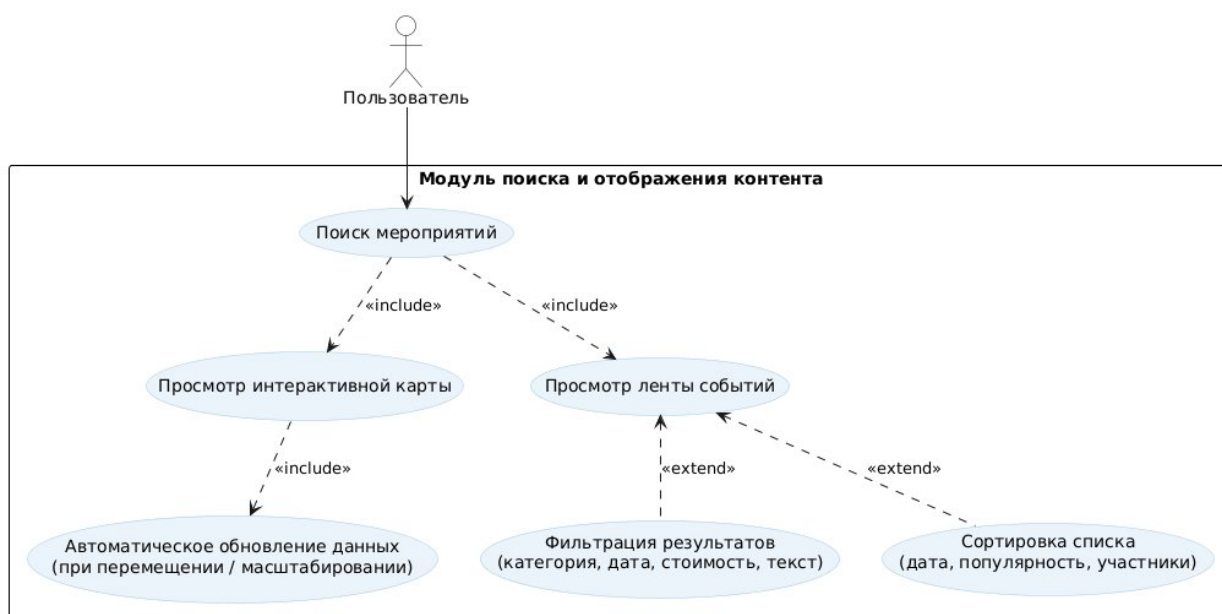


Рисунок 3.6 – Процесс поиска и отображения событий

3.2.6 Участие в событии

Пользователь может выразить интерес к мероприятию или подтвердить своё участие в нём. После регистрации информация сохраняется в системе и используется для формирования списка участников, отображения статистики события и реализации дополнительных возможностей взаимодействия между пользователями. При необходимости участие может быть отменено.

На рисунке 3.7 представлена Use-Case диаграмма участия в событии.

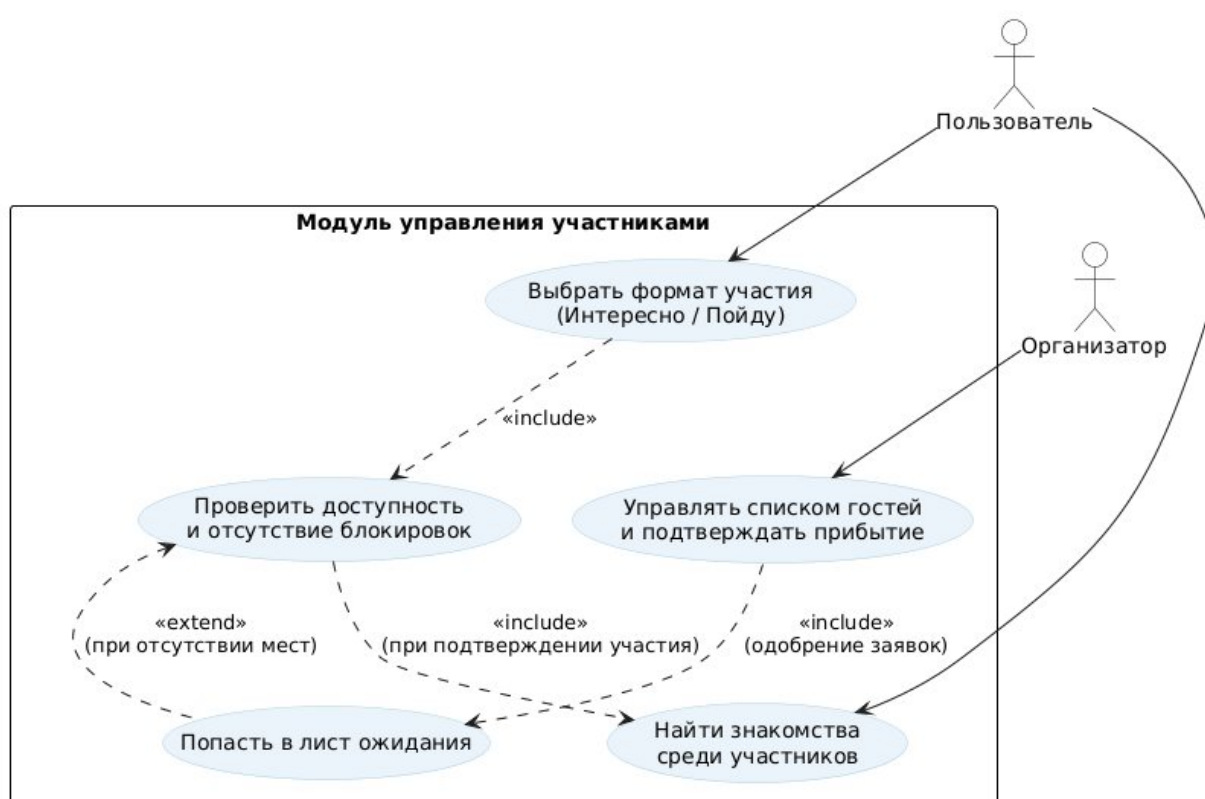


Рисунок 3.7 – Процесс участия в событии

3.2.7 Обработка мэтчей и генерация взаимного совпадения

Для организации знакомств между пользователями в системе реализован механизм оценки профилей. Пользователь может просматривать рекомендованные профили и выполнять различные действия, выражающие

заинтересованность или отказ от взаимодействия и др. Все действия фиксируются в системе и используются при формировании взаимных совпадений. В случае взаимного проявления интереса между двумя пользователями автоматически создаётся совпадение, после чего участники получают доступ к дополнительным возможностям взаимодействия. Формирование рекомендаций осуществляется с учётом параметров профиля, интересов, местоположения и участия в общих мероприятиях.

На рисунке 3.8 представлена Use-Case диаграмма обработки мэтчей и взаимных совпадений.

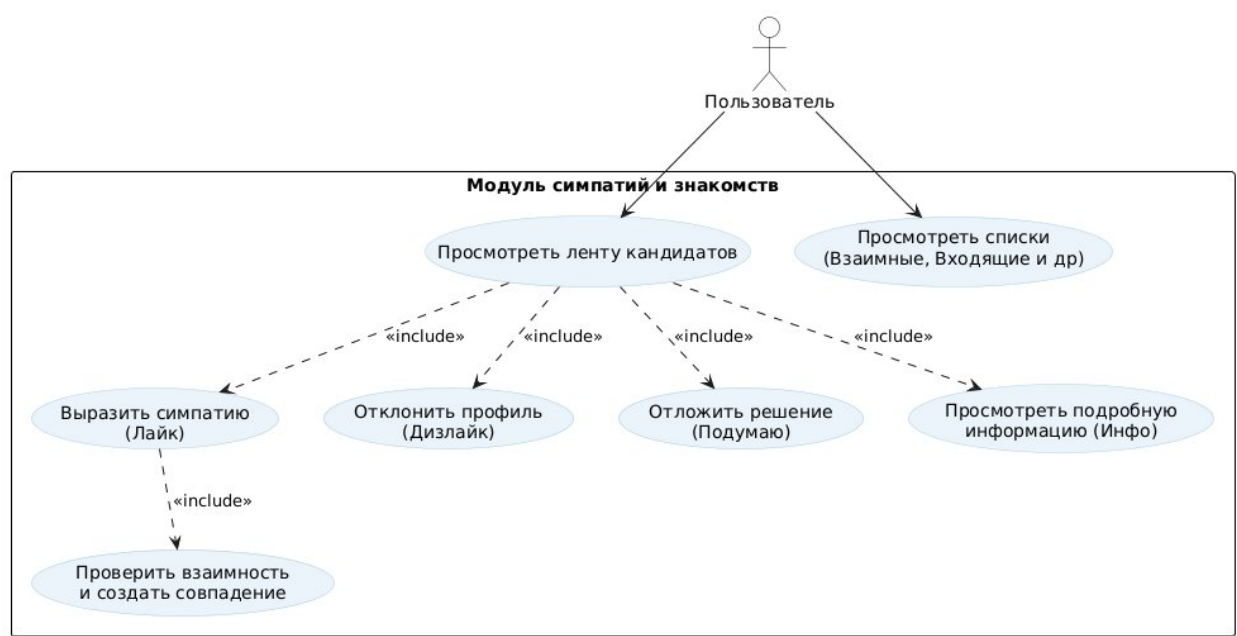


Рисунок 3.8 – Процесс мэтчей и взаимных совпадений

3.2.8 Загрузка медиаконтента

Для работы с изображениями в системе реализован специализированный механизм загрузки медиаконтента. Пользователи могут добавлять фотографии профиля, дополнительные изображения и иллюстрации к создаваемым мероприятиям. Перед сохранением выполняется проверка корректности и размера файлов, после чего медиаматериалы размещаются в

отдельном хранилище, а ссылки на них сохраняются в системе. Использование специализированного хранилища позволяет эффективно работать с мультимедийными данными, снижать нагрузку на основную базу данных и обеспечивать быстрый доступ к загруженному контенту.

На рисунке 3.9 представлена Use-Case диаграмма загрузки медиаконтента.

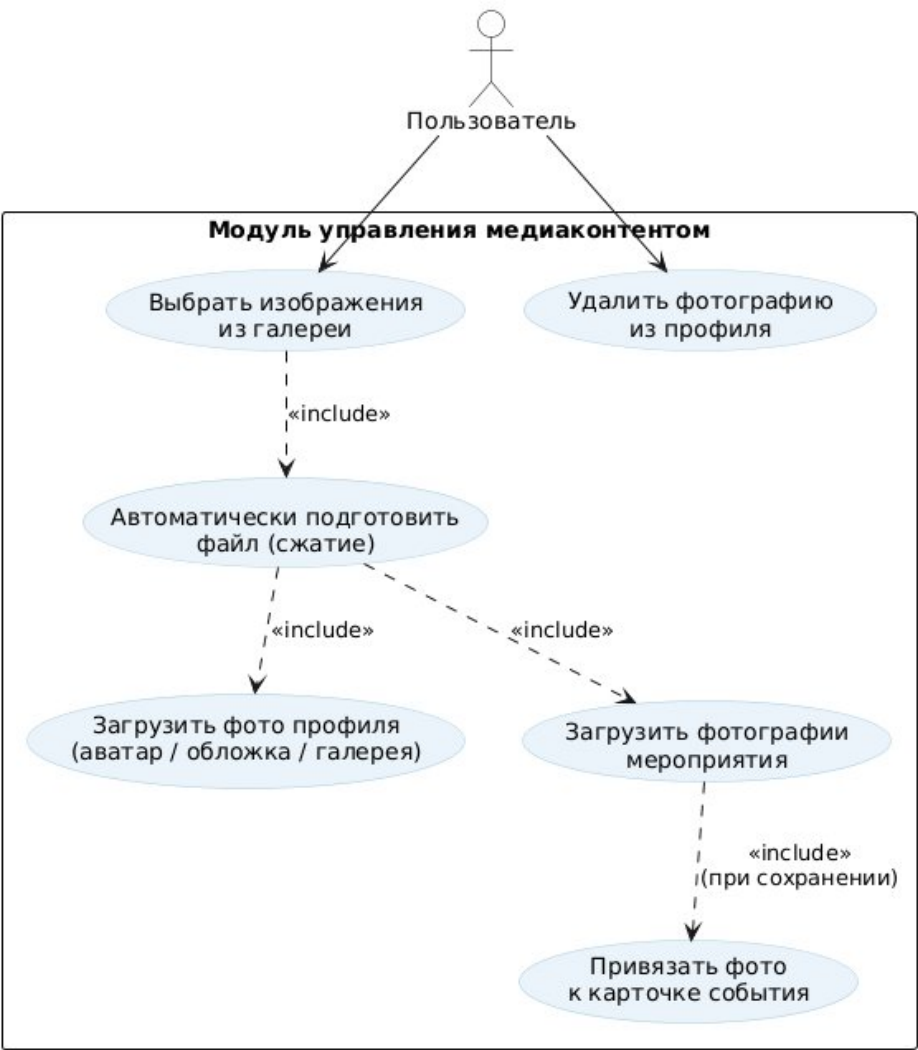


Рисунок 3.9 – Процесс загрузки медиаконтента

3.2.9 Формирование ленты рекомендации в мэтчах

Для подбора потенциально интересных пользователей в системе реализован механизм формирования персонализированной ленты рекомендаций. При её построении учитываются такие параметры, как наличие общих интересов, участие в одних мероприятиях, географическая близость пользователей, полнота заполнения профиля и история предыдущих взаимодействий. Дополнительно система исключает уже просмотренные профили и сформированные совпадения. Использование данного подхода позволяет повысить релевантность рекомендаций и увеличить вероятность успешного взаимодействия между пользователями.

На рисунке 3.10 представлена DFD-схема формирования ленты рекомендации в мэтчах.

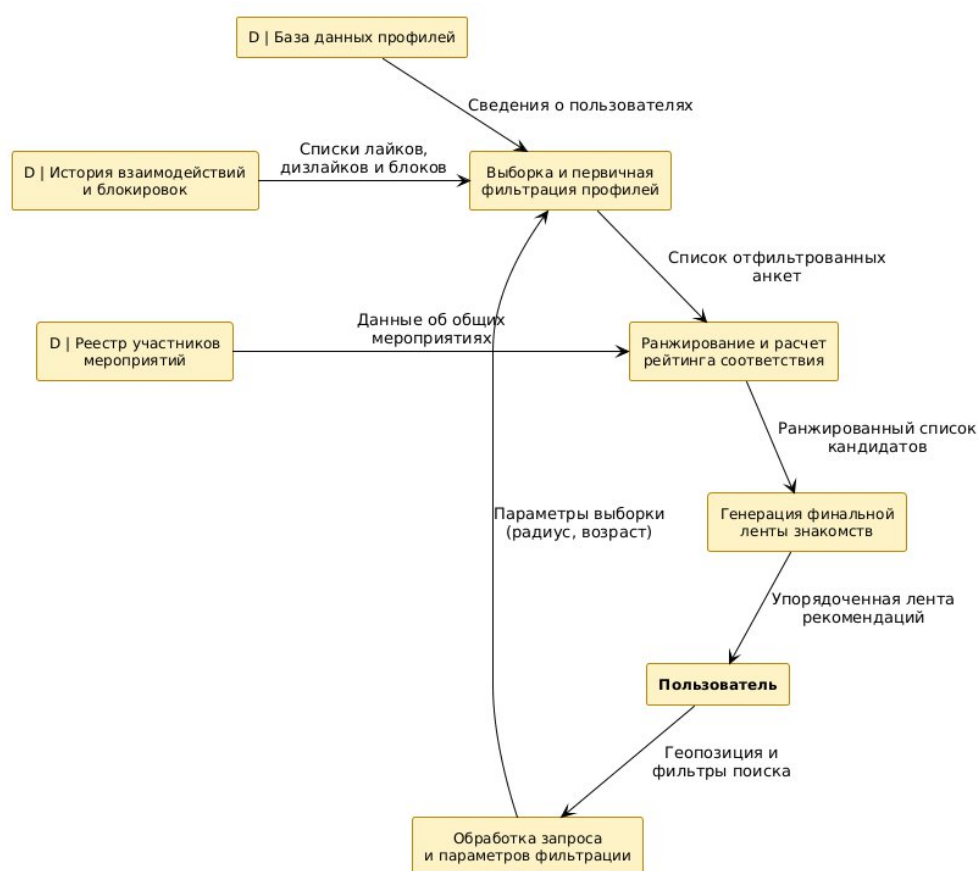


Рисунок 3.10 – Формирование ленты рекомендации в мэтчах

3.2.10 Расчет ключевых показателей

В системе реализован механизм автоматического расчёта ключевых показателей, используемых для отображения статистической и аналитической информации. К основным метрикам относятся количество участников мероприятий, рейтинг событий и организаторов, число взаимных совпадений между пользователями, а также расстояние до событий и актуальность проводимых мероприятий. Расчёт показателей выполняется на основе актуальных данных, хранящихся в системе, что позволяет обеспечивать достоверность отображаемой информации и поддерживать её в актуальном состоянии.

На рисунке 3.11 представлена DFD-схема расчёта ключевых показателей.

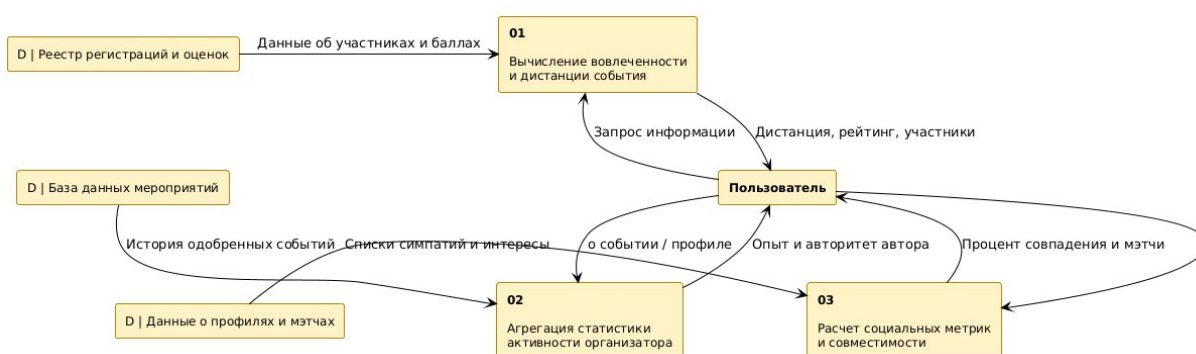


Рисунок 3.11 – Расчёт ключевых показателей

3.2.11 Просмотр активности через профиль

Профиль пользователя предоставляет доступ к информации о его активности в системе. В данном разделе отображаются сведения о созданных и посещаемых мероприятиях, взаимных совпадениях, входящих и исходящих взаимодействиях с другими пользователями, а также основные данные профиля. Дополнительно пользователю доступны функции редактирования

личной информации, управления настройками конфиденциальности и просмотра статистики собственной активности.

На рисунке 3.12 представлена Use-Case диаграмма просмотра активности через профиль.

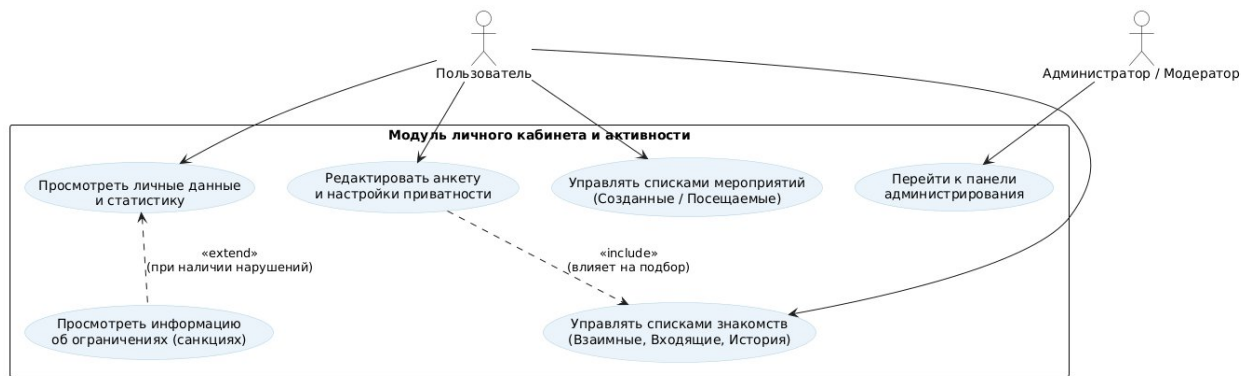


Рисунок 3.12 – Просмотр активности через профиль

3.2.12 Модерация событий и обработка жалоб

Для обеспечения качества публикуемого контента в системе реализован механизм постмодерации. Пользователи могут отправлять жалобы на мероприятия, нарушающие правила платформы, после чего обращения передаются на рассмотрение модераторам и администраторам. По результатам проверки могут быть приняты решения об отклонении жалобы, удалении события или применении ограничительных мер. Дополнительно предусмотрены механизмы санкций как для отдельных мероприятий, так и для пользователей, нарушающих правила использования системы.

На рисунке 3.13 представлена Use-case диаграмма модерации событий и обработки жалоб.

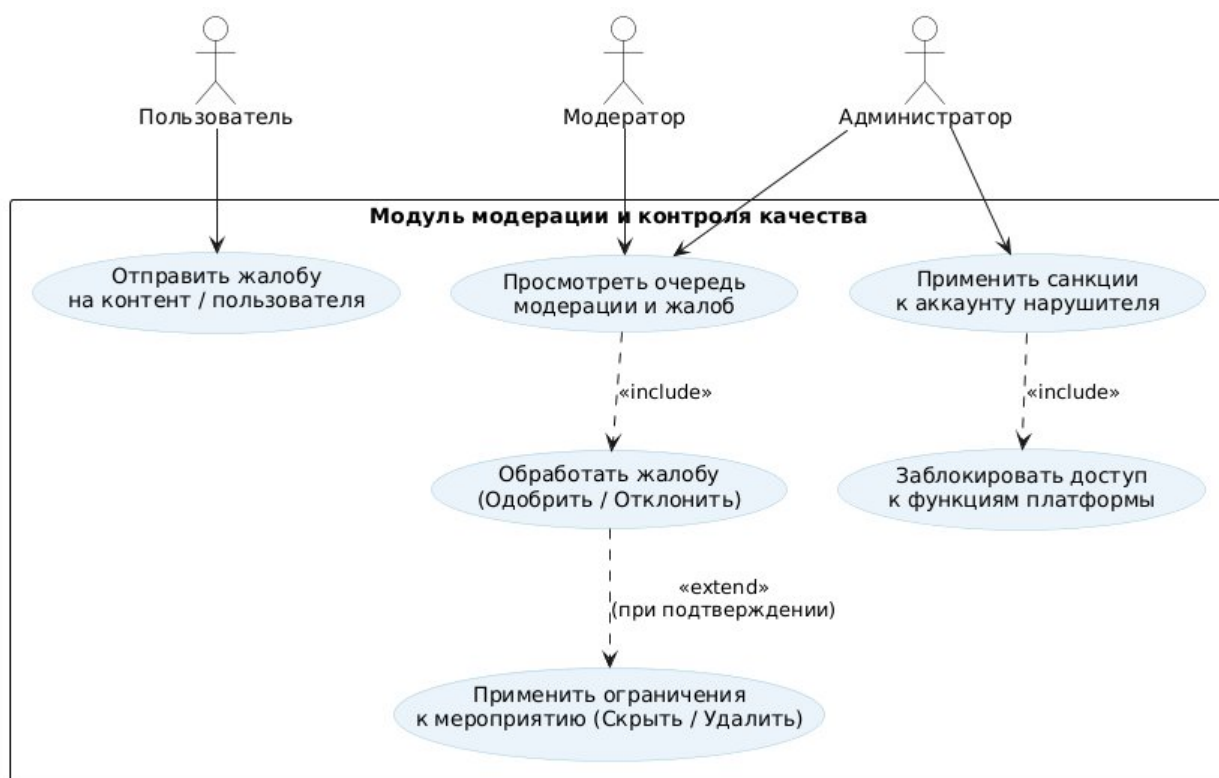


Рисунок 3.13 – Модерация событий и обработка жалоб

3.2.13 Управление настройками приватности

Для обеспечения контроля над персональными данными в системе предусмотрен раздел настроек приватности. Пользователь может управлять отображением своего профиля в рекомендациях, скрывать отдельные сведения о своей активности, настраивать параметры видимости и задавать критерии подбора потенциальных совпадений. Использование данных настроек позволяет повысить уровень конфиденциальности и обеспечить более гибкое управление процессом взаимодействия с другими пользователями платформы.

На рисунке 3.14 представлена Use-Case диаграмма настройки приватности.



Рисунок 3.14 – Настройка приватности

3.2.14 Просмотр сводной статистики администратором

Администраторам и модераторам системы доступны инструменты контроля и управления платформой, включающие работу с жалобами, модерацию мероприятий, управление пользователями и просмотр журнала действий. Для удобства обработки информации предусмотрены механизмы фильтрации и группировки данных, позволяющие оперативно выявлять проблемные ситуации и принимать необходимые меры. Использование административных инструментов обеспечивает поддержание безопасности платформы и соблюдение установленных правил её использования.

На рисунке 3.15 представлена Use-case диаграмма процесса просмотра сводной статистики администратором.

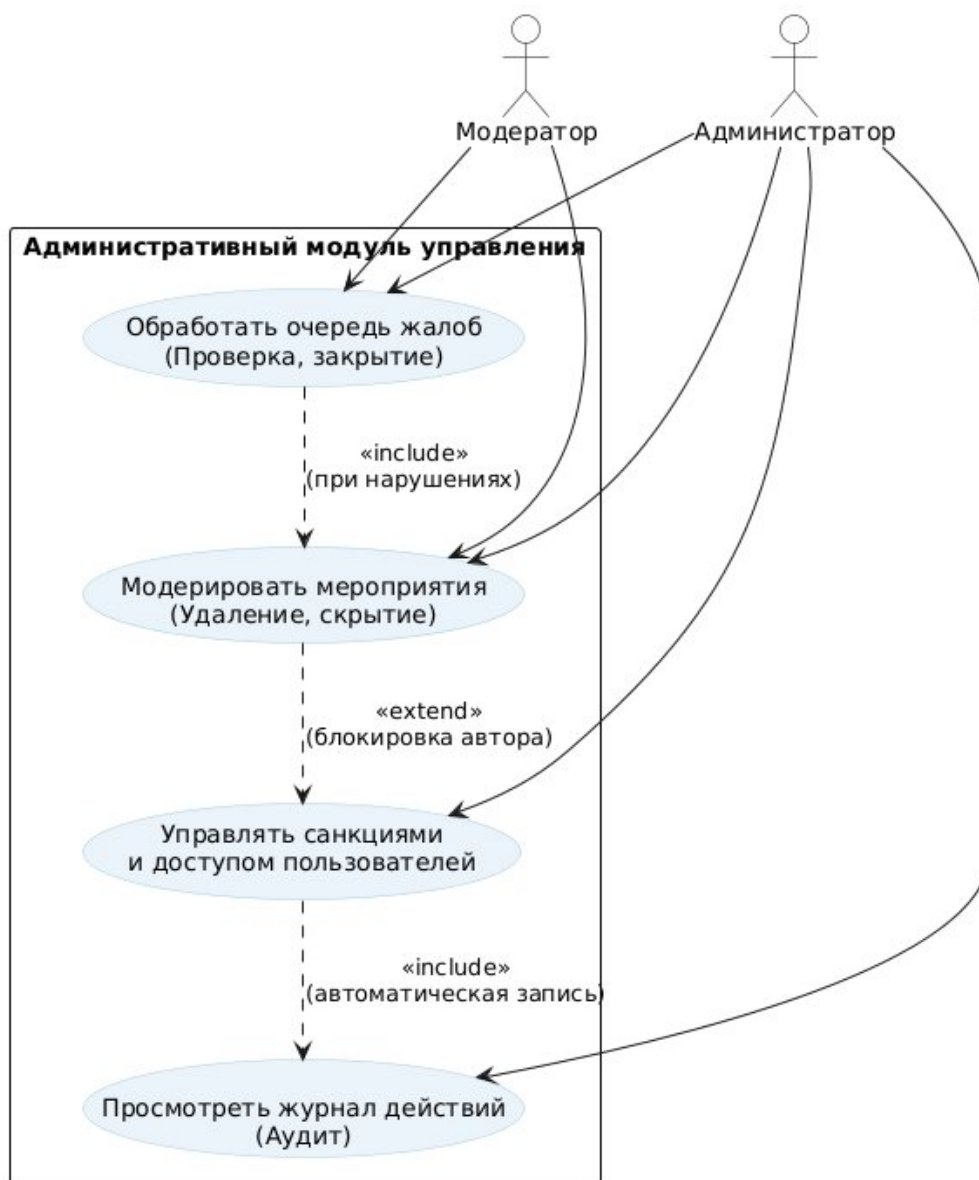


Рисунок 3.15 – Просмотр статистики администратором

3.3 Передача и хранение данных

Подсистема передачи и хранения данных обеспечивает надёжное, безопасное и эффективное взаимодействие между всеми компонентами программного продукта. Она объединяет клиентское приложение, серверные подсистемы, сервисы обработки данных и средства хранения информации, обеспечивая целостность, доступность и актуальность данных на всех этапах работы системы.

На входе подсистема принимает запросы пользователей, связанные с управлением профилями, поиском мероприятий, регистрацией участия, формированием рекомендаций и загрузкой медиаконтента. На выходе пользователям предоставляются обработанные данные в виде профилей участников, списков мероприятий, рекомендаций для знакомств, уведомлений и статистической информации.

Основными компонентами подсистемы являются реляционная база данных, предназначенная для хранения структурированной информации о пользователях, мероприятиях и взаимодействиях между ними, а также объектное хранилище, используемое для размещения изображений профилей и медиаматериалов мероприятий. Взаимодействие между клиентской частью и функциональными сервисами осуществляется через единый шлюз обработки запросов, обеспечивающий маршрутизацию обращений и контроль доступа.

Ключевые свойства реализации:

- централизованное хранение данных — информация о пользователях, мероприятиях, участниках, жалобах и совпадениях хранится в единой базе данных с логическим разделением по функциональным областям, что обеспечивает согласованность данных и упрощает их обработку;
- разделение структурированных данных и медиаконтента — основные сведения системы сохраняются в реляционной базе данных, тогда как изображения и другие мультимедийные материалы размещаются в специализированном файловом хранилище, что позволяет повысить производительность и снизить нагрузку на базу данных;
- геопространственная обработка данных — подсистема поддерживает хранение и обработку координат мероприятий, обеспечивая выполнение поиска объектов по расстоянию и отображение событий в зависимости от местоположения пользователя;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
63

– оптимизация обмена данными — клиентскому приложению передаются только необходимые данные и метаданные, что позволяет уменьшить объём сетевого трафика и повысить скорость работы системы;

– локальное кэширование информации — для наиболее востребованных данных используются механизмы временного хранения на стороне клиента, что снижает количество повторных обращений к серверу и повышает отзывчивость пользовательского интерфейса.

Использование описанного подхода обеспечивает стабильную работу системы при увеличении количества пользователей, мероприятий и объёма обрабатываемых данных, а также создаёт основу для дальнейшего масштабирования программного продукта.

3.3.1 Потоки передачи данных

Описание процессов передачи данных:

– аутентификация и идентификация пользователя: пользователь выполняет вход или регистрацию в системе. После успешной проверки учётных данных создаётся пользовательская сессия, загружается профиль пользователя и его персональные настройки. Дополнительно выполняется синхронизация параметров устройства, необходимых для отправки уведомлений;

– обновление пользовательских данных: в процессе работы приложения осуществляется передача и сохранение информации о профиле пользователя, его настройках конфиденциальности, интересах и текущем местоположении. Полученные данные используются для формирования рекомендаций и обеспечения работы геолокационных функций системы;

– поиск и отображение мероприятий: пользователь выполняет поиск событий посредством карты или ленты мероприятий. Запросы поступают в подсистему обработки событий, где выполняется фильтрация данных по

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
64

географическому расположению, параметрам поиска и актуальности мероприятий. После обработки результаты передаются клиентскому приложению для отображения;

– создание и публикация мероприятий: пользователь формирует карточку мероприятия, указывая его основные характеристики и при необходимости прикрепляя изображения. Данные сохраняются в системе, а мероприятие становится доступным для отображения другим пользователям платформы;

– регистрация участия в мероприятиях: пользователь может выразить интерес к мероприятию либо подтвердить своё участие. Информация о регистрации сохраняется в базе данных и используется для формирования списков участников, расчёта статистических показателей и организации взаимодействия между участниками события;

– формирование рекомендаций и совпадений: подсистема рекомендаций анализирует информацию о пользователях, их интересах, активности и местоположении. На основе полученных данных формируется список рекомендуемых профилей. Результаты пользовательских действий сохраняются в системе и используются для автоматического формирования взаимных совпадений;

– обработка медиаконтента: изображения профилей и мероприятий передаются в специализированное файловое хранилище, после чего ссылки на загруженные файлы сохраняются в базе данных и используются для отображения контента в пользовательском интерфейсе;

– модерация контента и обработка жалоб: пользователи могут отправлять обращения о нарушениях правил платформы. Жалобы регистрируются в системе и передаются модераторам для рассмотрения. По результатам проверки могут применяться различные ограничительные меры в отношении пользователей или опубликованного контента;

- хранение и обработка данных: структурированная информация о пользователях, мероприятиях, совпадениях, жалобах и уведомлениях хранится в реляционной базе данных. Мультимедийные файлы размещаются в отдельном файловом хранилище. Такое разделение обеспечивает эффективную обработку данных и повышает производительность системы;
- мониторинг и актуализация информации: система обеспечивает постоянное обновление пользовательских данных, пересчёт рекомендаций, актуализацию статистических показателей и контроль состояния основных компонентов программного продукта.

Таблица 3.1 — Потоки передачи данных

Процесс	Краткое описание	Роль в цепочке
Аутентификация пользователя	Выполняется проверка личности пользователя, загрузка профиля и определение прав доступа	Обеспечение безопасного доступа к функциональным возможностям системы
Синхронизация данных устройства	Выполняется обновление служебных параметров устройства и настроек уведомлений	Поддержание актуальности пользовательских данных и обеспечение доставки уведомлений
Обновление геолокации	Система получает и сохраняет текущие координаты пользователя	Обеспечение работы геозависимых функций и персонализированного поиска
Поиск мероприятий на карте	Выполняется выборка мероприятий в пределах выбранной области просмотра	Предоставление пользователю релевантных событий с учётом местоположения
Просмотр ленты мероприятий	Формируется перечень доступных мероприятий с возможностью дальнейшей фильтрации и сортировки	Предоставление информации о доступных событиях платформы

Продолжение таблицы 3.1

Процесс	Краткое описание	Роль в цепочке
Создание и публикация мероприятия	Пользователь формирует карточку мероприятия и размещает её в системе	Пополнение информационной базы новыми событиями
Регистрация участия в мероприятии	Пользователь подтверждает участие либо выражает интерес к мероприятию	Формирование состава участников и статистики мероприятия
Формирование ленты рекомендаций	Выполняется подбор потенциально интересных пользователей на основе установленных критериев	Поддержка механизма знакомств и персонализированных рекомендаций

3.3.2 Логическая модель данных

На рисунке 3.16 представлена логическая модель базы данных, реализованная в соответствии с нормализованной схемой 3NF.

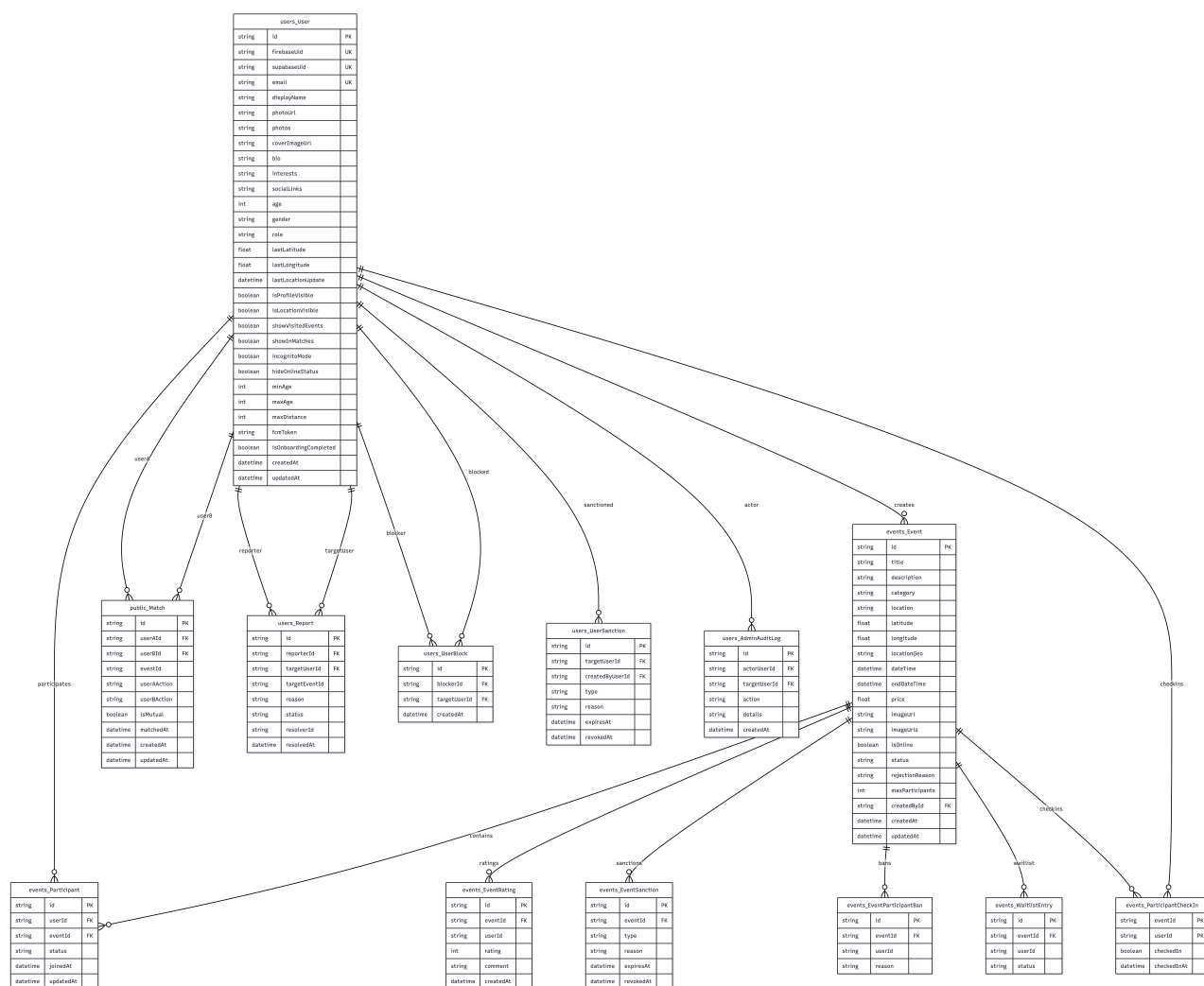


Рисунок 3.16 — Логическая модель базы данных

Основные сущности и связи:

- User — центральная сущность, содержащая данные об учётных записях пользователей, включая профильную информацию, актуальные географические координаты, настройки приватности и системные роли;
- Event — описывает созданное мероприятие, включая его точное местоположение (координаты), категорию, временные рамки проведения, описание, ценовые параметры и текущий статус модерации;
- Participant — фиксирует связь между пользователем и событием, определяя статус вовлечённости (заинтересованность или подтверждённое участие) и время регистрации;

- Match — хранит результаты социального взаимодействия (свайпов) между парой пользователей, фиксируя типы совершенных действий и наличие факта взаимного совпадения интересов;
- Report — содержит сведения о жалобах пользователей на ненадлежащий контент или поведение участников, включая причину обращения, идентификаторы объектов и статус рассмотрения модератором;
- UserSanction / EventSanction — описывают ограничительные меры, наложенные на пользователей или конкретные события в случае нарушения правил платформы (предупреждения, блокировки, скрывание из поиска);
- EventRating — хранит оценки и текстовые отзывы, оставленные участниками после посещения мероприятий, используемые для формирования рейтинга событий и организаторов;
- WaitlistEntry / ParticipantCheckIn — обеспечивают управление очередью на популярные события при достижении лимита мест и фиксацию фактического прибытия участника на место проведения;
- UserBlock — содержит информацию о персональных блокировках между пользователями для обеспечения комфортной и безопасной среды взаимодействия;
- AdminAuditLog — представляет собой детальный журнал действий административного персонала, фиксирующий все изменения прав доступа и применение санкций для обеспечения аудита безопасности системы.

Таблица 3.2 — Логическая модель базы данных

Таблица	Схема	Ключи	Назначение и основные связи
User	users	id (PK); firebaseUid, supabaseUid, email (UNIQUE)	Профиль, координаты (lastLatitude/lastLongitude), приватность, fcmToken, роль. 1:M → Event (createdById); 1:M → Participant; 1:M → Match; 1:M → Report

Продолжение таблицы 3.2

Таблица	Схема	Ключи	Назначение и основные связи
Report	users	id (PK); reporterId (FK → User); targetUserId (FK → User); targetEventId	Жалоба на пользователя или событие. Статус PENDING / RESOLVED / DISMISSED. M:1 → User (автор жалобы)
UserBlock	users	id (PK); UNIQUE (blockerId, targetUserId); FK → User (обе стороны)	Блокировка между пользователями. Учитывается при GET /users/matches. M:1 → User
UserSanction	users	id (PK); targetUserId (FK → User); createdByUserId (FK → User)	Санкции аккаунта: WARNING, MUTE, EVENT_CREATE_BAN, FULL_BAN. M:1 → User (цель)
AdminAuditLog	users	id (PK); actorUserId (FK → User); targetUserId (FK → User)	Журнал действий администратора (action, details JSONB). M:1 → User
Event	events	id (PK); createdById (FK → users.User)	Событие: latitude/longitude, locationGeo (PostGIS geography), status, медиа. 1:M → Participant, EventRating, EventSanction, WaitlistEntry
Participant	events	id (PK); UNIQUE (userId, eventId)	Участие INTERESTED / GOING. M:1 → User; M:1 → Event
EventRating	events	id (PK); UNIQUE (userId, eventId); eventId (FK → Event)	Оценка события 1–5 и комментариев. M:1 → Event
EventSanction	events	id (PK); eventId (FK → Event)	Санкции на событие (HIDE_VISIBILITY, FREEZE_PARTICIPATION и др.). Фильтруются в публичной выборке. M:1 → Event
EventParticipantBan	events	id (PK); UNIQUE (eventId, userId); eventId (FK → Event)	Запрет участия конкретного пользователя на событии. M:1 → Event
WaitlistEntry	events	id (PK); UNIQUE (eventId, userId); eventId (FK → Event)	Лист ожидания при переполнении maxParticipants. Статус PENDING / APPROVED / REJECTED. M:1 → Event
ParticipantCheckIn	events	PK (eventId, userId); eventId (FK → Event)	Отметка чек-ина участника организатором. M:1 → Event
OrganizerBlock	events	id (PK); UNIQUE (organizerUserId, blockedUserId)	Блок-лист организатора; блокирует участие в его событиях. Связь логическая с User
Match	public	id (PK); userAId, userBId (FK → users.User); UNIQUE (пара userA/userB + eventId)	Действия LIKE / DISLIKE / SUPER_LIKE, флаг isMutual, опциональный eventId. M:1 → User (оба участника)

3.4 Серверная часть

Серверная часть платформы Andex Events является центральным компонентом системы и отвечает за обработку пользовательских запросов, управление событиями, организацию взаимодействия между пользователями, обеспечение безопасности данных и поддержку административных функций. Именно на серверном уровне реализуются механизмы аутентификации, хранения и обработки информации, геопоиска мероприятий, матчинга пользователей, модерации контента и работы с медиаданными.

Серверная часть построена на основе микросервисной архитектуры и включает набор независимых сервисов, взаимодействующих через API-шлюз. Такой подход обеспечивает высокую масштабируемость, отказоустойчивость и возможность независимого развития отдельных компонентов системы. Для хранения структурированных данных используется реляционная база данных PostgreSQL с расширением PostGIS, а для хранения медиаконтента применяется объектное хранилище MinIO.

Основные функции серверной части:

- аутентификация и авторизация пользователей на основе JWT-токенов;
- управление профилями пользователей и настройками приватности;
- обработка и хранение геолокационных данных;
- создание, редактирование, поиск и модерация событий;
- организация участия пользователей в мероприятиях;
- формирование и обработка жалоб;
- реализация механизма знакомств и взаимных совпадений между пользователями;
- ранжирование рекомендаций для ленты знакомств;
- загрузка, хранение и выдача медиаконтента;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
71

- управление санкциями и административными ограничениями;
- ведение журнала аудита действий модераторов и администраторов;
- предоставление данных клиентскому приложению через единый API.

Для обеспечения надёжной работы все сервисы развёртываются в контейнерах Docker и функционируют в единой инфраструктурной среде. Использование контейнеризации упрощает сопровождение системы, автоматизирует процесс развертывания и позволяет масштабировать отдельные компоненты по мере роста нагрузки. Такой подход обеспечивает стабильную работу платформы при одновременном обслуживании большого количества пользователей, событий и операций обмена данными.

3.4.1 Контекстная диаграмма IDEF0

На рисунке 3.17 показан верхне-уровневый контур работы серверной части.

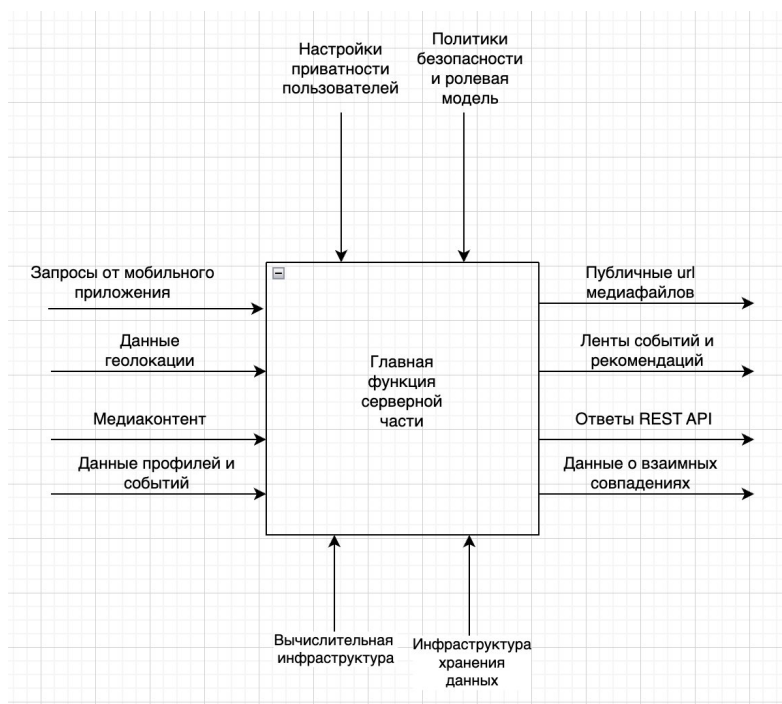


Рисунок 3.17 — Контекстная диаграмма IDEF0

Система получает на вход:

- запросы от мобильного приложения — HTTP-запросы, инициируемые действиями пользователей в интерфейсе;
- данные геолокации — географические координаты (широта и долгота) для работы пространственных функций поиска;
- медиаконтент — бинарные файлы (изображения), загружаемые пользователями для оформления профилей и событий;
- данные профилей и событий — структурированная информация о пользователях и создаваемых мероприятиях.

Выходами системы являются:

- публичные URL медиафайлов — доступные ссылки на контент, размещенный в облачном хранилище;
- ленты событий и рекомендаций — сформированные списки мероприятий и потенциальных знакомств, ранжированные согласно алгоритмам системы;
- ответы REST API — программные ответы сервера в формате JSON, содержащие результат обработки запросов;
- данные о взаимных совпадениях — информация о возникновении взаимного интереса между пользователями.

Используемые механизмы:

- микросервисная архитектура — структурная организация серверной части в виде набора независимых сервисов;
- хранилища данных — реляционные и пространственные базы данных (PostgreSQL с расширением PostGIS) для работы со структурированной информацией;
- объектное хранилище — специализированная система (MinIO) для хранения и раздачи статического контента;
- среда контейнеризации — технология (Docker), обеспечивающая изоляцию и удобство развертывания компонентов системы.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
73

Управлениями являются:

- настройки приватности пользователей — заданные пользователями параметры видимости профилей и координат;
- политики безопасности и ролевая модель — правила разграничения доступа (RBAC) и верификации запросов;
- правила модерации событий и жалоб — регламенты, по которым осуществляется контроль качества публикуемого контента;
- схемы миграций базы данных — механизмы управления версионностью и структурой хранимых данных

3.4.2 Декомпозиция IDEF0

На рисунке 3.18 представлены детализированные подпроцессы, реализующие главную функцию серверной части. Каждый модуль реализован как изолированный сервис с явными интерфейсами и возможностью масштабирования.

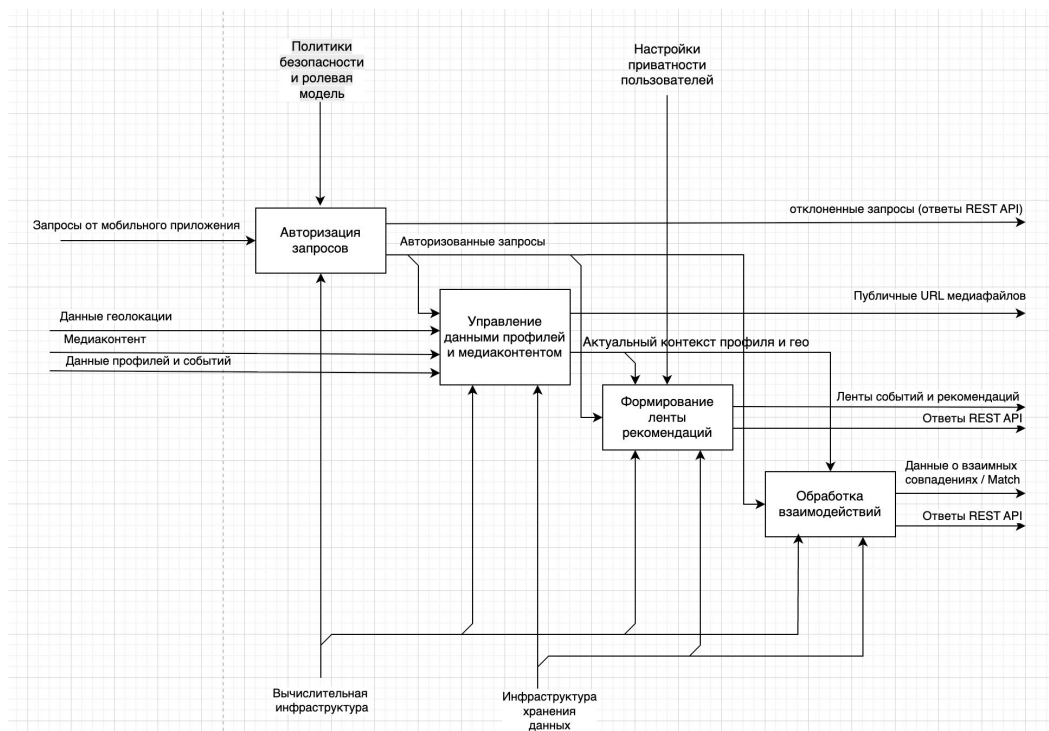


Рисунок 3.18 — Декомпозиция IDEF0

Таблица 3.3 — Декомпозиция IDEF1X

A1. Авторизация запросов	Проверка подлинности пользователя, валидация токенов доступа и применение политик безопасности перед передачей запроса в систему	Firebase Authentication, JWT, Auth Middleware, RBAC
A2. Управление данными профилей и медиаконтентом	Хранение и обработка данных пользователей, координат, настроек приватности, а также загрузка и выдача медиаконтента	users-service, upload-service, PostgreSQL, MinIO
A3. Формирование ленты рекомендаций	Геолокационный поиск событий и пользователей, фильтрация данных и формирование персонализированных рекомендаций на основе контекста пользователя	events-service, recommendation logic, PostGIS, PostgreSQL
A4. Обработка взаимодействий	Обработка пользовательских действий (лайки, дизлайки)	match-service, PostgreSQL, Firebase Admin SDK

3.5 Пользовательская часть

Пользовательская часть программного комплекса реализована в виде кроссплатформенного мобильного приложения на базе фреймворка Flutter. Она предназначена для интерактивного взаимодействия пользователей с системой: регистрации и авторизации, публикации и поиска мероприятий, управления профилем и участия в механизмах социального взаимодействия (матчинга). Приложение функционирует как тонкий клиент, взаимодействующий с серверной частью через унифицированный REST API. Для обеспечения безопасности используется механизм авторизации на основе JWT-токенов, а архитектурный паттерн BLoC (Business Logic Component) позволяет эффективно разделять интерфейс и логику обработки данных, минимизируя нагрузку на клиентское устройство и сетевой трафик.

3.5.1 Интерфейс приложения

Графический интерфейс мобильного клиента спроектирован с учетом современных стандартов проектирования пользовательского опыта (UX) и

обеспечивает интуитивно понятный доступ к функционалу системы. Основная навигация организована через нижнюю панель вкладок, разделяющую приложение на четыре ключевых функциональных модуля:

- Интерактивная карта — визуализация мероприятий в реальном времени с привязкой к текущему географическому положению пользователя и области видимости;
- Лента событий — структурированный список актуальных публикаций с возможностью глубокой фильтрации и сортировки по заданным критериям;
- Знакомства — интерфейс для подбора потенциальных знакомств, реализованный на базе механики колоды карточек;
- Профиль — персональный кабинет пользователя, содержащий статистику активности, историю взаимодействий и инструменты настройки приватности.

Интерфейс является адаптивным, корректно отображается на устройствах с различными разрешениями экранов и поддерживает ролевую модель: для учетных записей с привилегированными правами (модератор, администратор) динамически активируются дополнительные элементы управления.

3.5.2 Потоки работы пользователя

3.5.2.1 Авторизация, регистрация, восстановление

Первый экран после установки приложения — экран приветствия (OnboardingScreen). Он предоставляет переход к входу в существующий аккаунт или регистрации нового пользователя через Firebase Authentication.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
76

Действия пользователя:

- на экране приветствия выбрать «Войти» или «Зарегистрироваться»;
- ввести электронную почту и пароль либо выполнить вход через учётную запись Google;
- при необходимости перейти на экран восстановления пароля и указать адрес электронной почты для получения письма со ссылкой сброса;
- при первой регистрации подтвердить адрес электронной почты, если система этого требует.

Результат действий пользователя:

- пользователь успешно аутентифицируется; приложение получает доступ к защищённым функциям сервера;
- на сервере создаётся или загружается профиль пользователя;
- если адрес электронной почты не подтверждён, приложение перенаправляет на экран подтверждения с инструкцией и возможностью повторной отправки письма;
- если первичное заполнение профиля не завершено, пользователь направляется на пошаговый онбординг;
- при успешной авторизации и завершённом онбординге открывается главный экран с четырьмя вкладками и начинается фоновое обновление местоположения;
- при выходе из аккаунта сеанс завершается, пользователь возвращается на экран входа.

3.5.2.2 Онбординг

Онбординг состоит из трёх последовательных экранов и блокирует доступ к основным разделам приложения до завершения первичной настройки профиля.

					090207.193754.ПЗ	Лист 77
Изм.	Лист	№ докум.	Подпись	Дата		

Действия пользователя:

- шаг 1 — указать возраст и пол, при желании загрузить фотографию; шаг можно пропустить без заполнения полей;
- шаг 2 — выбрать не менее трёх интересов из предложенного списка (музыка, спорт, информационные технологии и др.); без трёх интересов переход на следующий шаг заблокирован;
- шаг 3 — разрешить доступ к геолокации и передать координаты либо нажать «Пропустить» и завершить настройку без указания места.

Результат действий пользователя:

- загруженная фотография сохраняется на сервере и отображается в профиле;
- выбранные интересы записываются в профиль и учитываются при подборе знакомств;
- координаты передаются на сервер при выдаче разрешения; при пропуске шага профиль создаётся без привязки к месту;
- на сервере фиксируется завершение онбординга; статус сохраняется локально для быстрого запуска при временной недоступности сети;
- пользователь перенаправляется на главный экран приложения.

3.5.2.3 Карта событий

Карта событий — одна из четырёх вкладок главного экрана. Отображает актуальные мероприятия в виде маркеров на интерактивной карте с подгрузкой по видимой области.

Действия пользователя:

- открыть вкладку «Карта» через нижнюю навигационную панель;
- перемещать и масштабировать карту для изучения событий в интересующем районе;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
78

- нажать на маркер события для перехода к детальной карточке;
- воспользоваться текстовым поиском и списком событий в нижней панели;
- нажать центральную кнопку «+» для создания нового события или перейти на экран поиска.

Результат действий пользователя:

- при каждом значимом смещении карты сервер получает запрос на события в текущей видимой области;
- на карте отображаются только маркеры мероприятий, попадающие в видимую область; новые результаты дополняют уже загруженный список, а не заменяют его;
- нижняя панель показывает те же видимые события с возможностью текстового поиска и сортировки по близости к текущему положению пользователя;
- при нажатии на маркер открывается экран с полной информацией о мероприятии.

3.5.2.4 Афиша событий

Вкладка «Афиша» показывает список мероприятий карточками с фильтрацией и сортировкой на стороне приложения.

Действия пользователя:

- просматривать и прокручивать список событий;
- применять фильтры по городу, дате, категории, цене и формату;
- искать события по названию и обновлять список.

Результат действий пользователя:

- отображается отфильтрованный список актуальных мероприятий;
- при выборе карточки открывается экран деталей события.

3.5.2.5 Детали события и участие

Полная информация о мероприятии: описание, место, дата, фото, рейтинг, участники.

Действия пользователя:

- ознакомиться с карточкой события;
- отметить интерес или подтвердить участие;
- просмотреть участников, построить маршрут, перейти к матчам среди участников;
- организатору — управлять списком участников и листом ожидания.

Результат действий пользователя:

- статус интереса и участия сохраняется на сервере;
- при переполнении лимита участников пользователь попадает в лист ожидания;
- из карточки события доступен экран матчей среди подтвердивших участие;
- после завершения события участники могут оставить оценку.

3.5.2.6 Создание события

Форма публикации открывается с главного экрана. При санкции на создание показывается сообщение об ограничении.

Действия пользователя:

- заполнить описание, категорию, дату, место и цену;
- прикрепить фотографии;
- опубликовать мероприятие.

Результат действий пользователя:

- событие сразу становится доступно на карте и в афише;

— предварительная модерация не требуется; контроль — по жалобам после публикации.

3.5.2.7 Мэтчи

Вкладка «Матчи» и экран матчей у события реализуют подбор знакомств через свайпы по карточкам профилей.

Действия пользователя:

- просматривать рекомендованные профили;
- выражать симпатию, отказ или отложить профиль жестами или кнопками;
- обновить подборку при её исчерпании.

Результат действий пользователя:

- действия сохраняются на сервере;
- при взаимной симпатии фиксируется совпадение;
- взаимные совпадения отображаются в профиле пользователя.

3.5.2.8 Профиль и настройки

Личный кабинет: мероприятия пользователя, история симпатий, редактирование данных и настройки приватности.

Действия пользователя:

- просматривать созданные, посещаемые и понравившиеся события;
- просматривать списки симпатий по категориям;
- редактировать профиль и параметры приватности;
- выйти из аккаунта;
- при наличии прав — открыть панель модератора или администратора.

Результат действий пользователя:

					090207.193754.ПЗ	Лист 81
Изм.	Лист	№ докум.	Подпись	Дата		

- изменения профиля сохраняются на сервере;
- настройки приватности учитываются в подборе матчей;
- при активных санкциях отображается предупреждение об ограничениях.

3.5.2.9 Панель модератора и администратора

Доступна из профиля для пользователей с соответствующими правами.

Действия пользователя:

- просматривать события с жалобами и принимать решения;
- обрабатывать пользовательские обращения;
- администратору дополнительно — управлять пользователями, ролями и просматривать журнал аудита.

Результат действий пользователя:

- нарушающий контент скрывается от пользователей;
- жалобы закрываются;
- санкции немедленно ограничивают действия нарушителя в приложении.

3.5.3 Основные экраны и их функции

На таблице 3.4. отображены и описаны основные реализованные экранные формы приложения, их назначение и ключевые элементы

Таблица 3.4 — Экранные формы

Экран	Назначение	Ключевые элементы интерфейса
Знакомство с приложением	Первый запуск	Краткое описание возможностей сервиса
Вход и регистрация	Аутентификация	Email, пароль, вход через Google
Подтверждение почты	Верификация email	Инструкция и повторная отправка письма

Продолжение таблицы 3.4

Экран	Назначение	Ключевые элементы интерфейса
Онбординг	Первичная настройка	Профиль, интересы, геолокация
Главный экран	Основная навигация	Вкладки «Карта», «Афиша», «Матчи», «Профиль», кнопка «+»
Карта	Геопоиск событий	Карта с маркерами, поиск, список nearby
Афиша	Каталог событий	Список карточек, фильтры и сортировка
Поиск	Текстовый поиск	Строка поиска и результаты
Матчи	Подбор знакомств	Колода карточек, свайпы
Матчи у события	Знакомства участников	Свайпы в контексте мероприятия
Детали события	Карточка мероприятия	Описание, участие, маршрут, рейтинг, матчи
Создание события	Публикация	Поля события, карта, фото
Профиль	Личный кабинет	Мероприятия, симпатии, быстрые действия
Редактирование профиля	Изменение данных	Фото, описание, интересы
Приватность	Настройки видимости	Участие в матчах, инкогнито, возрастной фильтр
Панель модератора	Контроль контента	Модерация, отчёты
Панель администратора	Управление системой	Пользователи, аудит

3.5.4 Ролевые особенности интерфейса

Обычный пользователь работает с четырьмя вкладками главного экрана, создаёт события, участвует в мероприятиях и использует матчи. Административные разделы скрыты.

Модератор дополнительно получает доступ к модерации событий и обработке жалоб.

Администратор имеет полный доступ: модерация, жалобы, управление пользователями и журнал аудита.

При санкциях интерфейс ограничивает создание событий или доступ к разделам приложения; на профиле отображается предупреждение об активных ограничениях.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
83

3.5.5 Безопасность и UX-детали

Для обеспечения безопасности и улучшения пользовательского опыта в приложении реализованы следующие меры:

- доступ к функциям сервера только после аутентификации;
- геолокация передаётся после явного разрешения пользователя;
- медиафайлы хранятся отдельно от основной базы данных;
- ошибки и ограничения отображаются понятными уведомлениями;
- настройки приватности исключают пользователя из подборки матчей;
- санкции ограничивают действия без удаления учётной записи

3.6 Вывод по проектированию программного продукта

В ходе раздела 3 выполнено полное формальное описание архитектуры системы, всех процессов и структур данных, что позволило получить целостное представление о будущем программном комплексе. Ключевые результаты проектирования сводятся к следующему.

Таблица 3.5 — Спроектированный программный продукт

Объект проектирования	Итоговые решения	Эффект
Архитектура	Микросервисная архитектура на базе Java 21 / Spring Boot 3, Go / Gin, API-шлюза Traefik, PostgreSQL 16 + PostGIS, MinIO, Firebase Authentication и FCM, Docker Compose; клиент — Flutter (iOS / Android)	Высокая масштабируемость, независимое развитие и развёртывание компонентов
Серверная часть	Контекстная IDEF0-диаграмма (рисунок 3.17) и декомпозиция (рисунок 3.18) фиксируют четыре изолированных подпроцессов; Traefik используется как reverse проху и единая точка входа API	Чёткие интерфейсы между службами, возможность вынести обработку свайпов и загрузку медиа в отдельные модули

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
84

Продолжение таблицы 3.5

Объект проектирования	Итоговые решения	Эффект
Передача и хранение данных	DFD-цепочки процессов (в т. ч. рисунки 3.10, 3.11) и двухконтурная схема хранения: оперативные данные в PostgreSQL (схемы users, events, public) и медиафайлы в MinIO; обмен JSON по REST через Bearer-токен Firebase	Надёжность хранения, минимизация трафика и обеспечение безопасности
Логическая модель	Нормализованная ER-модель (рисунок 3.16) с основными сущностями User, Event, Participant, Match, Report, UserSanction, EventSanction, AdminAuditLog и др.; версионирование схем — Flyway	Целостность ссылок, простота миграций и расширений
Пользовательская часть	Кроссплатформенное мобильное приложение с навигацией по вкладкам «Карта», «Афиша», «Матчи», «Профиль»; описание пользовательских сценариев и экранных форм (таблица 3.4);	Единый пользовательский опыт на iOS и Android, минимальные затраты на поддержку одной кодовой базы

В результате проектирования определена архитектура платформы, объединяющей поиск городских мероприятий и знакомство пользователей. Серверная часть разделена на независимые модули, каждый из которых отвечает за свой участок функциональности: профили и подбор матчей, каталог и геописк событий, обработку свайпов, загрузку фотографий и модерацию контента. Данные хранятся в единой базе с логическим разделением на области пользователей, событий и матчей; медиафайлы вынесены в отдельное хранилище.

Спроектированы основные бизнес-процессы: регистрация и онбординг, публикация событий с немедленным отображением на карте и в афише, участие в мероприятиях, свайпы и взаимные совпадения, постмодерация по жалобам. Для мобильного клиента описаны пользовательские сценарии, экранные формы и ролевые особенности интерфейса для обычного пользователя, модератора и администратора.

4 Реализация программного продукта

4.1 Выбор и обоснование выбора средств реализации разработанной системы

Для реализации программного комплекса мобильного приложения геолокационной социальной сети — мобильной платформы поиска городских мероприятий и знакомств пользователей в контексте событий — применён современный технологический стек, обеспечивающий производительность, масштабируемость, кроссплатформенность клиента и удобство сопровождения серверной части.

В соответствии с выбранной микросервисной архитектурой разработка системы осуществлялась с использованием следующих инструментов и программных средств (таблица 4.1).

В следующих подразделах данного раздела, будет обосновано почему выбраны именно данные технологии и инструменты, и приведены аналоги.

Таблица 4.1 — Используемые технологии и инструменты

Область применения	Технологии и инструменты
Языки программирования	Dart (мобильный клиент), Java 21 (серверная часть — доменные сервисы), Go 1.24 (серверная часть — матчинг и загрузка файлов)
Фреймворки backend-разработки	Spring Boot 3.3.x (auth-, users-, events-service), Gin (match-service, upload-service)
Фреймворки frontend-разработки	Flutter (SDK из pubspec.yaml), пакет flutter_bloc для управления состоянием
API-шлюз	Traefik — единая точка входа, маршрутизация по префиксам /api/...
СУБД	PostgreSQL 16 + расширение PostGIS — хранение профилей, событий, геоданных и матчей
Доступ к БД	Spring Data JDBC + Flyway (Java-сервисы), pgx (Go-сервисы)
Объектное хранилище	MinIO (S3-совместимое) — изображения событий и профилей
Аутентификация	Firebase Authentication — регистрация, вход, выдача ID Token; проверка JWT на backend
Картографический SDK	Yandex MapKit — отображение событий на карте

Продолжение таблицы 4.1

Область применения	Технологии и инструменты
Система контроля версий	Git
Контейнеризация	Docker + Docker Compose — развёртывание инфраструктуры и всех сервисов

4.1.1 Серверная часть

Серверная часть реализована в виде полиглот-микросервисов: доменная логика пользователей, событий и авторизации — на Java, узкоспециализированные сервисы мэтчинга и загрузки файлов — на Go. Такое разделение обусловлено разным характером нагрузки и требованиями к компонентам системы, а не выбором одного «универсального» языка для всего backend.

Java выбрана для сервисов auth-service, users-service и events-service по следующим причинам:

- предметная область включает разветвлённую бизнес-логику: профили, онбординг, модерация событий, участие, жалобы, ролевая модель (USER, MODERATOR, ADMIN) — Java и экосистема Spring Boot хорошо подходят для структурированного описания таких доменов;
- статическая типизация и зрелые средства валидации снижают число ошибок при работе со сложными моделями данных и REST-контрактами;
- встроенная интеграция с Flyway и Spring Data JDBC упрощает версионирование схем PostgreSQL по принципу schema-per-service;
- Java 21 и Spring Boot 3.3.x широко применяются в промышленной разработке, что облегчает сопровождение и развёртывание в Docker-контейнерах.

В таблице 4.2 приведено сравнение Java-фреймворков для REST-микросервисов. В качестве основного выбран Spring Boot.

Таблица 4.2 – Сравнение Java-фреймворков

Критерий	Spring Boot	Quarkus	Micronaut
Зрелость экосистемы	очень высокая	высокая	средняя
Интеграция JDBC + Flyway	нативная	нативная	нативная
Документирование REST (OpenAPI)	SpringDoc	SmallRye OpenAPI	OpenAPI
Время старта контейнера	среднее	низкое	низкое
Порог входа / объём документации	умеренный, обширная	выше	выше
Распространённость в enterprise	очень высокая	растущая	ограниченная

Go выбран для match-service и upload-service по следующим причинам:

- оба сервиса выполняют узкий набор операций с упором на быстрый приём HTTP-запросов: регистрация свайпа, выдача списков мэтчей, запись файла в MinIO — без сложной многослойной доменной модели;
- компактный бинарный образ контейнера и короткое время запуска снижают ресурсозатраты при локальном развёртывании через Docker Compose;
- стандартная библиотека и простая модель конкурентности (goroutines) удобны для I/O-операций — обращений к PostgreSQL и MinIO;
- общий пакет shared/pkg позволяет переиспользовать код проверки Firebase JWT между Go-сервисами без дублирования логики на Java.

В таблице 4.3 приведено сравнение Go-фреймворков для HTTP API. Выбран Gin как наиболее распространённое и документированное решение для REST-сервисов в экосистеме Go.

Таблица 4.3 — Сравнение Go-фреймворков

Критерий	Gin	Echo	Fiber
Производительность (routing)	высокая	высокая	очень высокая
Распространённость / community	очень высокая	высокая	высокая
Размер зависимостей	минимальный	минимальный	минимальный
Middleware и группировка маршрутов	есть	есть	есть
Совместимость с net/http	полная	полная	ограниченная (fasthttp)
Документация и примеры	обширная	хорошая	хорошая

4.1.1.1 Система управления базами данных

В таблице 4.4 приведено сравнение кандидатов на роль СУБД. В качестве системы управления базами данных используется PostgreSQL 16 с расширением PostGIS:

- предметная область требует хранения координат событий и пользователей с эффективным поиском «рядом» (функции ST_DWithin, geography-типы);
- связи «пользователь — событие — участник — матч» естественно описываются реляционной моделью с внешними ключами и транзакциями;
- JSONB-поля (например, socialLinks в профиле) дополняют структурированные данные без перехода на document-store;
- все микросервисы используют одну физическую базу andexevents с изоляцией по схемам (users, events, auth, public).

Таблица 4.4 – Сравнение промышленных СУБД

Критерий	PostgreSQL 16 + PostGIS	MySQL 8	MongoDB 7
Реляционная целостность (FK, ACID)	полная	полная	ограниченная
Геопространственные запросы	PostGIS (ST_DWithin и др.)	ограниченные	GeoJSON-индексы
JSON-поля	JSONB + GIN-индексы	JSON	BSON (нативный)
Расширяемость (PostGIS, pgcrypto)	высокая	средняя	низкая
Зрелость в production	очень высокая	очень высокая	высокая

4.1.1.2 Доступ к базе данных

В таблице 4.5 приведено сравнение средств доступа к данным. Для Java-сервисов применены Spring Data JDBC и Flyway, для Go-сервисов — драйвер pgx:

- Flyway обеспечивает версионирование SQL-миграций при старте каждого Java-сервиса и соответствует принципу schema-per-service;
- Spring Data JDBC даёт прямой контроль над SQL-запросами без overhead полноценного JPA/Hibernate — это важно для PostGIS-запросов и сложных выборок;
- pgx в Go-сервисах обеспечивает нативную работу с PostgreSQL, пул соединений и низкую задержку при коротких транзакциях (свайп, запись метаданных файла).

Таблица 4.5 – Средства доступа к PostgreSQL

Критерий	Spring Data JDBC + Flyway	Hibernate (JPA)	pgx (Go)	database/sql + lib/pq
Контроль над SQL	полный	частичный (JPQL)	полный	полный
Миграции схемы	Flyway (встроено)	Liquibase/Flyway	внешние/SQL в геро	внешние
PostGIS-запросы	прямой SQL	через native query	прямой SQL	прямой SQL
Overhead ORM	минимальный	высокий	нет	нет
Применение в проекте	Java-сервисы	не используется	match-, upload-service	не используется

4.1.1.3 Объектное хранилище

В таблице 4.6 приведено сравнение средств хранения медиафайлов. Для изображений событий и профилей выбран MinIO с отдельным upload-service на Go:

- S3-совместимый API позволяет разворачивать хранилище локально в Docker без привязки к облачному vendor lock-in;
- upload-service принимает multipart-загрузку, записывает объект в бакет и возвращает публичный URL по префиксу /uploads/..., проксируемому через Traefik;
- отделение медиа от основной БД снижает нагрузку на PostgreSQL и упрощает масштабирование статического контента.

Таблица 4.6 – Сравнение объектных хранилищ

Критерий	MinIO	Firebase Storage	Локальная файловая система
S3-совместимый API	да	нет	нет
Локальное развёртывание в Docker	да	нет	да
Публичные URL для клиента	через upload-service	через Firebase SDK	требуется отдельного прокси
Масштабирование	горизонтальное	managed (Google Cloud)	ограниченное
Независимость от Firebase	да	нет	да

4.1.1.4 Аутентификация

В таблице 4.7 приведено сравнение средств аутентификации. Выбран Firebase Authentication:

- мобильный клиент на Flutter имеет официальный SDK Firebase для регистрации, входа по email/паролю, входа через Google и восстановления пароля;
- backend не хранит пароли — достаточно проверить подпись Firebase ID Token (JWKS на Java, общий пакет на Go) и сопоставить firebaseUid с записью в таблице "User";
- это сокращает объём собственного кода безопасности и снижает риски ошибок при реализации хранения учётных данных.

Таблица 4.6 – Сравнение средств аутентификации

Критерий	Firebase Auth	Собственный JWT + bcrypt	Keycloak
Готовые провайдеры (Google, email)	да	требуется реализации	да
Нагрузка на backend	минимальная (проверка токена)	высокая	средняя
Сложность развёртывания	низкая (SaaS)	средняя	высокая
Интеграция с Flutter SDK	нативная	через REST	через OIDC
Хранение паролей на backend	не требуется	требуется	делегировается

4.1.2 Пользовательская часть

В таблице 4.8 приведено сравнение технологий мобильной разработки. Для реализации клиентской части выбран Flutter по следующим причинам:

- одна кодовая база на Dart для целевых платформ дипломного проекта (Android и iOS);
- высокая производительность отрисовки интерфейса за счёт движка Skia;
- зрелая поддержка Material Design и декларативного построения экранов;
- наличие официальных и community-пакетов для Firebase, HTTP-клиента, BLoC и Yandex MapKit;
- возможность задать базовый URL API через `--dart-define=API_BASE_URL` при отладке на физическом устройстве.

Таблица 4.8 – Сравнение средств мобильной разработки

Критерий	Flutter	React Native	Kotlin / Swift (натив)
Единая кодовая база (Android + iOS)	да	да	нет
Производительность UI	высокая (Skia)	bridge-зависимая	максимальная
Интеграция с Firebase	официальный SDK	community-модули	отдельная на каждой платформе
Интеграция с Yandex MapKit	через плагины	через native-модули	нативная
Скорость разработки MVP	высокая	высокая	ниже

Управление состоянием выполняется с помощью паттерна BLoC (flutter_bloc): выделены блоки AuthBloc, EventBloc, ProfileBloc, что обеспечивает предсказуемый поток данных между UI и REST API.

Сетевое взаимодействие — через пакет http; авторизационный заголовок Authorization: Bearer <firebase-id-token> добавляется централизованно при обращении к защищённым эндпоинтам backend.

4.2 Реализация серверной части

4.2.1 Реализация целевой архитектуры

Серверная часть системы построена по микросервисной архитектуре с единой точкой входа через API-шлюз Traefik. Каждый сервис инкапсулирует отдельную предметную область, имеет собственный Docker-контейнер и может масштабироваться независимо. Общая инфраструктура (PostgreSQL, MinIO, Traefik) поднимается через docker-compose.yml.

Таблица 4.9 — Компоненты подсистемы и их характеристика

№	Компонент подсистемы	Краткая характеристика
1	Подсистема маршрутизации (Traefik)	Приём HTTP-запросов на порт 80, маршрутизация по префиксу пути к соответствующему контейнеру
2	auth-service (Java)	Валидация Firebase ID Token, получение текущего пользователя, привязка UID к внутреннему userId
3	users-service (Java)	CRUD профиля, онбординг, геолокация, лента кандидатов для матчинга, жалобы, административные операции
4	events-service (Java)	Создание и модерация событий, участие, рейтинги, геопоиск, управление участниками
5	match-service (Go)	Свайпы (like/dislike/super-like), взаимные мэтчи, входящие лайки
6	upload-service (Go)	Загрузка изображений в MinIO, выдача публичных URL
7	PostgreSQL + PostGIS	Единое хранилище данных; схемы users, events, auth, таблица "Match" в public
8	MinIO	Объектное хранилище медиафайлов
9	Firebase (внешний)	Аутентификация пользователей, FCM-токены

4.2.2 Структура базы данных

Данные хранятся в одной базе andexevents. Изоляция достигается разделением по схемам; имена таблиц с заглавной буквы ("User", "Event", "Public") сохранены для совместимости с исторической моделью.

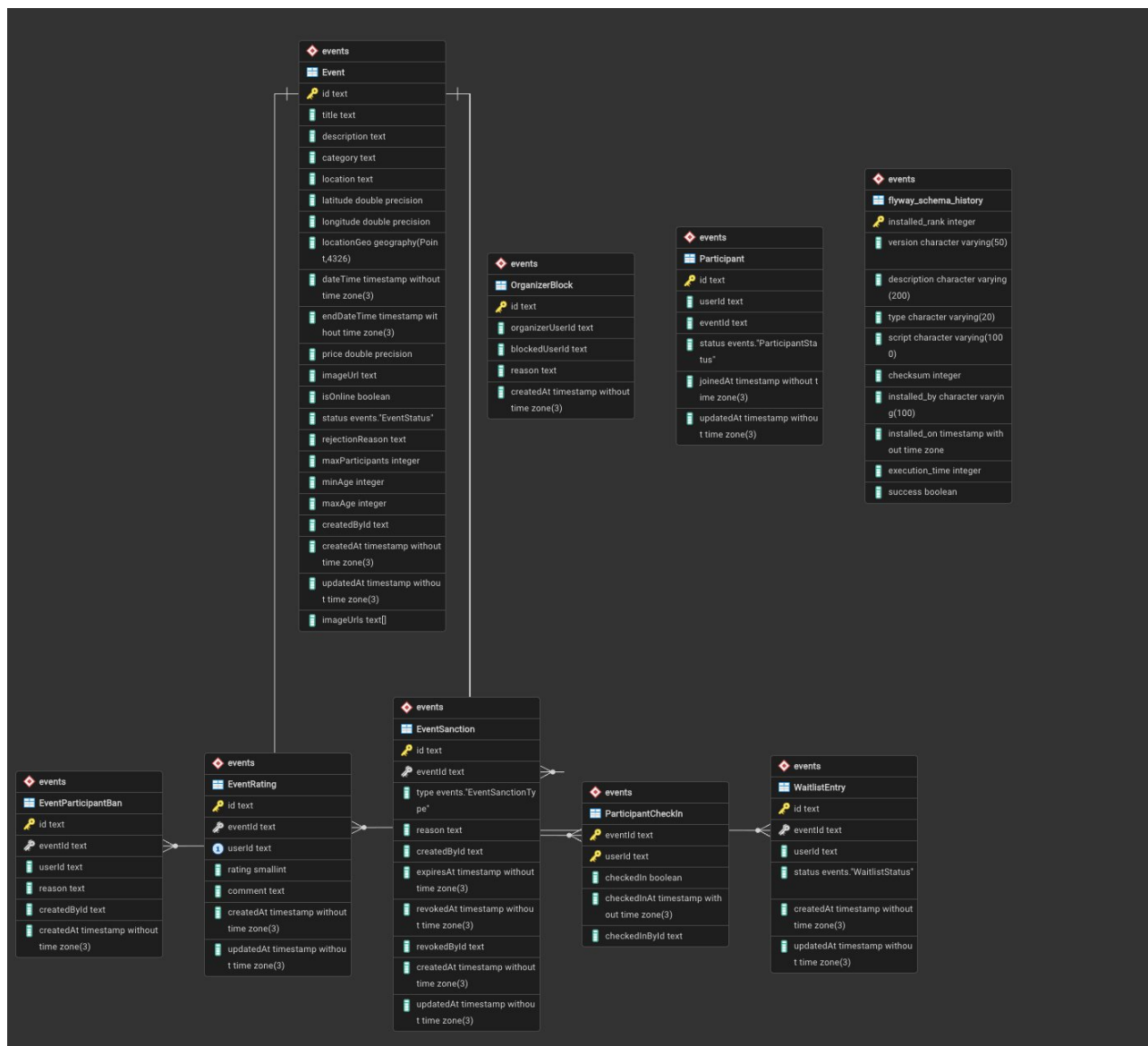


Рисунок 4.1 — Структура “Events”

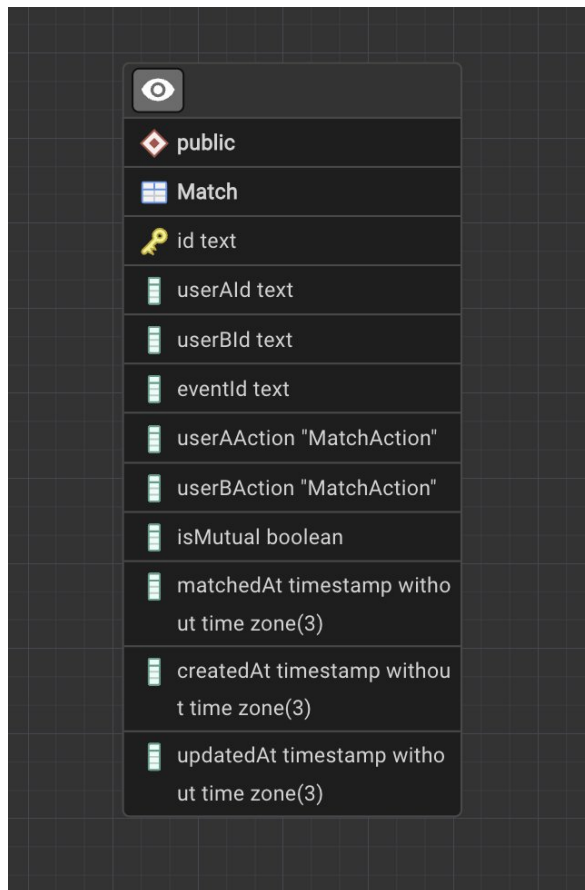


Рисунок 4.2 — Структура “Events”

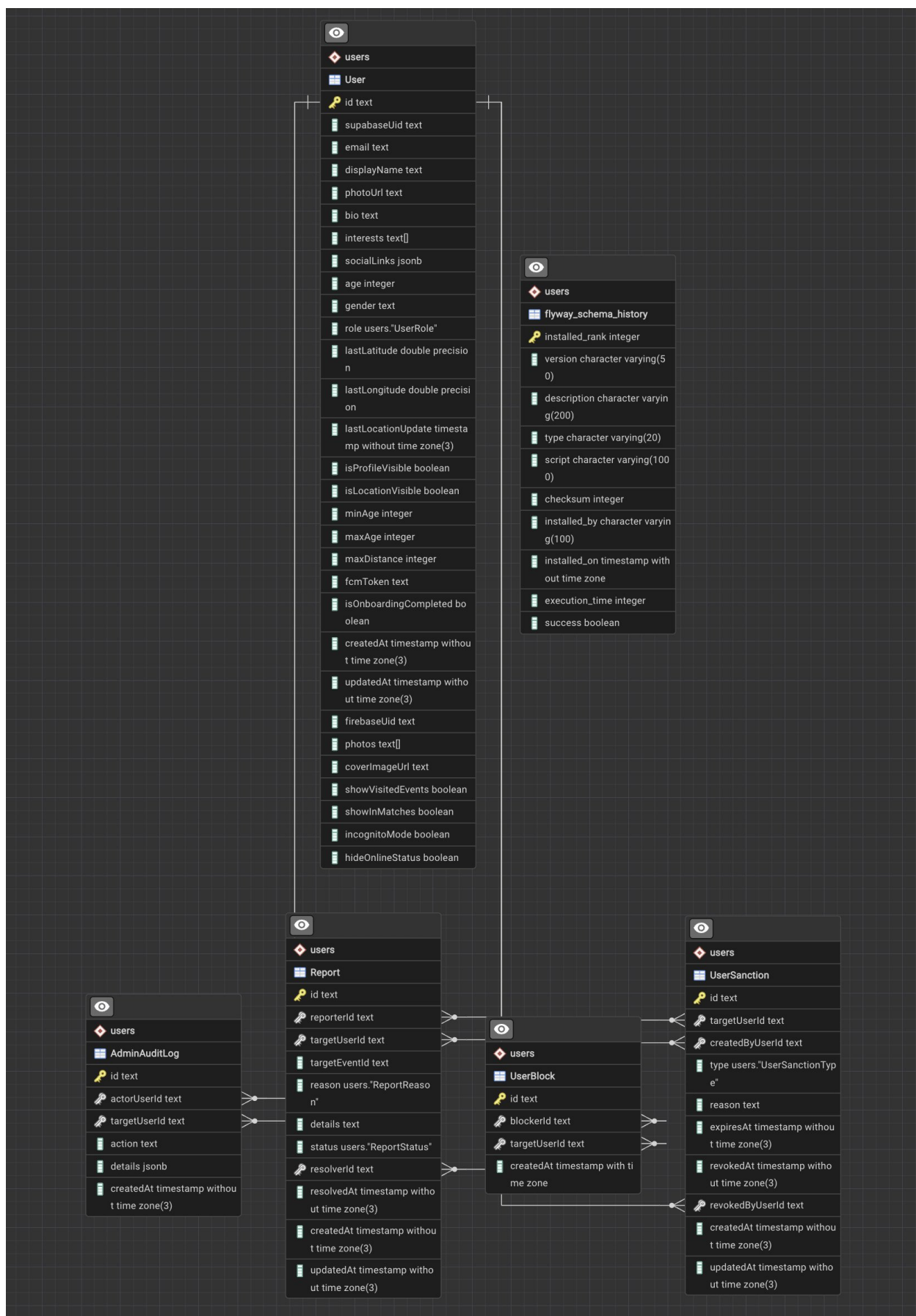


Рисунок 4.3 — Структура “Users”

4.2.2.1 Таблица user (Схема “Users”)

Таблица предназначена для хранения профилей пользователей приложения.

Основные поля:

- id — уникальный идентификатор пользователя (TEXT, первичный ключ);
- firebaseUid — идентификатор учётной записи Firebase (уникальный индекс);
- email — адрес электронной почты (уникальный);
- displayName, photoUrl, bio — отображаемые данные профиля;
- interests — массив интересов (TEXT[]);
- age, gender — демографические параметры для подбора;
- role — роль пользователя (перечисление UserRole: USER, MODERATOR, ADMIN);
- lastLatitude, lastLongitude, lastLocationUpdate — последняя известная геопозиция;
- isProfileVisible, isLocationVisible, minAge, maxAge, maxDistance — настройки приватности и фильтров матчинга;
- fcmToken — токен для push-уведомлений;
- isOnboardingCompleted — признак завершения первичной настройки;
- createdAt, updatedAt — метки времени.

Таблица связана с "Report", "UserBlock", "UserSanction", "AdminAuditLog" и таблицей "Match" (через userAId / userBId).

4.2.2.2 Таблица event (Схема “Events”)

Таблица предназначена для хранения мероприятий.

Основные поля:

- id — уникальный идентификатор события (TEXT, первичный ключ);
- title, description, category — название, описание и категория;
- location — текстовый адрес или описание места;
- latitude, longitude, locationGeo — координаты и geography-точка PostGIS;
- dateTime, endDateTime — время начала и окончания;
- price — стоимость участия (0 — бесплатно);
- imageUrl — URL обложки (MinIO);
- isOnline — признак онлайн-формата;
- status — статус модерации (EventStatus: PENDING, APPROVED, REJECTED);
- rejectionReason — причина отклонения (при наличии);
- maxParticipants, minAge, maxAge — ограничения участия;
- createdById — идентификатор создателя;
- createdAt, updatedAt — метки времени.

Таблица связана с "Participant", "EventRating", "EventSanction" и записями матчинга по событию.

4.2.2.3 Таблица participant (Схема “events”)

Таблица фиксирует участие пользователей в мероприятиях.

Основные поля:

- id — первичный ключ;

- userId, eventId — составной уникальный индекс (один пользователь — одна запись на событие);
- status — статус участия (ParticipantStatus: INTERESTED, GOING);
- joinedAt, updatedAt — метки времени.

4.2.2.4 Таблица match (Схема “public”)

Таблица хранит действия матчинга и взаимные совпадения между пользователями.

Основные поля:

- id — первичный ключ (UUID);
- userId, userBId — пара пользователей (внешние ключи на "User");
- eventId — необязательная привязка к событию, в контексте которого произошёл свайп;
- userAAction, userBAction — действия (MatchAction: LIKE, DISLIKE, SUPER_LIKE);
- isMutual — признак взаимного мэтча;
- matchedAt — время взаимного совпадения;
- createdAt, updatedAt — метки времени.

Уникальный индекс по паре пользователей и событию предотвращает дублирование записей.

4.2.2.5 Таблица Report

Таблица предназначена для хранения жалоб пользователей на других пользователей или события.

Основные поля:

- id — первичный ключ;
- reporterId — автор жалобы;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
99

- targetUserId, targetEventId — объект жалобы;
- reason — причина (ReportReason: SPAM, INAPPROPRIATE_CONTENT, HARASSMENT, FAKE_EVENT, OTHER);
- details — текстовое пояснение;
- status — статус рассмотрения (PENDING, RESOLVED, DISMISSED);
- resolverId, resolvedAt — данные модератора при закрытии жалобы.

4.2.2.6 Вспомогательные таблицы

Дополнительно в схемах users и events реализованы:

- "UserBlock" — блокировки между пользователями;
- "UserSanction", "EventSanction" — санкции администратора;
- "AdminAuditLog" — журнал административных действий;
- "EventRating" — оценки мероприятий;
- "EventParticipantBan", "OrganizerBlock", "WaitlistEntry", "ParticipantCheckIn" — расширенное управление участниками.

Миграции применяются автоматически при старте Java-сервисов (Flyway); инициализация таблицы "Match" выполняется скриптом postgres-init при первом запуске контейнера PostgreSQL.

4.2.3 Сервисная архитектура и процессы

Сервисный уровень реализует основную прикладную логику системы: приём HTTP-запросов от мобильного клиента, проверку авторизации, выполнение бизнес-правил, чтение и запись данных в PostgreSQL, взаимодействие с MinIO и (при необходимости) отправку push-уведомлений. Каждый сервис развёрнут как отдельный Docker-контейнер, слушает собственный порт и регистрируется в Traefik через labels; снаружи клиент

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
100

обращается только к единому базовому URL шлюза (<http://localhost/api/...> или production-домен).

Общие принципы реализации на сервисном уровне:

- REST + JSON — все контракты между клиентом и backend описаны HTTP-методами и телами запросов в формате JSON;
- Firebase ID Token — на защищённых маршрутах клиент передаёт заголовок `Authorization: Bearer <token>`; сервис извлекает токен, проверяет подпись по JWKS Firebase и получает внутренний `userId` из таблицы `users."User"`;
- Единый формат ответа — Java-сервисы возвращают обёртку `ApiResponse (success, data, message)`; Go-сервисы используют согласованные JSON-структуры для успеха и ошибок;
- Доступ к БД — Java: Spring Data JDBC + репозитории с явным SQL; Go: пул соединений `pgx`; обращения к таблицам выполняются с указанием схемы (`users."User"`, `events."Event"`, `public."Match"`);
- Изоляция доменов — `users-service` владеет схемой `users`, `events-service` — `events`, `match-service` работает с `public."Match"`, `auth-service` не хранит бизнес-данные, `upload-service` пишет файлы в MinIO и при необходимости обновляет URL в `users."User"`.

Взаимодействие сервисов с клиентом и инфраструктурой можно представить так: Flutter-приложение Traefik один из пяти микросервисов PostgreSQL и/или MinIO. Прямых HTTP-вызовов между микросервисами в текущей реализации нет: общие данные согласуются через общую базу (например, `events-service` читает роль пользователя из `users."User"` при проверке прав модератора).

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
101

4.2.3.1 Сервис auth-service

Сервис auth-service отвечает за проверку подлинности пользователя и предоставление сведений о текущей учётной записи. Он не хранит пароли и не ведёт бизнес-таблиц профиля — регистрация и вход выполняются в Firebase Authentication, а backend проверяет JWT и сопоставляет идентификатор Firebase с записью в users."User".

Сервис выполняет следующие функции:

- извлечение и проверка Bearer-токена из заголовка запроса;
- верификация подписи JWT по публичным ключам Google (JWKS);
- поиск внутреннего идентификатора пользователя в таблице users."User";
- возврат клиенту UID, email и userId через эндпоинты /api/auth/validate и /api/auth/me.

Сервис построен на Spring Boot и размещён за префиксом Traefik /api/auth. Он взаимодействует с users-service опосредованно: профиль создаётся и обновляется в users-service, а auth-service лишь подтверждает, что токен действителен и кому он принадлежит.

4.2.3.2 Сервис users-service

Сервис users-service является центральным компонентом домена «пользователь» и реализует жизненный цикл профиля в схеме users PostgreSQL.

Сервис обеспечивает:

- создание и обновление профиля после регистрации в Firebase;
- прохождение онбординга (интересы, возраст, пол, флаг завершения настройки);

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
102

- сохранение и изменение геолокации для подбора событий и кандидатов «рядом»;
- формирование ленты кандидатов для свайпов с учётом расстояния, интересов, возрастных фильтров и блокировок;
- управление фотографиями профиля и настройками приватности;
- подачу и обработку жалоб на пользователей и события;
- административные операции для ролей MODERATOR и ADMIN (смена ролей, санкции, журнал аудита).

Сервис построен на Spring Boot с доступом к данным через JDBC и миграции Flyway. Для пространственных запросов используется PostGIS. Лента кандидатов отдаётся клиенту для вкладки «Матчи», а сами действия like/dislike обрабатывает match-service.

4.2.3.3 Сервис events-service

Сервис events-service реализует домен «мероприятия» и работает со схемой events базы данных.

Сервис включает следующие функции:

- создание, редактирование и удаление событий с привязкой к организатору;
- хранение координат и геоточки PostGIS для отображения на карте и поиска в радиусе;
- выдачу списка и карточки мероприятия с фильтрацией по статусу модерации (APPROVED для публичной афиши);
- регистрацию участия пользователя (INTERESTED / GOING);
- модерацию событий и наложение санкций администратором;
- сбор оценок и отзывов после посещения;
- инструменты организатора: управление участниками, лист ожидания, check-in, блокировка пользователей.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
103

Сервис реализован на Spring Boot, данные событий и участников хранятся в таблицах events."Event", events."Participant" и связанных сущностях. При проверке прав модератора сервис обращается к роли пользователя в users."User".

4.2.3.4 Сервис match-service

Сервис match-service отвечает за механику знакомств в контексте событий: фиксацию свайпов и определение взаимных совпадений. Данные хранятся в таблице public."Match".

Сервис обеспечивает:

- регистрацию действий like, dislike и super-like между парой пользователей;
- необязательную привязку действия к событию через поле eventId;
- создание или обновление записи в "Match" и вычисление признака взаимного мэтча (isMutual);
- выдачу списка взаимных мэтчей и входящих лайков;
- отправку push-уведомления при взаимном совпадении (частичная реализация через FCM).

Сервис реализован на Go с фреймворком Gin и драйвером pgx. Подбор кандидатов для отображения в ленте выполняет users-service; match-service обрабатывает только решение пользователя после просмотра карточки. Такое разделение упрощает независимое развитие алгоритма рекомендаций и логики свайпов.

4.2.3.5 Сервис upload-service

Сервис upload-service обеспечивает загрузку и отдачу изображений профилей и событий. Файлы сохраняются в MinIO (S3-совместимое

хранилище), а клиент получает публичный URL для отображения в приложении.

Функциональность сервиса включает:

- приём файла через multipart-запрос с проверкой Firebase-токена;
- валидацию типа и размера изображения;
- запись объекта в бакет MinIO (avatars, events, photos);
- обновление ссылки на фото в профиле пользователя при загрузке аватара или галереи;
- отдачу файла по маршруту /uploads/... без повторной авторизации;
- удаление изображения из хранилища и профиля по запросу клиента.

Сервис реализован на Go/Gin. Traefik проксирует как API загрузки (/api/upload), так и статическую раздачу (/uploads). URL формируется с учётом настроек прокси или переменной UPLOADS_PUBLIC_BASE_URL, чтобы ссылки были корректны при обращении с мобильного устройства.

Таким образом, upload-service выполняет роль медиа-шлюза между клиентом и MinIO, не дублируя бизнес-логику users-service и events-service, которые лишь сохраняют полученные ссылки в своих сущностях.

4.2.4 Подсистема маршрутизации

Traefik получает правила маршрутизации из Docker labels контейнеров. Префикс пути на входе шлюза совпадает с префиксом, который ожидают приложения.

Таблица 4.10 — Маршрутизация запросов через Traefik

Префикс пути	Сервис	Порт контейнера	Назначение
/api/auth	auth- service	8083	Валидация токена, текущий пользователь

Продолжение таблицы 4.10

Префикс пути	Сервис	Порт контейнера	Назначение
/api/users	users-service	8081	Профили, онбординг, геолокация, лента, жалобы
/api/events	events-service	8082	CRUD событий, участие, модерация
/api/matches	match-service	8005	Свайпы, мэтчи, входящие лайки
/api/upload, /uploads	upload-service	8006	Загрузка и отдача изображений

Таблица 4.11 — Основные группы REST-эндпоинтов auth-service и users-service

Метод	Путь	Назначение
POST	/api/auth/validate	Проверка Firebase JWT
GET	/api/auth/me	Текущий пользователь
POST	/api/users	Создание профиля
GET	/api/users/me	Получение своего профиля
PUT	/api/users/me	Обновление профиля
POST	/api/users/me/onboarding	Завершение онбординга
PUT	/api/users/me/location	Обновление координат
GET	/api/users/matches	Лента кандидатов
POST	/api/users/reports	Жалоба на пользователя/событие

Таблица 4.12 — Основные группы REST-эндпоинтов events-service и match-service

Метод	Путь	Назначение
GET	/api/events	Список событий (с фильтрами)
POST	/api/events	Создание события
GET	/api/events/{id}	Детали события
PUT	/api/events/{id}	Обновление события
POST	/api/events/{id}/participate	Участие в событии
GET	/api/events/nearby	События в bounding box (карта)
POST	/api/matches/swipe	Свайп (like/dislike/super-like)
GET	/api/matches/mutual	Взаимные мэтчи
GET	/api/matches/incoming	Входящие лайки

4.2.5 Безопасность серверной части

Защищённые маршруты требуют заголовок Authorization: Bearer <token>. Java-сервисы проверяют подпись Firebase JWT; Go-сервисы используют общий пакет верификации из shared/pkg. Ролевая модель (USER, MODERATOR, ADMIN) реализована на уровне users-service и events-service: административные операции доступны только пользователям с соответствующей ролью.

Секреты (ключи Firebase, строки подключения к БД, credentials MinIO) передаются через переменные окружения Docker Compose и не хранятся в исходном коде репозитория.

4.3 Реализация клиентского приложения пользователя

4.3.1 Общее описание

Пользовательская часть системы реализована в виде мобильного приложения на Flutter для платформ Android и iOS. Интерфейс построен по принципу вкладочной навигации с центральной кнопкой создания события и следует единому визуальному стилю.

Приложение интегрировано с backend через REST API по протоколу HTTPS (в production) или HTTP (локальная разработка через Traefik на порту 80). Авторизация выполняется через Firebase Authentication; полученный ID Token передаётся в каждом запросе к защищённым эндпоинтам.

Архитектура клиента разделена на слои:

- presentation — экраны (Widgets), BLoC-обработчики событий и состояний;
- data — HTTP-клиент, модели DTO, репозитории;
- core — конфигурация (AppConfig.baseUrl), тема, общие утилиты.

Базовый URL API определяется в AppConfig: в release-сборке используется `https://api.andexevents.com/api`, в debug — localhost или IP

машины разработчика (для физического устройства задаётся API_BASE_URL).

4.3.1.1 Экран приветствия

При первом запуске пользователь видит краткое знакомство с возможностями приложения — поиск событий на карте и в афише, знакомства с единомышленниками, создание собственных мероприятий.

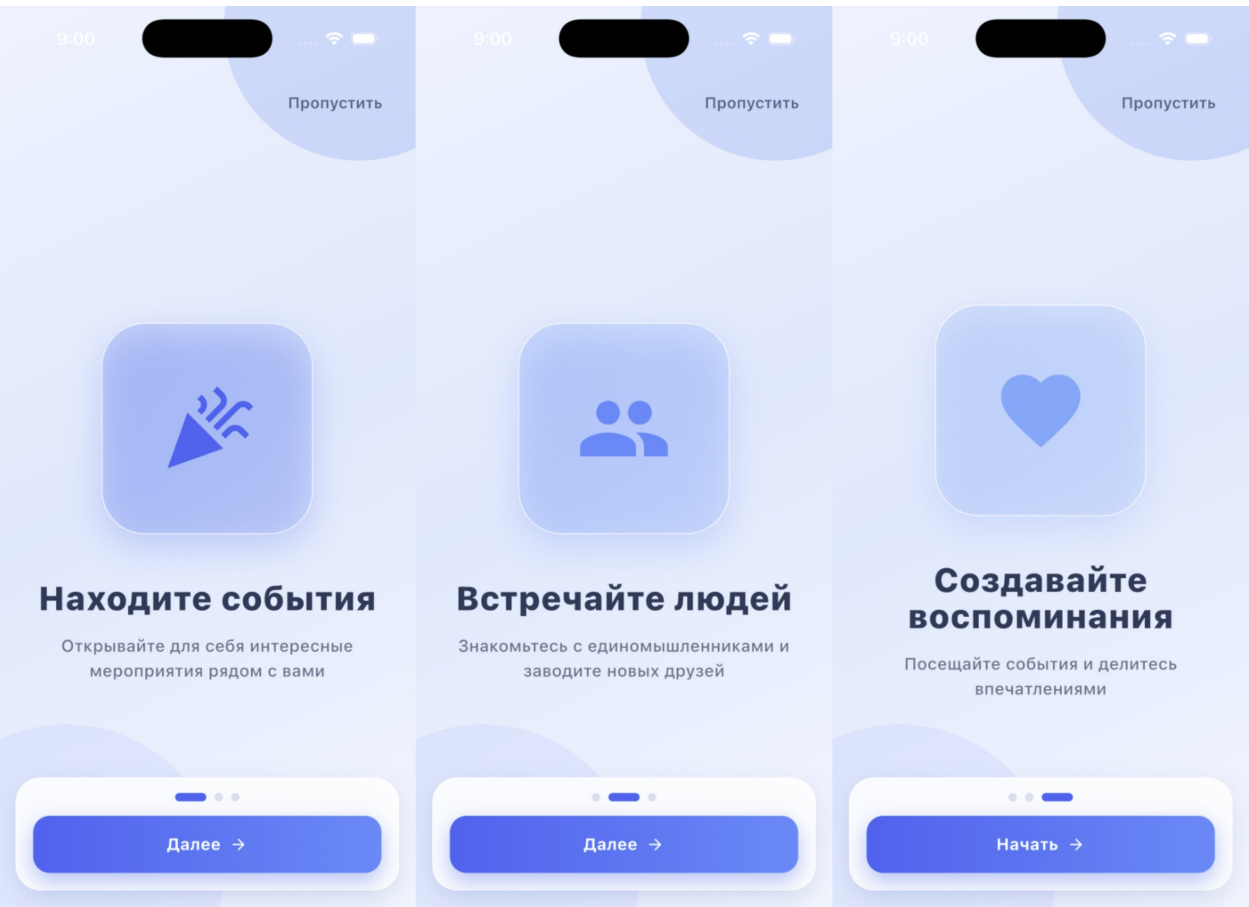


Рисунок 4.3 – Экраны приветствия

4.3.1.2 Экран авторизации

Экран входа содержит формы авторизации по email и паролю, регистрации нового аккаунта, входа через Google и восстановления пароля. При регистрации выполняется проверка корректности email и надёжности пароля на стороне Firebase. После успешного входа приложение запрашивает профиль у backend; при отсутствии записи создаётся новый профиль.

Так же при регистрации открывается экран подтверждения почты если email не подтверждён, пользователь перенаправляется на экран верификации с возможностью повторной отправки письма.

На рисунке 4.5 представлен экран авторизации. На рисунках 4.6, 4.7 — экраны регистрации.

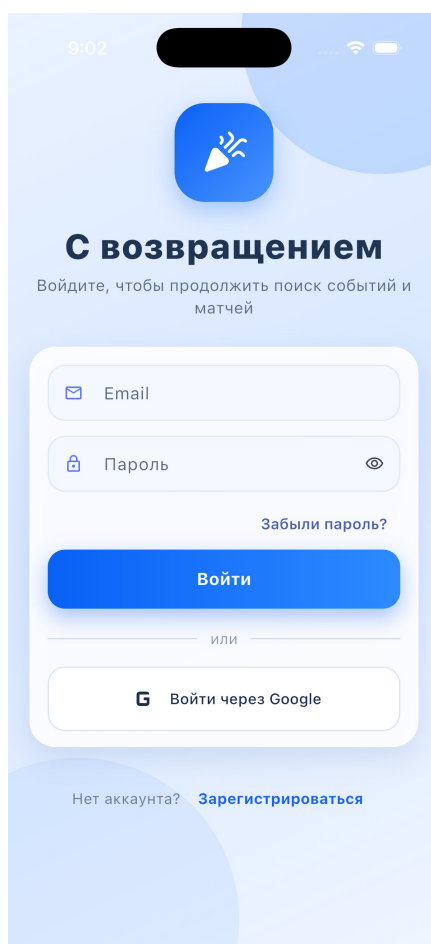


Рисунок 4.5 — Экран авторизации

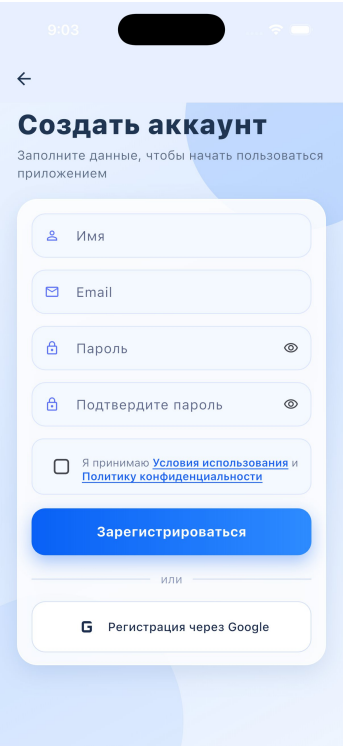


Рисунок 4.6 — Экран регистрации

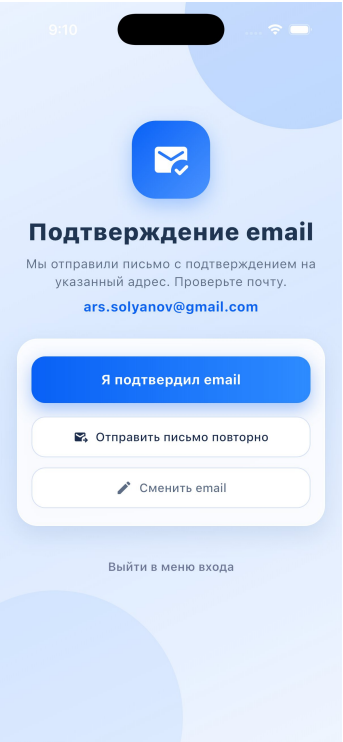


Рисунок 4.7 — Экран подтверждения почты

4.3.1.3 Экраны онбординга

После авторизации, если профиль не настроен, открывается онбординг из трёх шагов: базовые данные (возраст, пол, фото), выбор интересов (не менее трёх), разрешение геолокации.

На рисунке 4.8. представлен экраны онбординга.

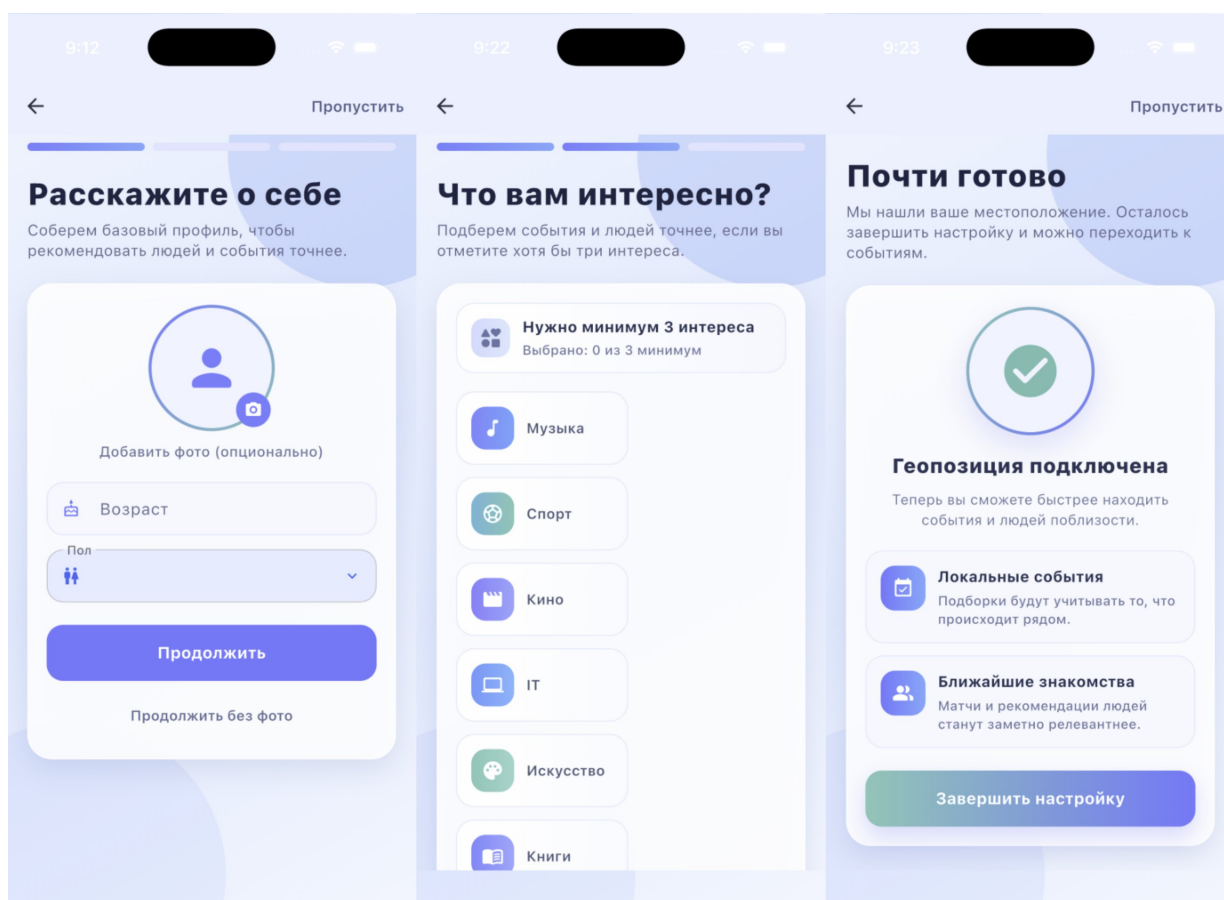


Рисунок 4.8 – Экраны онбординга

4.3.1.4 Главный экран «Карта»

После завершения онбординга открывается главный экран с нижней панелью навигации из четырёх вкладок и центральной кнопкой «+»:

- Карта — события на интерактивной карте Yandex MapKit;
- Афиша — список мероприятий с фильтрами;

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
111

- Матчи — свайпы и список взаимных совпадений;
- Профиль — данные пользователя и настройки.

Центральная кнопка открывает форму создания нового события независимо от текущей вкладки.

Вкладка «Карта» отображает одобренные мероприятия маркерами в видимой области. При перемещении или изменении масштаба карты приложение запрашивает у backend события в текущем bounding box. Выбор маркера или элемента в нижней панели открывает краткую карточку; переход к полной информации — на экран деталей события.

Доступен поиск по названию. Для работы карты используется API-ключ Yandex MapKit, задаваемый при сборке приложения.

На рисунке 4.9. представлен главный экран.

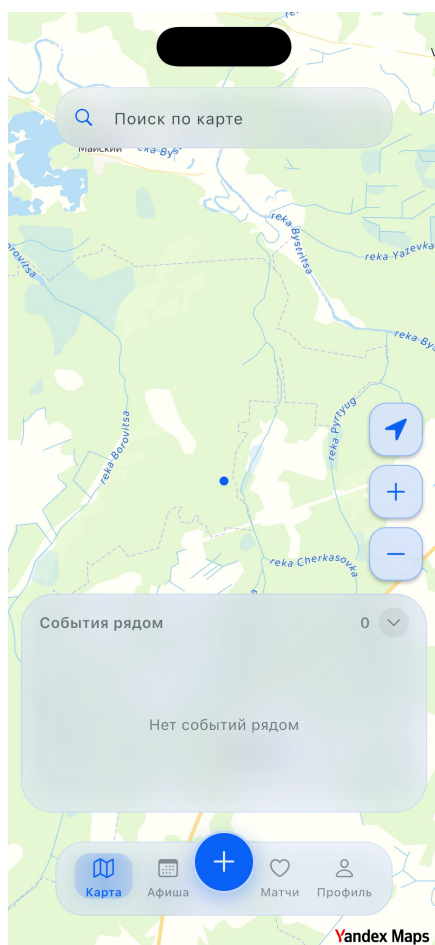


Рисунок 4.9 — Главный экран

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
112

4.3.1.5 Экран «Афиша»

Вкладка «Афиша» показывает список мероприятий в виде прокручиваемых карточек. Пользователь может фильтровать события по городу, дате, категории, цене и формату (онлайн / офлайн), а также искать по названию. Фильтрация части параметров выполняется на клиенте после загрузки списка APPROVED-событий.

Pull-to-refresh обновляет данные с сервера. Нажатие на карточку открывает экран деталей.

На рисунке 4.10 представлен экран афиши с фильтрами.

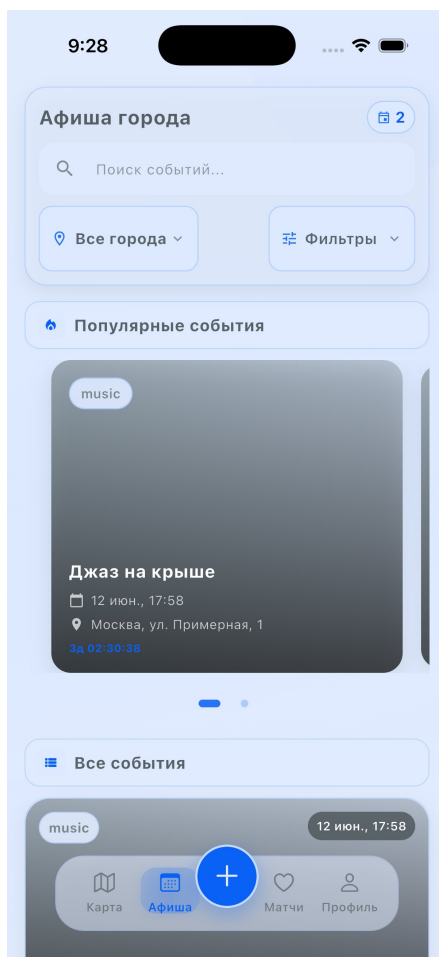


Рисунок 4.10 – Экран «Афиша»

4.3.1.6 Экраны создания события

Форма создания мероприятия доступна через центральную кнопку «+». Пользователь указывает название, описание, категорию, дату и время, место (с выбором точки на карте или вводом адреса), стоимость, формат (онлайн/офлайн), загружает обложку через upload-service.

После отправки событие сохраняется на backend. В текущей реализации новые события могут создаваться сразу со статусом APPROVED (в зависимости от конфигурации сервиса); администратор retains возможность отклонения и санкций.

На рисунке 4.11 представлен экраны создания события.

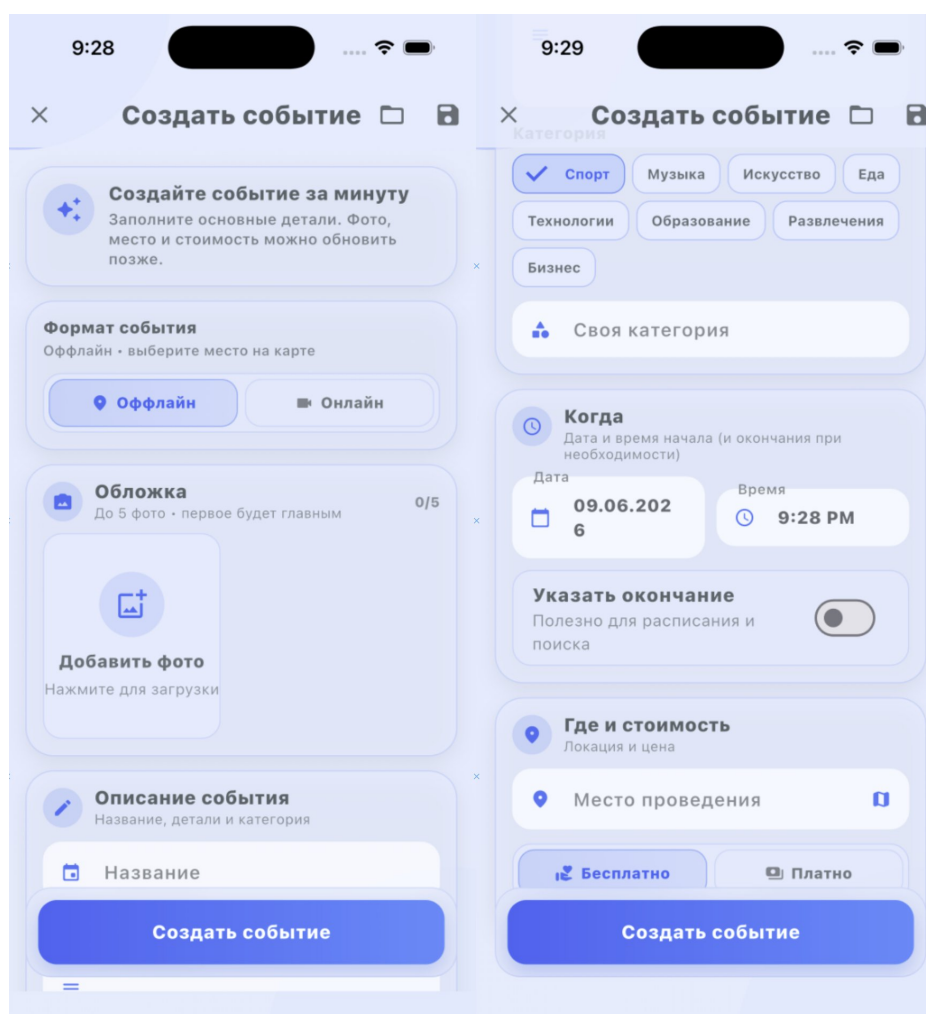


Рисунок 4.11 — Экран создания события

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

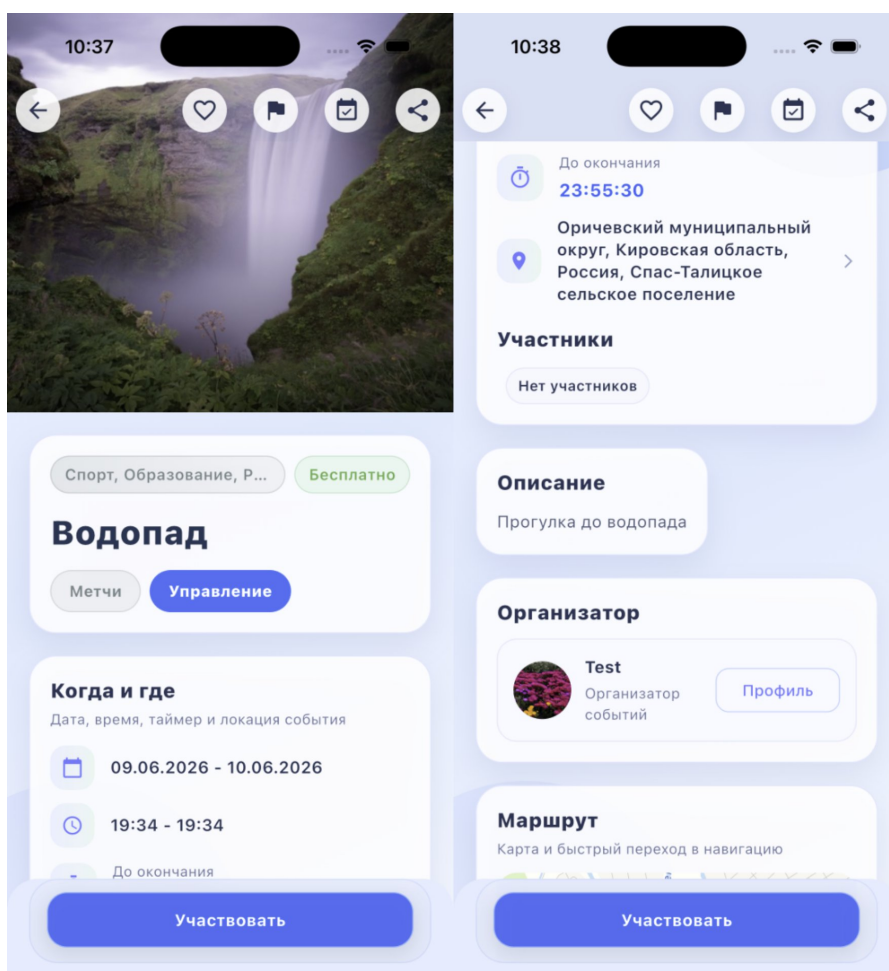
Лист
114

4.3.1.7 Экраны деталей события

Экран содержит полную информацию о мероприятии: обложку, описание, место и время, цену, рейтинг, список участников. Пользователь может отметить интерес или подтвердить участие (INTERESTED / GOING), оценить событие после посещения, пожаловаться на мероприятие или перейти к профилю организатора.

Организатор и администратор видят дополнительные элементы управления (редактирование, модерация участников).

На рисунке 4.12 представлен экраны деталей события.



Экран 4.12 – Экраны деталей события

4.3.1.8 Экран мэтчей

Вкладка «Матчи» объединяет механику свайпов и просмотр результатов знакомства. В режиме ленты пользователь просматривает карточки кандидатов (с учётом геолокации и интересов), выполняет like, dislike или super-like; действие отправляется в match-service. При взаимном лайке формируется мэтч.

Отдельные подразделы показывают список взаимных мэтчей и входящих лайков. Из карточки мэтча можно перейти к профилю собеседника.

На рисунке 4.13 представлен экран свайпов. На рисунке 4.14 предоставлен экран список взаимных мэтчей.

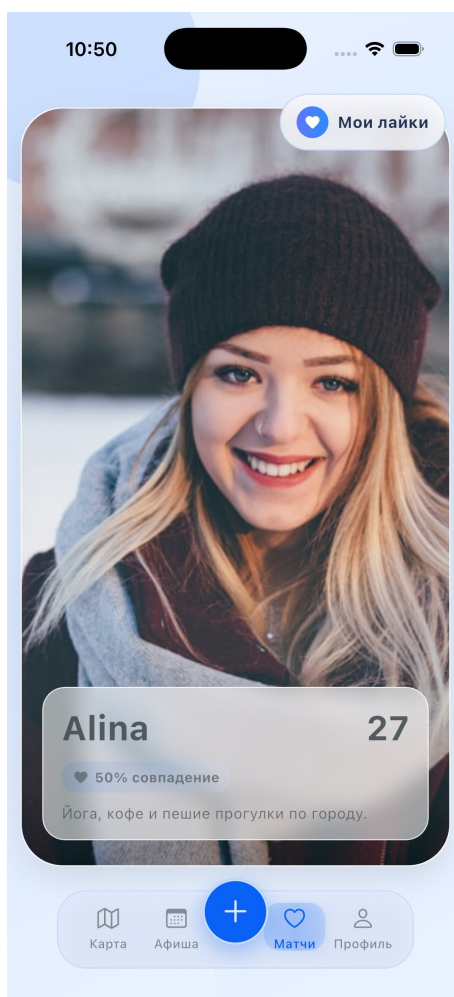


Рисунок 4.13 — Экран свайпов

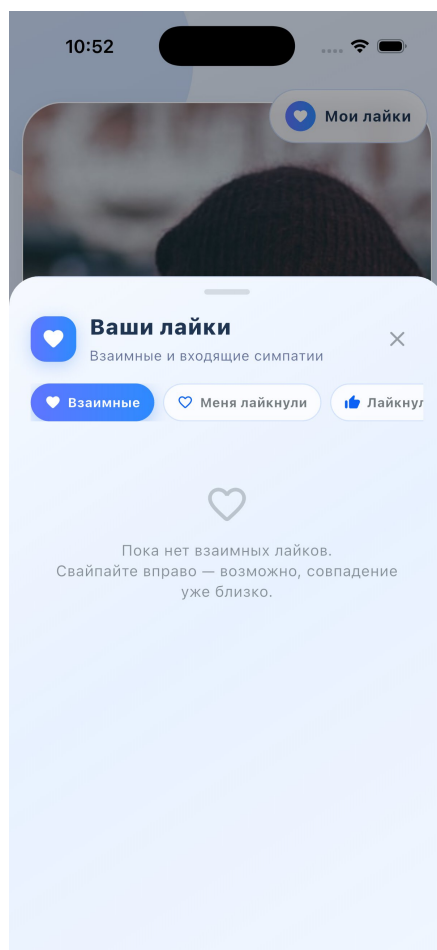


Рисунок 4.14 — Экран взаимных мэтчей

4.3.1.9 Экран профиля

Вкладка «Профиль» отображает данные текущего пользователя: имя, фото, биографию, интересы, настройки приватности (видимость профиля и геолокации, возрастной диапазон и радиус поиска для матчинга). Доступны редактирование профиля, смена фото, выход из аккаунта, отправка жалобы на другого пользователя.

Push-уведомления: клиент регистрирует FCM-токен и передаёт его в users-service; доставка уведомлений о новых мэтчах реализована частично и может быть расширена.

На рисунке 4.15 представлен экран профиля. На рисунке 4.16 — экран редактирования профиля. На рисунке 4.17 — настройки приватности.

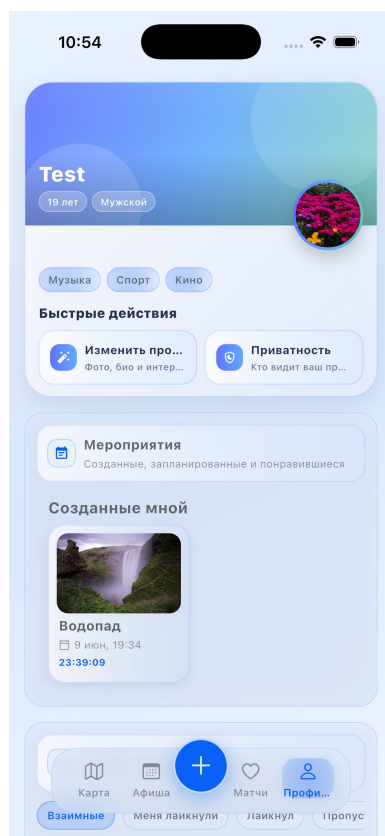


Рисунок 4.15 – Экран профиля

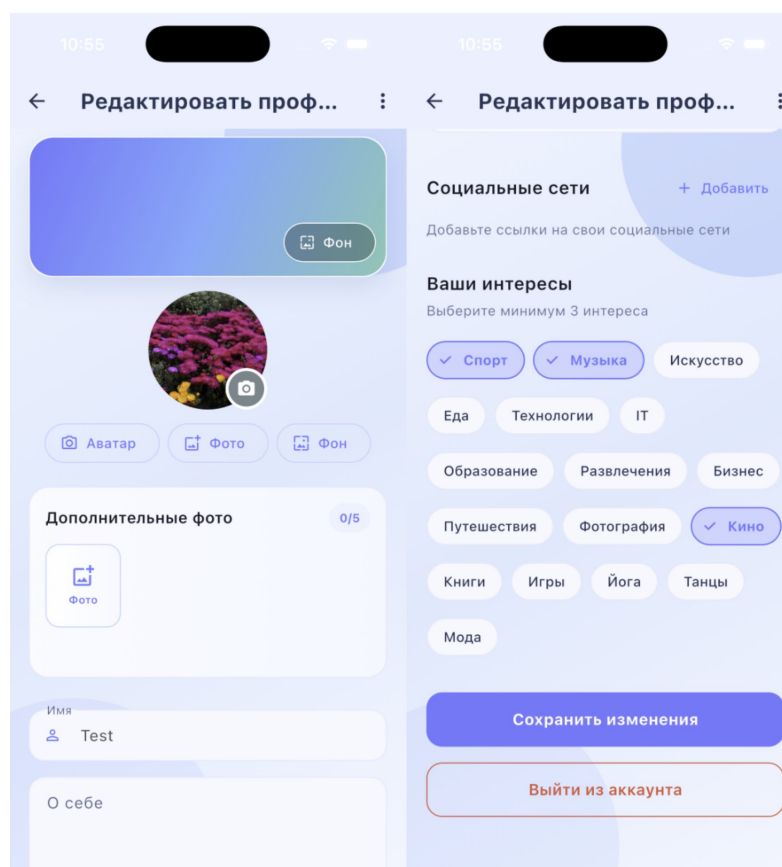


Рисунок 4.16 – Экран редактирования профиля

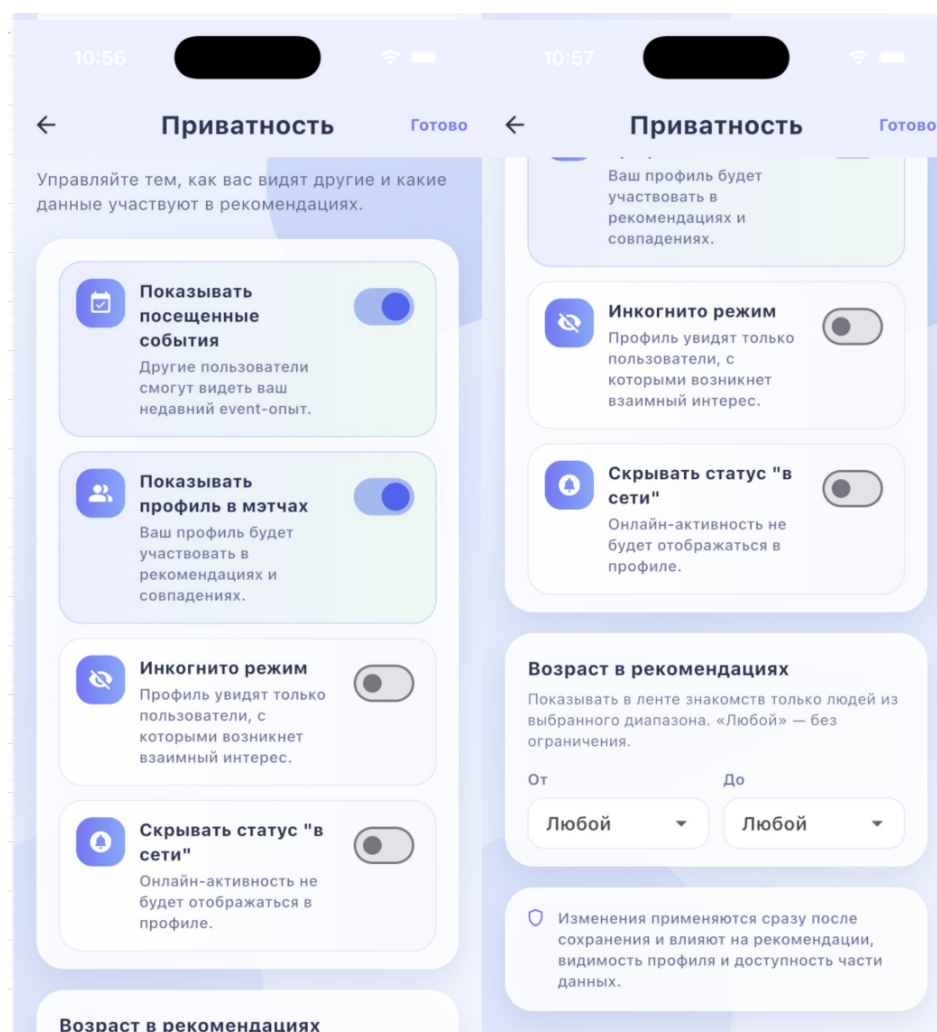


Рисунок 4.17 – Экран настроек приватности

4.3.1.10 Административная панель

Для пользователей с ролью ADMIN или MODERATOR в приложении доступны административные экраны: просмотр и обработка жалоб, модерация событий (одобрение, отклонение), управление санкциями, просмотр журнала аудита. Интерфейс администратора встроен в мобильное приложение и использует те же REST API, что и пользовательские экраны, с проверкой роли на backend.

На рисунке 4.18 представлен экран административной панели.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
119

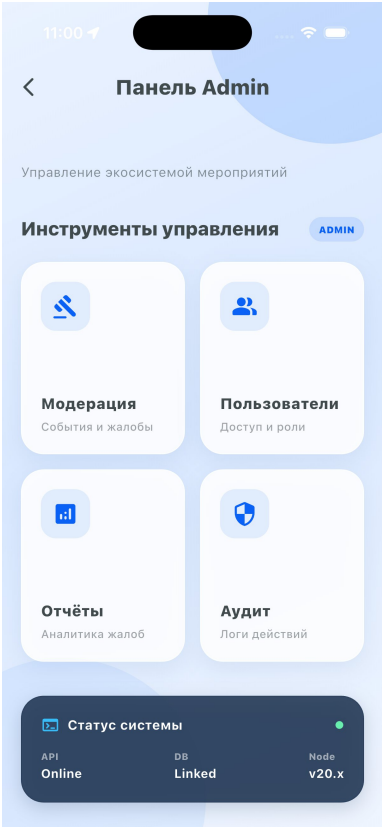


Рисунок 4.18 — Административная панель

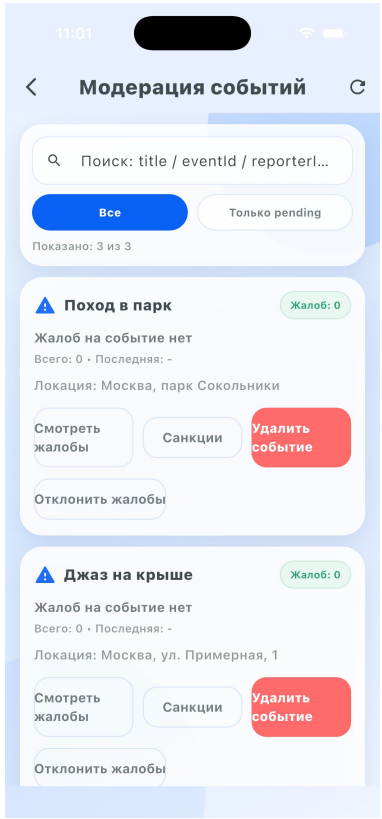


Рисунок 4.19 — Модерация событий

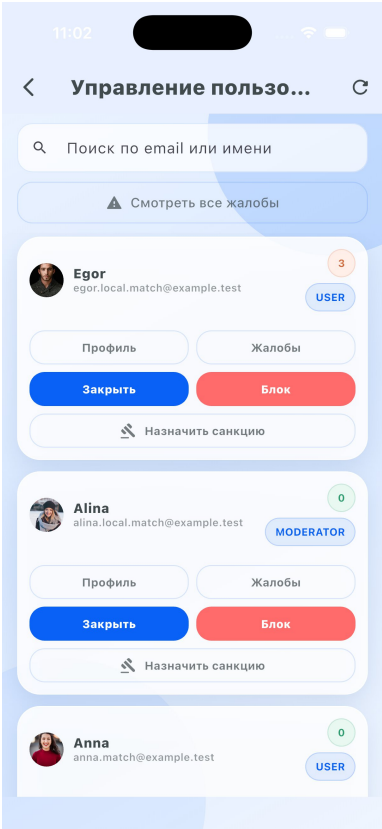


Рисунок 4.20 — Управление пользователями

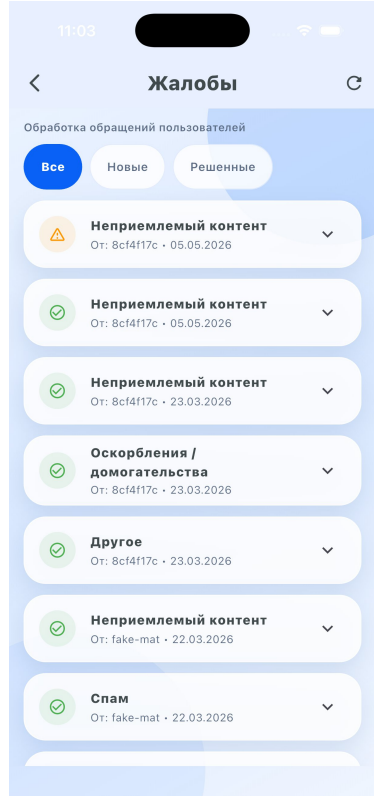


Рисунок 4.21 — Жалобы

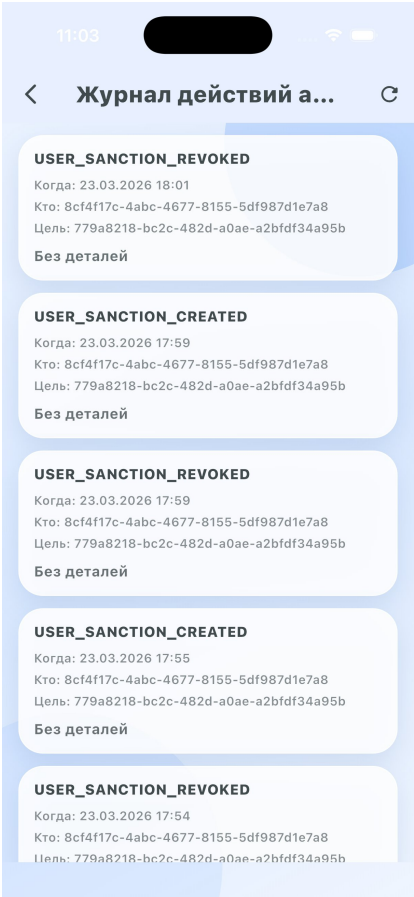


Рисунок 4.22 — Журнал действий

4.4 Вывод по реализации программного продукта

В результате разработки создана информационная система мобильного приложения геолокационной социальной сети, предназначенная для поиска городских мероприятий, участия в них и социального знакомства пользователей в контексте общих интересов и геолокации.

Серверная часть построена на базе микросервисной архитектуры с пятью независимыми сервисами (три на Java/Spring Boot, два на Go/Gin), единым API-шлюзом Traefik и СУБД PostgreSQL 16 с PostGIS. Такое решение обеспечило чёткое разделение ответственности между доменами «пользователи», «события», «матчинг» и «медиа», а также возможность независимого развёртывания и масштабирования компонентов. Аутентификация делегирована Firebase Authentication, что сократило объём собственного кода безопасности. Изображения хранятся в MinIO через выделенный upload-service.

Пользовательская часть представлена кроссплатформенным мобильным приложением на Flutter с управлением состоянием через BLoC. Реализованы ключевые экраны: авторизация и онбординг, главная навигация с вкладками «Карта», «Афиша», «Матчи» и «Профиль», создание и просмотр событий, механика свайпов и мэтчей, профиль пользователя и административные функции. Интеграция с Yandex MapKit обеспечивает наглядное отображение мероприятий на карте города.

Таким образом, программный продукт соответствует техническому заданию дипломного проекта: реализован полный цикл работы пользователя — от регистрации и настройки профиля до поиска событий, участия в них и знакомства с другими участниками. Архитектура системы обеспечивает основу для дальнейшего развития: расширения push-уведомлений, уточнения алгоритма подбора матчей, выделения отдельного Web- или Admin-клиента и масштабирования сервисов под растущую нагрузку.

Изм.	Лист	№ докум.	Подпись	Дата

090207.193754.ПЗ

Лист
123

ЗАКЛЮЧЕНИЕ

В рамках дипломного проекта на тему «Разработка мобильного приложения геолокационной социальной сети» был разработан программный комплекс, обеспечивающий полный цикл взаимодействия пользователей с сервисом: от регистрации и настройки профиля до поиска мероприятий по геолокации, участия в событиях и социального знакомства посредством механизма мэтчинга.

В ходе выполнения работы были получены следующие результаты:

- проведён анализ предметной области и исследованы существующие решения в области геолокационных социальных сервисов и приложений для организации городских мероприятий. Выявленные недостатки аналогов, такие как разрозненность функций поиска событий и знакомств, ограниченное использование геолокационных возможностей и сложность масштабирования монолитных решений, подтвердили актуальность разработки собственного программного комплекса, объединяющего данные функции в рамках единой платформы;
- сформулированы функциональные и нефункциональные требования к системе, определены роли пользователей и описаны основные пользовательские сценарии. Полученные результаты легли в основу проектирования архитектуры и функциональных возможностей приложения;
- разработана слабосвязная микросервисная архитектура клиент-серверного типа. Выполнено проектирование основных компонентов системы, построены диаграммы вариантов использования, потоков данных и модель базы данных, включающая сущности пользователей, событий, участия, совпадений, жалоб и санкций. Для хранения пространственных данных предусмотрено использование PostgreSQL с расширением PostGIS;
- реализована серверная часть приложения на основе микросервисного подхода. Архитектура включает отдельные сервисы

управления пользователями, событиями, аутентификацией, матчингом и медиаконтентом, объединённые посредством API-шлюза и единой инфраструктуры хранения данных;

- реализована система аутентификации пользователей с использованием Firebase Authentication. Разработаны механизмы регистрации, управления профилем, обновления геопозиции, формирования рекомендаций для знакомств, а также обработки действий пользователей в процессе матчинга с последующим формированием взаимных совпадений;

- разработана подсистема загрузки и хранения медиаконтента, обеспечивающая сохранение изображений профилей и событий с возможностью дальнейшего использования в пользовательском интерфейсе;

- реализовано кроссплатформенное мобильное приложение, включающее основные пользовательские сценарии: авторизацию, онбординг, поиск мероприятий на карте, просмотр ленты событий, участие в мероприятиях, систему знакомств на основе свайпов, управление профилем пользователя и административные функции модерации контента;

- выполнено контейнеризированное развёртывание программного комплекса, обеспечивающее воспроизводимость среды разработки и упрощающее дальнейшее сопровождение и масштабирование системы.

Практическая значимость разработанного решения заключается в создании единой платформы для поиска мероприятий, организации социального взаимодействия пользователей и установления новых знакомств на основе географической близости и общих интересов. Разработанный программный комплекс демонстрирует применение современных подходов к построению распределённых информационных систем, использующих микросервисную архитектуру, мобильные технологии и пространственную обработку данных.

Перспективными направлениями дальнейшего развития проекта являются расширение подсистемы уведомлений, внедрение дополнительных

средств мониторинга и журналирования, совершенствование алгоритмов формирования рекомендаций для знакомств, повышение уровня автоматизации тестирования, разработка веб-интерфейса для администрирования и управления системой а также улучшение безопасности пользователей путем внедрения подтверждения личности пользователя.

Таким образом, все задачи, поставленные во введении выпускной квалификационной работы, выполнены в полном объёме. Разработанное мобильное приложение геолокационной социальной сети соответствует сформулированным требованиям, обладает расширяемой архитектурой и может служить основой для дальнейшего развития и внедрения в практическую эксплуатацию.

					090207.193754.ПЗ	Лист
						126
Изм.	Лист	№ докум.	Подпись	Дата		

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) **Абдуллин, Р. Р.** Разработка мобильных приложений на языке Dart с использованием фреймворка Flutter / Р. Р. Абдуллин. — Москва : Финансы и статистика, 2023. — 240 с.
- 2) **Беннетт, Дж.** Java. Полное руководство / Дж. Беннетт ; пер. с англ. И. В. Берштейна. — 12-е изд. — Санкт-Петербург : Питер, 2025. — 1120 с.
- 3) **Браун, Э.** Go. Язык программирования / Э. Браун, А. Донован ; пер. с англ. А. А. Слинкина. — Москва : Вильямс, 2022. — 480 с.
- 4) **Вильямс, М.** Docker в действии / М. Вильямс ; пер. с англ. С. А. Чернышова. — 2-е изд. — Санкт-Петербург : БХВ-Петербург, 2024. — 432 с.
- 5) **Гонсалес, Л.** Проектирование геолокационных сервисов: от теории к практике / Л. Гонсалес, Р. Мартинес ; пер. с англ. О. В. Ефимовой. — Москва : ДМК Пресс, 2023. — 320 с.
- 6) **Джонсон, К.** Микросервисы и контейнеризация. Практическое руководство по Docker и Kubernetes / К. Джонсон ; пер. с англ. А. Н. Кузнецова. — Москва : Эксмо, 2024. — 384 с.
- 7) **Иванов, А. М.** Социальные сети: архитектура и разработка / А. М. Иванов, Е. В. Соколова. — Казань : Изд-во Казанского ун-та, 2022. — 216 с.
- 8) **Козлов, Д. В.** Технологии Go в веб-разработке / Д. В. Козлов. — Новосибирск : Сибирское университетское изд-во, 2023. — 198 с.
- 9) **Кузнецов, С. А.** Flutter: создание кроссплатформенных приложений / С. А. Кузнецов. — Москва : Лаборатория знаний, 2024. — 304 с.
- 10) **Линч, Т.** Go в действии / Т. Линч ; пер. с англ. А. В. Гусевой. — Санкт-Петербург : Питер, 2023. — 352 с.
- 11) **Марков, П. А.** Разработка мобильных приложений с использованием картографических сервисов / П. А. Марков. — Екатеринбург : Уральский федеральный ун-т, 2022. — 176 с.

12) **Нестеров, И. В.** Инфраструктура микросервисов: Traefik, Docker и оркестрация / И. В. Нестеров. — Москва : Техносфера, 2024. — 288 с.

13) **Николаев, Д. С.** Backend-разработка на Java для социальных сетей / Д. С. Николаев, О. В. Петрова. — Ростов-на-Дону : Феникс, 2023. — 260 с.

14) **Сидоров, В. Е.** Геолокационные технологии в мобильных приложениях / В. Е. Сидоров. — Москва : Горячая линия – Телеком, 2023. — 192 с.

15) **Шмидт, М.** Java Spring Boot: практическое руководство по созданию веб-приложений / М. Шмидт ; пер. с англ. Д. А. Климова. — Москва : Прогресс книга, 2024. — 416 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Список сокращений

API — Application Programming Interface — интерфейс программирования приложений.

BLoC — Business Logic Component — компонент бизнес-логики (паттерн управления состоянием во Flutter).

CI/CD — Continuous Integration / Continuous Delivery — непрерывная интеграция и непрерывная доставка.

DFD — Data Flow Diagram — диаграмма потоков данных.

ER — Entity-Relationship — «сущность–связь» (модель данных).

FCM — Firebase Cloud Messaging — облачный сервис push-уведомлений Firebase.

GIS — Geographic Information System — геоинформационная система.

GPS — Global Positioning System — глобальная система позиционирования.

HTTP — HyperText Transfer Protocol — протокол передачи гипертекста.

HTTPS — HyperText Transfer Protocol Secure — защищённый протокол передачи гипертекста.

IDEF0 — Integration DEFinition for Function Modeling — методика функционального моделирования.

IDEF1X — Integration DEFinition for Information Modeling — методика моделирования данных.

JSON — JavaScript Object Notation — текстовый формат обмена данными.

JWT — JSON Web Token — токен веб-аутентификации в формате JSON.

JWKS — JSON Web Key Set — набор открытых ключей для проверки подписи JWT.

ORM — Object-Relational Mapping — объектно-реляционное отображение.

PDF — Portable Document Format — универсальный формат электронных документов.

PostGIS — PostgreSQL Geographic Information System — геопространственное расширение СУБД PostgreSQL.

RBAC — Role-Based Access Control — ролевая модель управления доступом.

REST — Representational State Transfer — архитектурный стиль взаимодействия веб-сервисов.

RLS — Row-Level Security — строчная политика безопасности в базе данных.

S3 — Simple Storage Service — протокол объектного хранилища (S3-совместимый API, в проекте — MinIO).

SDK — Software Development Kit — комплект средств разработки программного обеспечения.

SQL — Structured Query Language — язык структурированных запросов.

TLS — Transport Layer Security — протокол защиты транспортного уровня.

UI — User Interface — пользовательский интерфейс.

UX — User Experience — пользовательский опыт.

UUID — Universally Unique Identifier — универсальный уникальный идентификатор.

WGS84 — World Geodetic System 1984 — мировая геодезическая система координат (используется в PostGIS).

ГИС — геоинформационная система.

ПО — программное обеспечение.

СУБД — система управления базами данных

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг серверной части

ПРИЛОЖЕНИЕ В

(обязательное)

Листинг пользовательской части